

O'REILLY®



The Agentic Enterprise

A Leader's Guide to Orchestrating,
Governing, and Scaling AI Agent Systems

Babak Hodjat &
Antoine Blondeau

Praise for *The Agentic Enterprise*

The emergence of enterprise AI agents represents a fundamental shift in how organizations develop, maintain, and improve their workflows, from deterministic applications to adaptive systems capable of autonomous execution. Realizing this opportunity requires far more than advances in foundation models; it demands new approaches to orchestration, governance, security, human oversight, and organizational transformation. *The Agentic Enterprise* captures this new paradigm with depth, combining technical rigor with operational insight. It is an important contribution to the global conversation on how enterprises can harness agentic AI to drive long-term competitiveness and innovation.

—Yann LeCun, Executive Chairman, Advanced Machine
Intelligence Labs

Former Chief AI Scientist, Meta
Jacob T. Schwartz Chaired Professor in Computer Science,
Courant Institute of Mathematical Sciences, New York
University

Founding Director of the NYU Center for Data Science
Recipient of the 2018 Association for Computing
Machinery (ACM) A. M. Turing Award

The real promise of agentic AI isn't smarter software, but a new model of work where humans and AI collaborate seamlessly to accomplish what neither could achieve alone. *The Agentic Enterprise* explores what it really takes to realize that vision, offering both the technical foundation and an organizational framework for humans and AI to work together effectively, responsibly, and at scale.

—Adam Cheyer, Cofounder of Siri

The Agentic Enterprise

A Leader's Guide to Orchestrating, Governing, and
Scaling AI Agent Systems

Babak Hodjat and Antoine Blondeau

O'REILLY®

The Agentic Enterprise

by Babak Hodjat and Antoine Blondeau

Copyright © 2026 Babak Hodjat, Antoine Blondeau. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: David Michelson

Development Editor: Shira Evans

Production Editor: Jonathon Owen

Copyeditor: Penelope Perkins

Proofreader: Rachel Rossi

Indexer: Judith McConville

Cover Designer: Susan Brown

Cover Illustrator: José Marzan Jr.

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

August 2026: First Edition

Revision History for the First Edition

- 2026-07-27: First Release

See <https://oreilly.com/catalog/errata.csp?isbn=9798341672512> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *The Agentic Enterprise*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-67251-2

[LSI]

Foreword

In September 2016, I finished writing *The Fall of the Human Empire: Memoirs of a Robot* (Bloomsbury Business). The narrator was an artificial intelligence named Lucie. She looked back from 2036 on the long road that had taken machines from the Dartmouth conference of 1956 to her own emergence as a superior intelligence. One chapter of that book was titled “2026: The Golden Age”. It described a world where machines spoke to other machines, where intelligent agents coordinated entire industrial networks, where humans had become the silent witnesses of conversations they had themselves set in motion.

I could not have written that book alone. Two of its most important contributors were Antoine Blondeau and Yann LeCun. Both were already convinced, years before the field caught up with them, that the trajectory of artificial intelligence would be defined not by isolated breakthroughs, but by systems of coordinated agents. At the time, friends found the 2036 timeline ambitious. A decade later, it looks almost conservative. The 2026 chapter, in particular, reads today less like fiction than like the executive summary of the book you are holding.

What I could not write then, because the engineering was not yet visible, is the chapter that Antoine and Babak Hodjat have now written. Lucie was a destination. *The Agentic Enterprise* is the architecture of the journey.

Antoine has been building toward this book his entire career, and he has not done it alone. His coauthor, Babak, is one of the rare people whose claim to the agentic idea predates the language we now use for it. Babak invented the agent-oriented natural language technology that became the foundation of Apple’s Siri, and he was already orchestrating specialized agents to map human intent into action in the 1990s. The two of them go back a long way: together with Adam Cheyer, they cofounded Sentient Technologies, one of the most ambitious distributed AI ventures of its generation. Long before “AI” became the term of art, both were serial entrepreneurs in what was then called algorithmic science, applying learning systems to language, behavior, and prediction at industrial scale. They understood early what most of the field only discovered after ChatGPT: the interesting question is

never how powerful a single model is, but how systems of them can be made to coordinate, learn, and act in the real world.

That is the central argument of this book, and it matters because the alternative is already failing in plain sight. Pilots multiply. Demos dazzle. Production deployments stall. The gap between what generative AI promises in a presentation and what it delivers in a real enterprise workflow is rarely a model problem. It is an architecture problem, a governance problem, a trust problem. Antoine and Babak have spent more years than almost anyone deploying multi-agent systems in operating businesses, from Allianz cutting claims settlement from thirty days to three, to Franklin Templeton scaling agents across investment workflows. They write from that ground, not from the slide.

I first met Antoine in 2015, over video calls between Paris and San Francisco, then in Hong Kong, then across most of the world. I was running a multi-billion consulting firm, trying to make sense of what was about to happen to our clients. He was already building multi-agent systems on both sides of the Pacific, in a world where almost no one used the words “agentic AI” yet. The conversation we started then has not really stopped. Over the past ten years, we have invested together, debated together, occasionally disagreed loudly, and ultimately cofounded AICapital.ai. Some partnerships endure because they are convenient. Ours has endured because Antoine has been right about technology, about timing, and about people far more often than most of us in this field.

Three things will strike senior leaders who open the book.

First, its intellectual honesty. Most AI books either oversell autonomy or retreat into theory. This one acknowledges what LLMs cannot yet do, where governance frameworks remain immature, where liability is unresolved. That honesty is what makes the rest of the book trustworthy.

Second, the bridge it builds. Agentic AI raises questions of strategy, organization, talent, and culture that are not strictly technical. But to address them credibly, executives need a solid understanding of how the technology

actually works. Without that, governance becomes performative and risk mitigation theater.

Third, the framework for what comes next. The book closes on the emergence of a World Wide Agentic Web, where autonomous agents broker, negotiate, and transact on behalf of humans and organizations. Markets, brands and intermediaries will be reshaped by buyers who are no longer human. The question becomes civilizational, not technical.

This is where Antoine and Babak's book connects with the questions I explored in *The Fall of the Human Empire*: what happens when increasingly autonomous intelligent systems begin to rival or exceed human capabilities and the balance of control between humans and machines starts to shift? The dominant actors of the last decade—both technological and geopolitical—have often operated in spaces where effective rules and institutions lag behind, turning that imbalance into a source of power. Agentic AI can entrench that dominance, or it can reopen the game. The orchestration layer, the governance protocols, and the trust frameworks described in these pages are what will decide which.

A word on tempo: AI is moving so fast that any static description of its state is outdated before it is printed. The authors knew this, and they made the right choice. They stayed above the daily noise, focusing on concepts and implications rather than on the model du jour. The model names will have changed in five years. The architectural principles in this book will not.

Lucie, in my novel, told her story from 2036. The chapters Antoine and Babak have written here are what will decide whether her version of the future is the one we end up living in, or another, one that we deliberately choose to engineer.

Charles-Edouard Bouée

Former CEO of Roland Berger; cofounder of AICapital.ai, Adagia, and Quartermaster.ai

***Author of The Fall of the Human Empire: Memoirs of a Robot
UAE, 2026***

Preface

Back in 1999, I got a phone call. My head of business development, Rudy, was in California and had an urgent message. “Hey, I met these folks here in San Jose, and they have something you ought to take a look at. It’s kind of a natural language-based command and control interface.”

Thus began my agentic AI journey.

And thus began my partnership with my fellow coauthor and dear friend, Babak Hodjat.

What Babak had imagined was, in fact, pretty much what Rudy had described—how to operate devices and simple applications by way of text-based or voice-based commands. He had come up with the idea by challenging himself with “Wouldn’t it be cool to talk to my TV?” And he had it working.

I flew across the Pacific from my then-home base in Hong Kong to meet with him and the team. Soon, I joined him and his startup, Dejima. The rest, including the DARPA project that led to the development of CALO (Cognitive Assistant that Learns and Organizes) and the Siri spin-off that was sold to Apple, as one says, is history.

Fast-forward more than 25 years, and history is in the making again. For the first time on our great humankind journey, we can envision digital, artificial beings whose capabilities equal and surpass those of humans themselves, and not in narrow, specialized domains anymore, but on a holistic basis—across our vast field of knowledge, and, increasingly, across our vast field of actions.

AI agents are a reality in 2026.

How is that possible? What happened since the days Babak and I were busy, and having a hard time, convincing the world that a machine could act upon natural language commands?

For a start, with the advent of ChatGPT in late 2022, natural language understanding became a solved problem. Gone were the days when one had to engineer ontologies to educate machines as to what a sentence actually meant. Through sheer brute force—essentially massive pattern recognizers, a lot of compute, and immense datasets—machines got to master language. No matter that no one had imagined this could happen so fast; the fact was that machines handled language as well as humans did, and in fact better than most humans did.

Since most of humans' actions are actually language-based (“Could you please make sure the updated budget is ready for next week’s board meeting and have finance send us the revised deck beforehand?”), it was only a matter of time until we figured the next logical step—plug large language models (LLMs) into machine command lines, or APIs (application programming interfaces) in order to start enabling language-based actions.

So we need the updated budget for next week’s board meeting? Well, Claude Cowork, OpenAI’s Codex, or Google’s Antigravity will:

1. identify and source the existing budget files,
2. read from email, Slack, or Microsoft Teams exchanges, determine what the updates are about, and identify who owns them,
3. if need be, ping the relevant department heads and finance department primes for these updated numbers (or automatically compute them based on either the revised documents shared between team members or the latest CRM and ERP applications-sourced numbers),
4. plug in the revised figures into the master spreadsheet,
5. make sure the numbers actually make sense (are we within the CEO/CFO/board-mandated guidelines? How are we tracking against budget?, etc.), and
6. format that into a presentable deck, with a headline commentary “please,” for circulation and consumption a week prior to the board

session.

That is no longer magic. We are at an inflection point where any and every human-designed workflow is game for agentic AI enablement.

The last few months have seen skyrocketing token consumption by LLMs, precisely because enterprises and individuals alike are discovering the incredible leverage that AI agents deliver.

Babak and I wrote this book because we believe the agentic AI revolution will be immensely consequential for us all. Given the enormity of the stakes—machines taking actions on our behalf, machines having agency, humans gaining in productivity in unprecedented ways, workflow and knowledge workers being augmented then replaced by machines, humans having to ensure that machines behave in compliant ways—we think it is of paramount importance to explain what is happening, to help guide our fellow humans towards maximizing the beneficial use of agentic AI, and to outline the pitfalls and help mitigate them by establishing relevant guardrails.

What This Book Is About

This book walks the reader through each important business and technological, as well as organizational and ethical, aspect of agentic AI in the enterprise.

It provides real-life examples of what works and what doesn't. It references the work done by maverick early adopters, including by the authors and the companies they work for or sponsor, as well as by more mature companies, as useful backgrounder to highlight the tried and tested methods that deliver substantial benefits.

What This Book Is Not

This book is not a step-by-step guide to leveraging agentic AI frameworks in order to deliver AI-enabled workflows. We outline a framework for organizations to approach agentic AI and get it right, but we are not writing prompts, nor are we walking you through an actual end-to-end implementation.

We are also not making the case for or against agentic AI from a philosophical or societal standpoint. We are not philosophers or sociologists, and although we of course harbor our own opinions and we discuss a few of the societal angles, we have the humility to leave the broader implications for the experts that be to outline and debate. This book is not a pamphlet.

Who This Book Is For

This book provides a comprehensive framework to assess the potential of agentic AI in the enterprise, enable it within one's organization, and prepare for, and work out, the consequences. Therefore, it will be useful to boards, CEOs, chief information officers, chief technology officers, and the rest of the C-suite, as well as decision-makers, business owners, functional leads, workflow designers and owners, architects, and developers.

Navigating This Book

We open with a foundational question: what is an AI agent, really? We trace the idea from Alan Turing's earliest writings through decades of AI research, all the way to the LLM-powered systems reshaping enterprise software today. The book establishes the shared language and conceptual framing that the chapters that follow build on.

Part I (Chapters 1–4) starts with the case for why this matters to your business right now. We walk through how enterprise AI has evolved, what the real-world evidence says about returns and risks, and what agentification looks like across industries, from insurance and financial services to software development, retail, telecoms, and health care. Readers primarily focused on strategy and investment decisions will find the most relevant material here.

Part II (Chapters 5–6) turns to the technical foundations—how agents are built, coordinated, fine-tuned, and designed into reliable systems. Architects, engineers, and technically curious leaders will want to engage with this part closely, though the concepts are introduced with enough context to be accessible to nonspecialists.

Part III (Chapters 7–8) takes the next step, addressing trust and scale—how to govern agentic systems responsibly, assign accountability, satisfy regulators, and expand agent networks without creating fragile or vendor-dependent architectures. This section is essential reading for risk, compliance, and technology leadership.

Finally, **Part IV (Chapter 5)** looks ahead at the near-term trajectory of multi-agent systems, the emergence of an agentic web, and the deeper questions about human agency and societal choices that autonomous AI forces us to confront.

While each chapter is designed to stand on its own and readers may navigate directly to the sections most relevant to their role, we have written this book with a single argument running through all four parts, and it is a more coherent and consequential story read in full.

We are in 2026, and the agentic AI journey onto which Babak and I embarked back in 1999, is still very much in its infancy. Many books will be written about AI agents. We hope this first one lays, in all humility, a foundation for subsequent writers to build on, as the journey has just begun.

Happy reading!

Antoine Blondeau

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

NOTE

This element signifies a general note.

O'Reilly Online Learning

NOTE

For more than 40 years, **O'Reilly Media** has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata and any additional information. You can access this page at <https://oreil.ly/the-agentic-enterprise>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

I would like to thank Adam Cheyer for his encouragement and camaraderie in those early days of dabbling into multi-agent systems, my coauthor and dear friend of 26 years, Antoine, who taught me so much about the practical and business side of AI, my Cognizant family, the **CAIL team** for their trust and support, and my brother Siamak, for coinventing much of AAOSA and CRUSE with me all those years ago. I would also like to thank my friend of 40 years, my AI pair programming buddy, Hormoz Shahrzad, who got me on this AI journey during that long walk we had together in our junior year.

Babak Hodjat

I would like to thank all my fellow travelers on this almost 30-year AI journey of mine: Wally Ritchie, Gary Kovacs, and my coauthor and longtime business partner Babak Hodjat, as well as Adam Cheyer, Siamak Hodjat, and Hormoz Shahrzad, the three of whom Babak thanked above (that's two commendations for the price of one!), Peter Harrigan, Nigel Duffy, Risto Miikkulainen, Tom Whittaker, Gurmeet Lamba, Yann LeCun, Charles-Edouard Bouée, and the many more scientists, developers, product managers, and business development, marketing, sales, finance, and HR folks I worked with over the years to further AI science and build my businesses on top of that. Trailblazing is a team effort, and I could not have done it without you.

The team at [AICapital.ai](#) is second to none in terms of its quality, depth, diversity and focus on performance, and I thank you all for letting me divert a chunk of my time to write this book.

Alain LeCouédic was instrumental in researching topics, helping craft the narrative, and project managing. This book would not exist without him.

Antoine Blondeau

The authors would like to thank the phenomenal team at O'Reilly who made all of this possible, chiefly among them our development editor Shira Evans, David Michelson, and Jonathon Owen. To our reviewers Eoin Woods, Urvashi Tyagi, Amir Banifatemi, thank you for bearing with us and with our writing. We truly appreciate your efforts to get this book into shape.

Introduction

Humans are agents. When we say “we have agency,” we mean we are able to postulate a goal of our own making, devise an action plan, and execute to deliver on that goal. As agents, we have the ability to organize and impact our world, get feedback from our actions, and modify our actions based on that feedback in order to improve.

Machine intelligence has improved such that it is taking a greater share of our decision making, complementing and augmenting our world as AI agents.

Like AI itself, and most other modern digital technology, we owe the concept of AI agents to Alan Turing. In his 1950 treatise *Computing Machinery and Intelligence*, he describes machines that perceive input and act toward goals. This definition is silent on the environment within which the agent is operating, and perhaps Turing’s idea was that the agent would be unconstrained to act in the world.

Today, the world in which an AI agent can operate has vastly expanded. Not only do we have a virtual world in the shape of the internet to complement the real world, we can also define any manner of virtual world we can imagine and engineer it into existence through programming.

So which world do AI agents operate in? Is it fair to assume that ultimately an AI agent is to be able to perceive any given environment, and act toward any given goal? Who defines the goals? Are agents autonomously to decide on which goals to pursue? While this is possible at lower levels, at some point in the hierarchy of agents there needs to be an uber-goal that directs the system and its highest order agents in how they define their goals.

Even if we borrow from nature and define the higher level goal to be survival, we, as humans, are still engineering this into the AI agents. Would we trust them then, to set themselves up to survive and let them loose to operate fully autonomously?

The same applies to inputs. Surely an AI agent cannot be perceiving the world in its entirety, and so we’d have to engineer sensors for it, designed

based on what we believe to be the most important information it would need in pursuit of its goals.

So AI agents are an engineering concept, and most of the engineering today falls on us humans.

A Brief History

Through the years, as computing, storage, and AI have progressed, this definition of an AI agent has been adopted, implemented, and experimented on in a variety of ways.

In the 1970s and 1980s, AI scientists started off thinking we could create a single entity that would be artificially intelligent with no need for constraints. This generally capable AI agent would primarily be able to operate in the real world, perceiving any input and acting toward any given goal. This ambitious definition failed, in part due to lack of compute and labeled data resources, and in part because the AI algorithms were simply not up to the job.

In the mid-1990s, the problem of AI was simplified by simplifying the world within which the intelligence would manifest itself. What if we build an intelligent agent, representing us, not in the complex world we live in, but in the simpler world of the internet, which back then, was mostly text and hyperlinks?

Thus, agent-based AI took center stage. A **popular AI textbook**, published in 1995, was called *Artificial Intelligence: A Modern Approach*, and the modern approach was this new, constrained agent-based definition of AI.

Before long, people started wondering what would happen if your agent ran into someone else's agent. Should they collaborate or compete? Would there be a **common inter-agent language**?

Multi-agent systems became a subfield of AI, with some, including the authors, suggesting a new software paradigm called agent-oriented software engineering¹ (**Figure I-1**) as a follow on to the object-oriented movement.

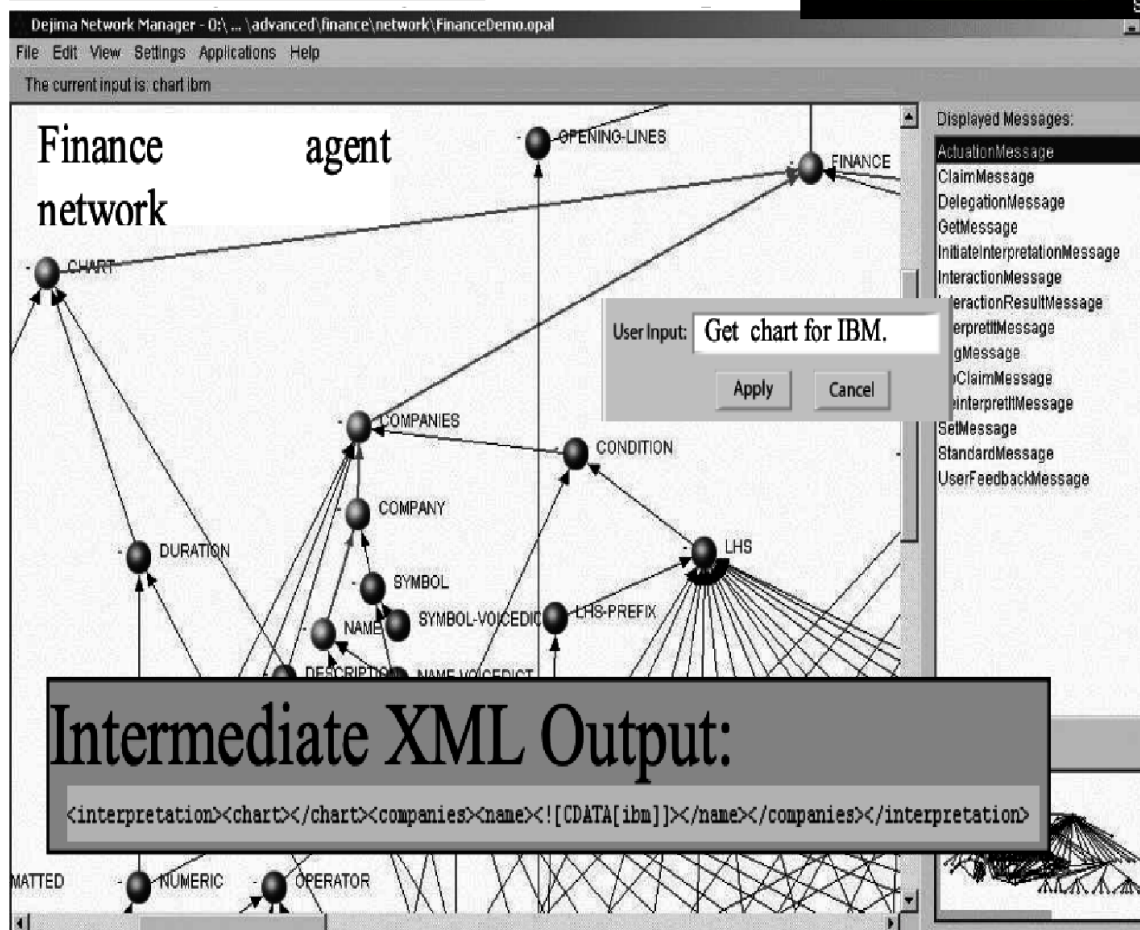


Figure I-1. An early natural language interface, shown here, used an agent-based architecture. Non-LLM agents operated in a hierarchy and were designed, developed, and tested in the platform shown. A similar technology was also used in the original pre-Apple Siri shown here.

The Age of Generative AI

Fast-forward to today: generative AI (Gen AI) has shown unexpected emergent capabilities, the most important of which is the understanding of natural languages. Large language models, or LLMs, are able to understand, translate, and even write code or produce images based on a natural language description. Natural language processing (NLP), understanding, and translation—problems that were the first non-numerical application of computers and considered very difficult to solve—have now been effectively addressed through the use of LLMs.

What’s more, these AI models can express intent and context in natural language. The very human ability to put concepts together to form new concepts, describe new contexts, or express new meaning is now available in machines, which makes them much more robust than any artificial system ever made.

Language was developed by humans as a means for structured, high conceptual bandwidth communications. Without language, human societies would not be functional. Over the years, language has become richer as new concepts have emerged. Now that machines “get” human language, we have a universal protocol for human-to-machine communication: we can talk to them in our language, and they can respond in kind.

Interestingly, as a consequence of the invention of LLMs, we now also have a much richer and more robust machine-to-machine communication language in the form of our own natural language. Since machines now “understand” English (or German, Chinese, or Hindi), we can have them communicate with each other in one or more languages, which has the added benefit that their communication is understood by us, which is helpful if we want to understand and follow the decision-making process. Of course, natural language is also inherently ambiguous, which brings about its own challenges, but we will address them later in the book.

So, out of the box, we have a universal protocol for inter-agent communications. Because of the flexibility of natural language to express intent, and the ability of LLM-based agents to map such intent to the manner in which their tools are called, software systems of the future will be much less brittle than the ones we have today: an agent responsible for a functionality takes care of mapping intent to the specific API calls, and so it will be much less of a hassle to change the API, or employ a different software with similar capabilities. This will give organizations more flexibility in upgrades or with third-party services.

If inter-agent communications and coordination are done in a manner that guarantees the encapsulation of responsibilities, then future enterprise applications will be much more robust, with new capabilities and functionality being introduced to a preexisting network of agents with little to no need for reengineering the system that is already there.

Given how a machine learning approach, using deep learning and transformer architectures and training on language modeling, has resulted in such capable systems as LLMs, it is natural to think that we are back on track to create a single model that is generally intelligent. In other words, some people think that building ever larger LLMs, and training them on more and more data, will result in an LLM that is more intelligent than humans in every aspect of human intelligence. It seems, however, that for very different reasons, we are now back to considering agent-based systems as a next natural step for software (see [Figure I-2](#)).

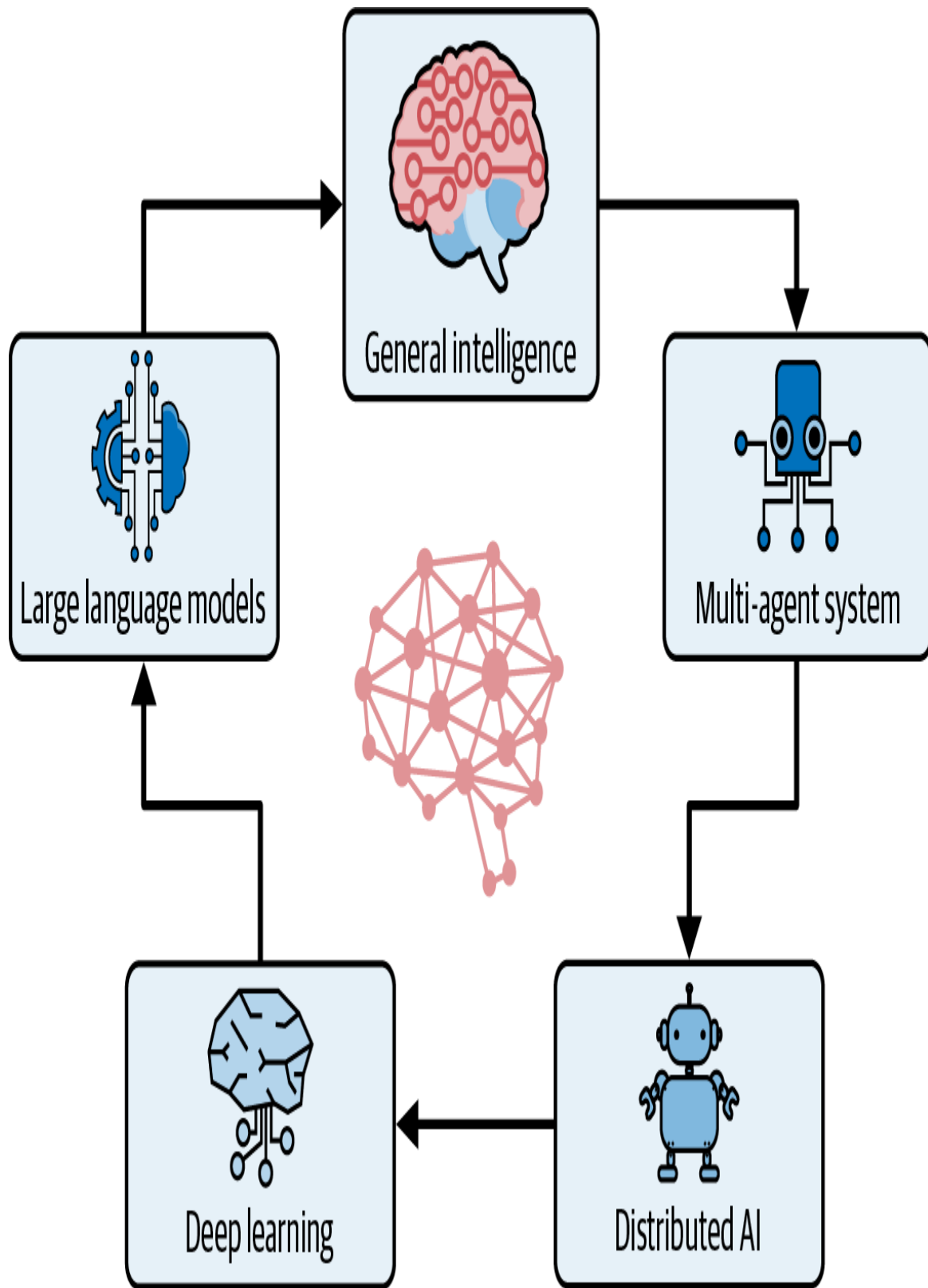


Figure I-2. AI has come full circle: initial failure in building generally intelligent systems resulted in breaking the problem into smaller sub problems, handled by agents in multi-agent systems, which, in

turn, led to distributed AI. Large neural networks, or deep learning, can be thought of as a distributed AI system, and LLMs are a special kind of deep learning. We make the case that a single, very capable LLM cannot be generally intelligent, unless it is designed as a multi-agent system.

LLMs are pretrained, and so it is difficult to make them fundamentally change their world model online. LLMs are typically trained on publicly available data, so, not having information in their training that is specific to a commercial problem domain, say proprietary data a company may be collecting and using, they are not generally useful on their own for business applications, and an engineered harness needs to be built around them to mitigate these shortcomings.

Also, LLMs need to be given a *system prompt* to be useful. A system prompt is part of the LLMs input context that keeps being repeated with every distinct input, so as to remind the LLM of its role and what is expected of it. Through system prompts, LLMs can take on different personas and operate differently. This means that, to play a specific role within a knowledge workflow, we need to give an LLM specific instructions, declare it to have specific expertise, and give it a specific job description and set of responsibilities.

The ability of machines to both understand highly complex human-designed concepts and execute specific instructions has led to a resurgence of interest in agent-based AI, or agentic AI. Can we get machines to chart their own path toward executing complex goals and engineer the set of instructions to make that happen and do so autonomously?

The challenge is altogether very different to that which the early AI pioneers faced in the 1990s: back then, we were forced to simplify the problem domain because of a lack of compute resources and model capabilities. Now, we still simplify the problem domain but that is because generative AI models are too powerful and flexible, and we need to contain them and task them with specific roles within our organizational use cases. Thus, the advent of modern LLM-based agents.

To summarize, an LLM-based agent is a combination of an LLM with code that, in its simplest form, actuates on the LLM's intentions, and in more complex systems, provisions useful augmentations for the LLM such as

planning task lists and memory. Any code or prompt written to enable an LLM to autonomously get a task done is called an *agentic harness*, and systems built to automatically, and even sometimes agentially, create or improve harnesses are called *meta-harnesses*.

Multi-Agent Systems

Many uses of generative AI call for confining an LLM to a specific role. In **Figure I-3**, for example, we could choose to have the LLM respond to the query “What is the average weight of penguins on Biscoe?” It may already have learned the answer from its general knowledge, but if it has not, it will try to guess, and the answer might not be reliable. But there is a more reliable alternative: if we have access to a database that we know to include reliable data, say on penguins, it is preferred that we tell the LLM about the schema of the database and the type of SQL used to query it, and have it translate our natural language inquiry into a call into the database.

LLMs are great at writing code, but they don’t have the facility to make the actual call and retrieve the data, so we need some helper code for that. Our code would pattern match this intent of the LLM and make the call to the database, retrieve the answer, and pass it on to the LLM to formulate it into a natural language response (see **Figure I-3**).

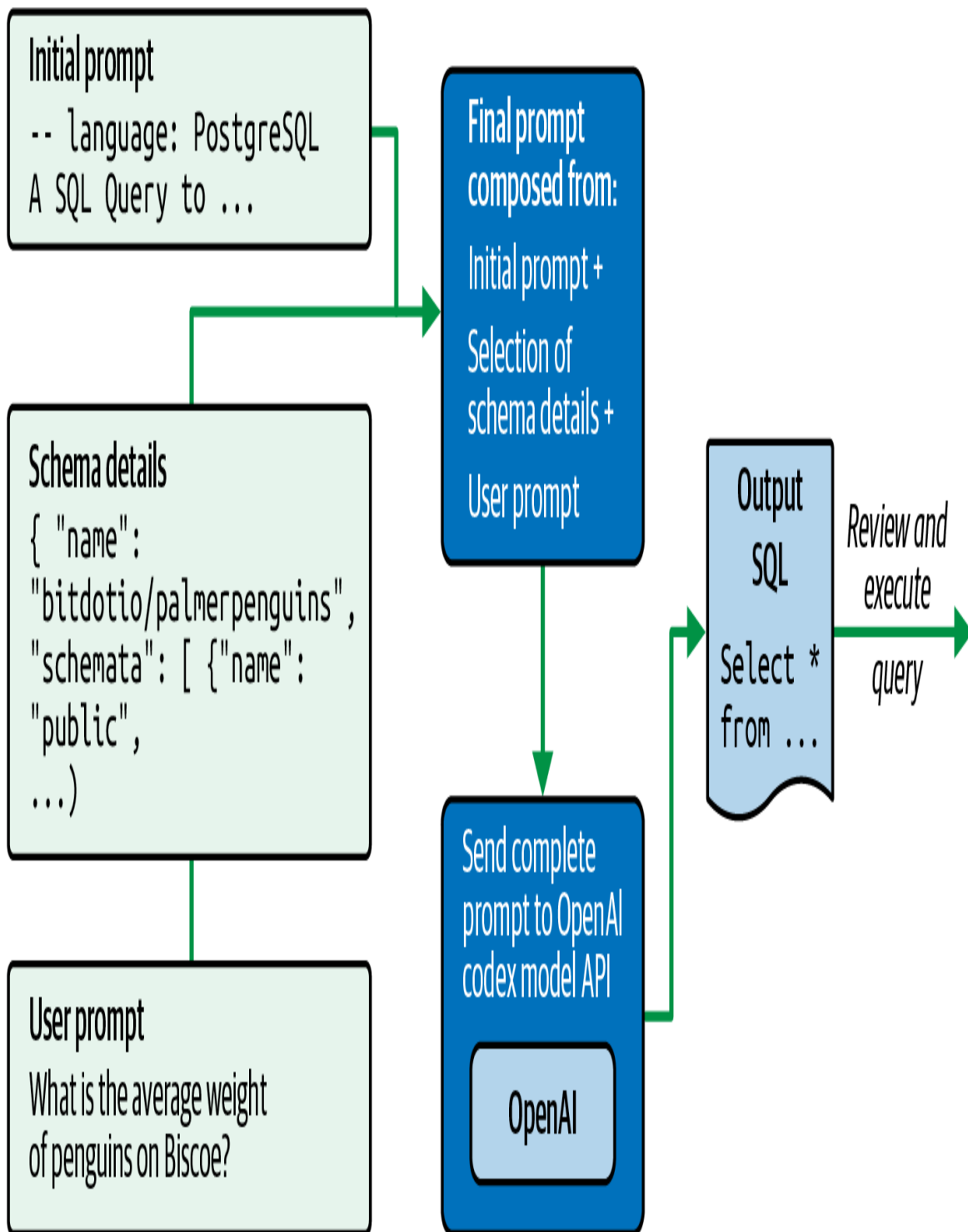


Figure I-3. Using an LLM as an agent to make a call to a database.

In fact, a popular use case of LLMs these days is retrieval-augmented generation (RAG).² Many companies are seeking a ChatGPT-like interface to their proprietary document repositories, and this is implemented by

summarizing the documents offline and storing them, along with the LLM's representation of the summaries (known as embeddings), in a vector database (a special kind of database used for storing and retrieving vectors). The embeddings are vectors, or lists of numeric values, capturing the essence of the summaries. By storing them in a vector database, we can calculate numerical distances between the summaries, revealing the degree of similarity of different pieces of text. This makes it easier to search by meaning, rather than by strict and rigid pattern matching of the words and sentences. This system can then respond to future natural language questions regarding the proprietary documents by mapping the requests to queries to the vector database, and finding summaries that are closest in meaning to the query at hand.

Hence, you can see here too that RAG can be thought of as a multi-agent system, with an agent for offline summarization, and one tasked with responding to the user's natural language queries (see **Figure I-4**).

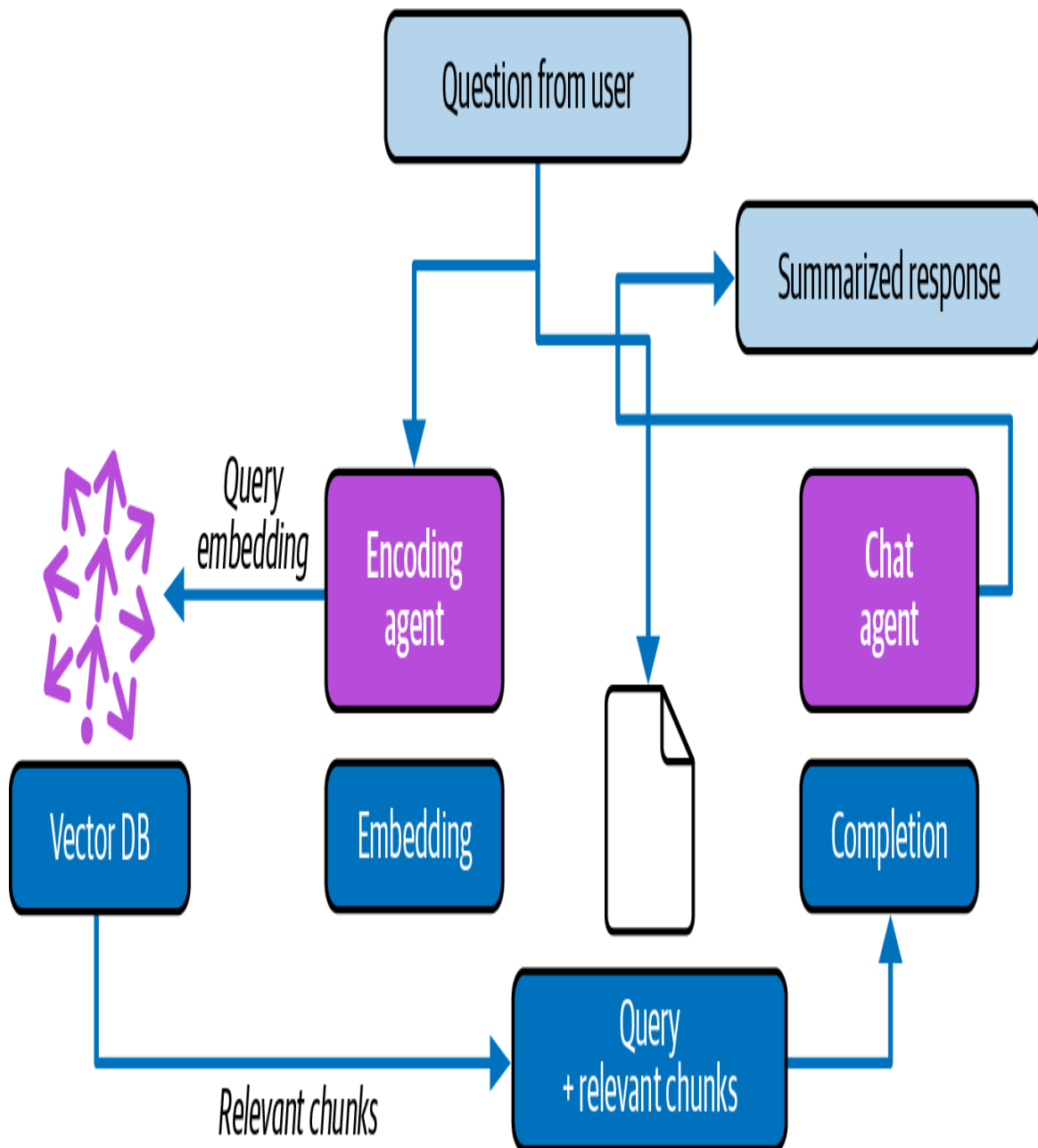


Figure I-4. RAG can be thought of as a multi-agent system.

But much more can be done. Any workflow requiring knowledge workers can be thought of as a multi-agent system. In **Figure I-5**, for instance, we can have a planning agent come up with various plans, say for an upcoming vacation. An actuation agent with knowledge of APIs for internet services such as Expedia or Hotels.com, can then call the API and get actual numbers, say for airline price or hotel dates, into the plan. And a “critic” agent can check the results and decide whether they are plausible (e.g., is

the connection time for a flight reasonable?) and send them back for refinement if they are not.

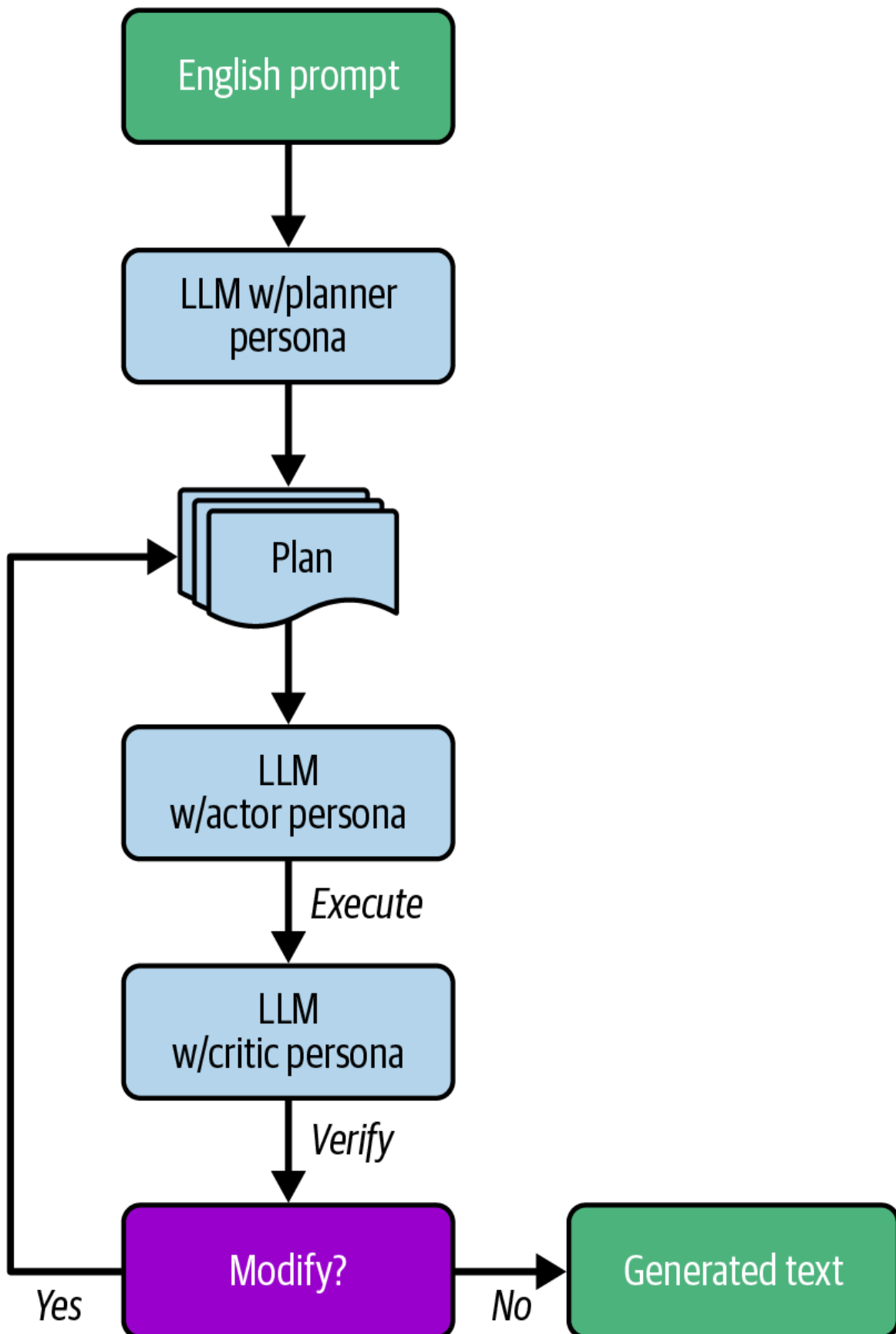


Figure I-5. A multi-agent planning system.

Let's formalize our view for an agent-based system a bit. What we mean by an agent is a generative AI LLM wrapped around a module, function, service, or database. This allows an end user to interact with that functionality using natural language (see [Figure I-6](#)). It also allows us to log the intent of the agent upon servicing every input from the user. In other words, we can log the “why” and “what” of the agent's behavior, because the agent “understands” its context and input, and can produce natural language describing every step it is taking. This also allows us to introduce safeguards on the agent's behavior, by having it observed by another agent, let's call it the safeguard agent. This agent can be tasked with our ethics, compliance, or regulation guidelines, and intervene if what is asked of the system is not compliant.

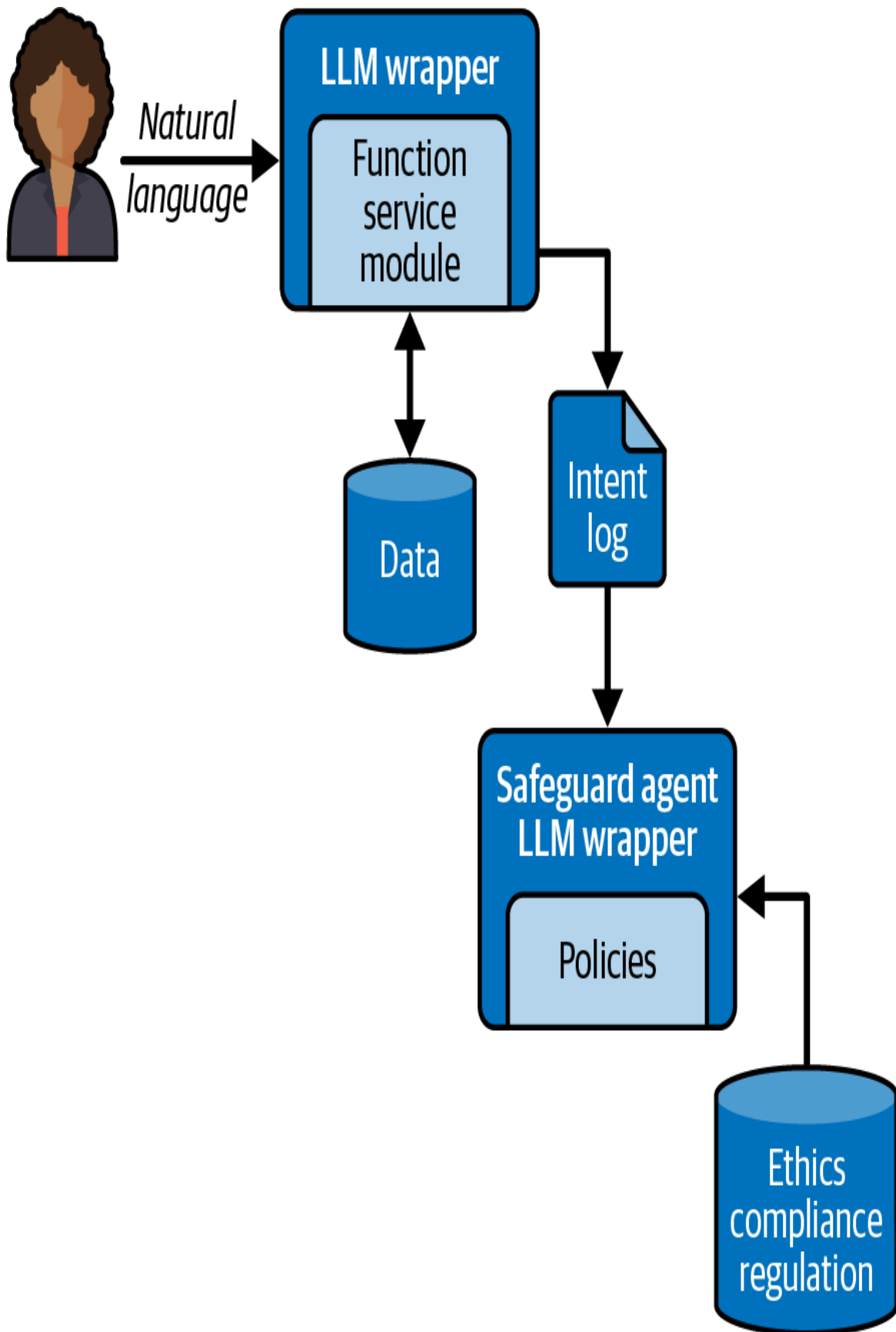


Figure I-6. An LLM agent is an LLM wrapped around some functionality, for instance a database.

By the same token, let's now assume that, in addition to the database in **Figure I-6**, our application also includes a microservice (e.g., an API call to a CRM backend), and some of the user's queries or commands are better handled through that. We have a choice in how the user interacts with the agent: the user can separately use an agent for that service, or the agent can collaborate with another agent that is in direct interaction with the user, making it seamless and more usable.

To make this happen, we can have the original agent servicing the user aware of the agent wrapping the microservice and have them communicate to fulfill user queries. This communication can be in natural language, which means it can capture the intent and be robust to changes in the format and syntax of the specific calls to the underlying services. The first agent, in other words, can "tell" the second agent, "my user needs XXX done. Do it for them, if you can." (Of course, this would cost more and take more time, so there are certainly trade-offs. We are paying in time and money, in exchange for modularity, reliability, and safety. More on this later.)

An enterprise's entire software and applications can be agentified in this way (see **Figure I-7**). Of course, allowing for such agentification to happen organically is fraught with risks, which we will discuss later in the book.

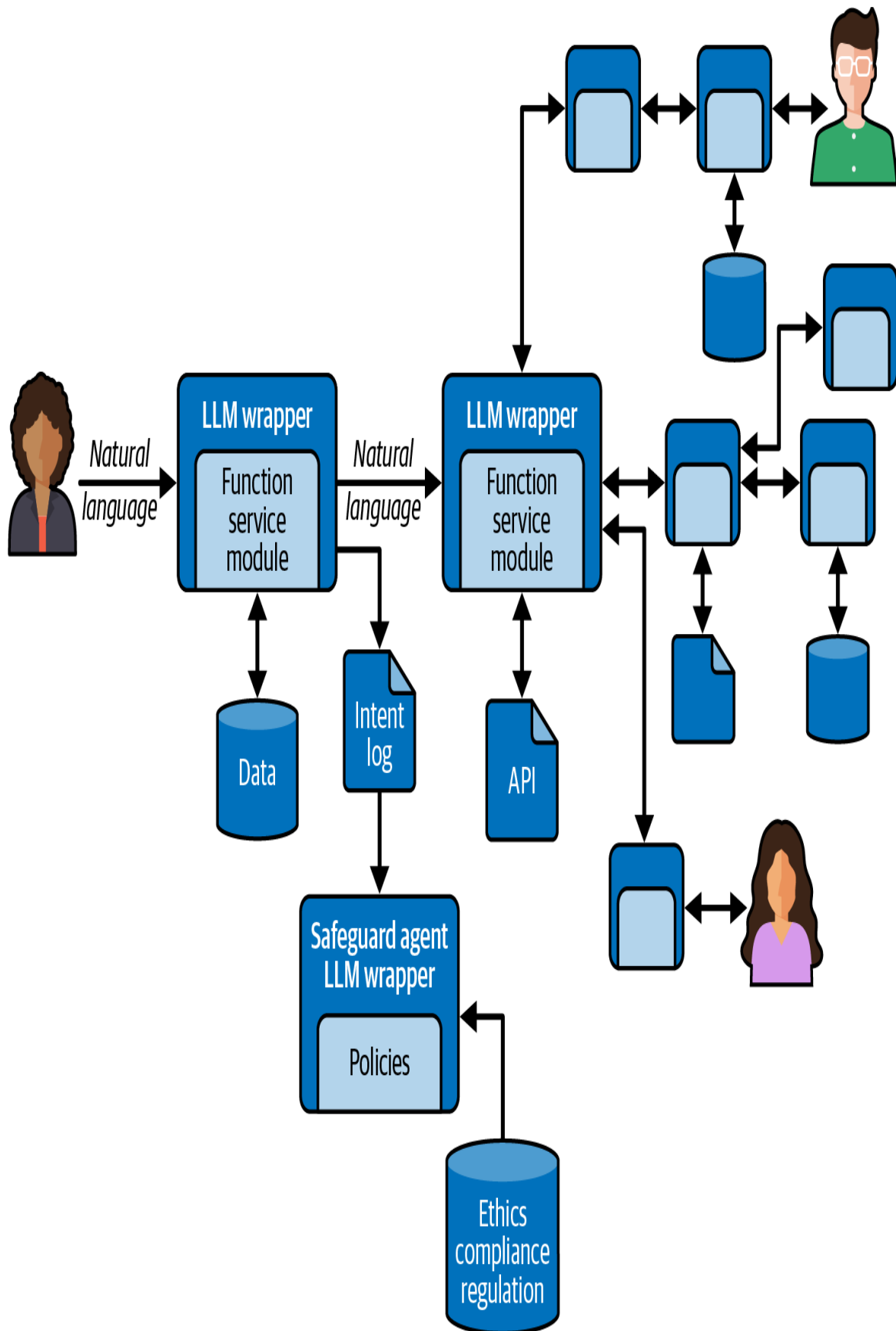
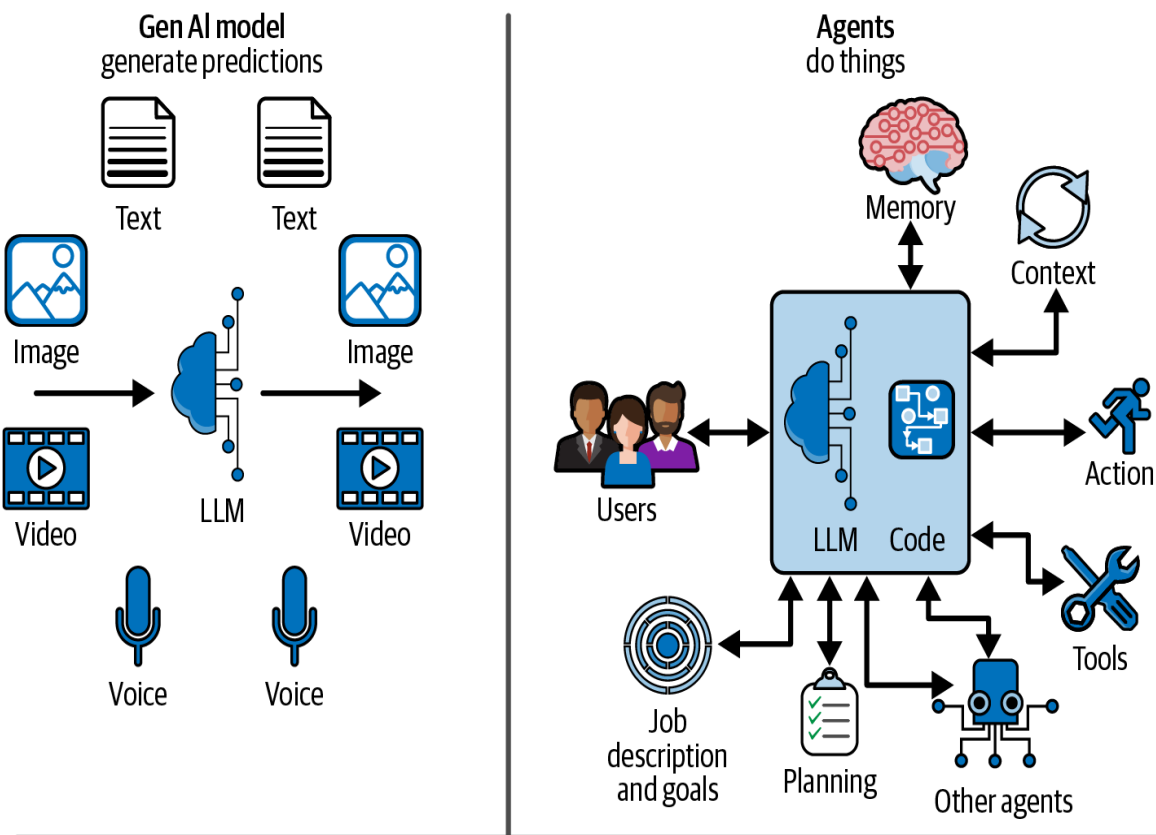


Figure I-7. An agentified business.

Figure I-8 will help us clarify the distinction between a model and an agent. A model is a transformation function. It takes data as input, and produces data as its output. A modern-day AI agent consists of, at least, (access to) an LLM that has been enabled to ask for calls to be made to some tools.

To summarize, an LLM takes some input and, through the process of predicting what is expected, produces some output. For example, we give it a coding task, and it produces some code. Depending on the complexity of the task, the code may be usable as is, but to run it, you will have to copy-paste it into your programming environment and manually hit “run.”

By contrast, an agent, given the right set of tools for coding, would start by having its LLM generate the code, then run it in a container, observe the results, and modify the code, therefore have a better chance of producing useful code. In fact, taking an LLM and running it on a coding benchmark results in lower success rates than when the same LLM is powering a coding agent and is given the same coding benchmark.



Multi-agent systems work together

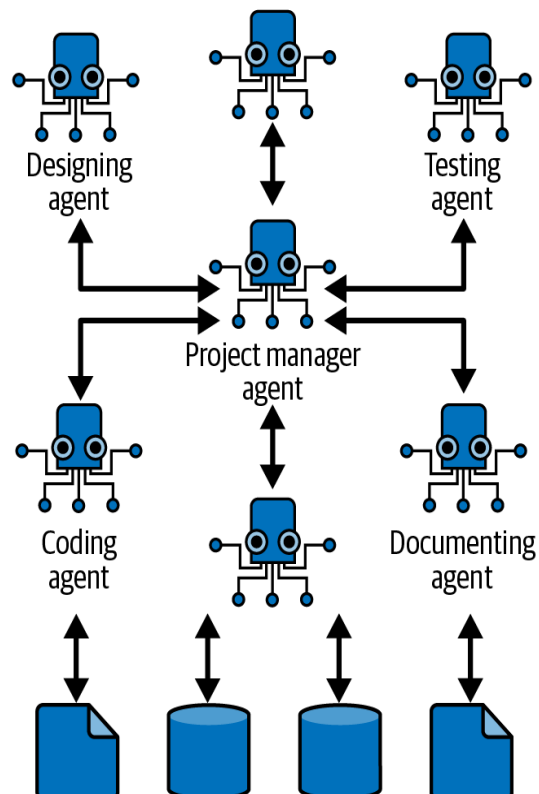


Figure I-8. Model versus agents.

The field of multi-agent AI calls for agents that can communicate and coordinate. This can be in the context of a team of agents, which implies collaboration, or it may be across teams of agents with varying levels of trust and even adversity.

Given the power of multi-agent systems, one can expect that the fabric of future organizations will consist of interconnected agents; however, for the foreseeable future, and unless a seismic shift occurs in culture and society and agents are conferred some level of personhood, humans will remain the sole bearers of responsibility. This means that multi-agency will be controlled and designed with humans as the ultimate arbiters and approvers of higher risk behavior.

¹ Paolo Ciancarini and Michael Wooldridge, eds., *Agent-Oriented Software Engineering: First International Workshop, AOSE 2000 Limerick, Ireland, June 10, 2000 Revised Papers* (Springer, 2003).

² Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems* 33 (2020): 9,459-9,474.

Part I. Why the Agentic Enterprise Matters

We will now make a case for the agentic future of business. We will first show how this is a natural next state for enterprises, given their progressive quest for automation ([Chapter 1](#)). In [Chapter 2](#) we will make the business case for why enterprises should agentify, and then, in [Chapter 3](#), we will ground our arguments by reviewing a number of actual agentification use cases in a variety of business domains. We will close this part by discussing in [Chapter 4](#) how the return on investment (ROI) and risk of agentification should be defined and measured.

Chapter 1. From Automation to Agents: The Evolution of Enterprise AI

Most early enterprise applications of AI were rule-based expert systems, with human designed taxonomies helping with business decisions, and in the case of RPA (Robotic Process Automation), automating some steps in a rigid process. For example, MYCIN,¹ a rule-based expert system for diagnosing bacterial infections and recommending antibodies, was built at Stanford University in the early 1970s, and Blue Prism,² introduced in 2001, was a popular RPA platform for automated, structured back-office workflows like data entry and claims processing, used in such sectors as banking, insurance, and government.

With the advent of deep learning, large neural network models trained on data to help with classification and prediction problems, some experimentation and niche use cases started, with the hopes that such models would uncover patterns in the data that had remained hidden to earlier statistical regression or machine learning (ML) approaches.

It must be mentioned here that, prior to the introduction of LLMs, the field of applied AI employed a much more diverse set of tools. These included symbolic regression, evolutionary optimization, predicate logic systems, and random forest models, to name a few. Today, some of these approaches are not even recognized as AI, and applied AI is mostly LLM-based. Approaches such as deep learning are commonly referred to as “traditional” ML models now, and LLMs have overshadowed the variety of AI tools and approaches in the field.

While we acknowledge the fact that the field of AI is much richer and wider than LLMs, in this book, we define AI agents as using their more recent

configuration, which always includes an LLM, wrapped in a program that actuates the LLM's intent.

The ChatGPT Effect

November 2022 was a pivotal moment for AI. OpenAI released GPT 3.5, also known as ChatGPT, and suddenly, a much wider audience got an idea for how profoundly powerful and generally applicable AI systems can be. With all its flaws, ChatGPT 3 was able to “understand” any command or query, attempt to respond to it, and explain itself in clear, natural language.

The form by which this powerful AI was revealed has had a lasting effect on how it has been deployed: a chatbot interface for retrieving data and information, best suited for support calls, FAQs, knowledge repositories, and general help.

OpenAI, and others, were also quick to point out an interesting behavior of LLMs: confabulation. These models seemed to make things up, what is commonly known as “hallucination.” This, paired with the fact the LLM models pulled from a probability distribution and were therefore inconsistent in their behavior, raised many flags with businesses looking to deploy AI. There was also a data security consideration due to the fact that the most powerful LLMs were hosted by commercial entities like OpenAI, Google, and Anthropic. It didn't help that OpenAI announced it would store user queries and use them to improve its AI (the company later retracted this).

Businesses, therefore, initially focused mostly on low-risk, high-return use cases, and ended up scratching the surface of applicability. The two main application areas, outside of personal employee productivity, were RAG-based information retrieval and coding companions (i.e., copilots).

High expectations for AI, yet limited use cases, meant that the initial expected ROI on LLM-based AI deployments went unmet. Considering that LLMs in their useful form were introduced in late 2022, and most

experimentation started in 2024, perhaps more patience and experimentation were in order.

Another factor beyond the limited application areas was the expectation that AI systems, being generally intelligent, would be deployable out of the box, with very little—if any—customization. This was exacerbated by the fact that most AI labs were started, and got their funding, to produce AGI (artificial general intelligence), which implies a monolithic, one-size-fits-all AI model. It took the world a year or two to realize that applied AI should be engineered and customized to the use case.

Engineering AI

Today, there is still a case to be made as to why we should engineer multi-agent systems versus relying on a single LLM-based agent to do everything.

To start with, the manner by which LLMs are fine-tuned on chain of thought (CoT) reasoning is useful for certain agentic tasks requiring reasoning or coding, but it does seem to bind the agent to a certain average reasoning behavior that makes it less efficient, brittle, repetitive, or poorly matched to tasks outside the scope of fine-tuning (e.g., orchestration, delegation, exploration). In using reasoning models for coding, these systems often create a relatively fixed “default mode” of reasoning.

For example, a reasoning LLM may decide that a bug is due to version dependency issues and fall into a cycle of advising us to try variations of library package updates, reinstalls, or reconfigurations, and fail to pop out of this cycle and recommend checking other possible causes of the failure. A multi-agent system of agents specialized in various aspects of the code or software development process would have a much better chance of finding the actual source of the bug. This is where the architecture of the agentic system prevents a single stream of reasoning from looping endlessly.

The moment we move from using models (e.g., LLMs) to using LLM-based agents, we are entering engineering territory. The manner by which an agent’s responsibilities and tools are defined and customized for a use case

requires elaborate design and verification. This is all the more important when we are designing multi-agent systems because the interconnection and roles of the various agents need to be engineered. For the same use case, the design, including the testing, validation, monitoring, and operation of an agentic system, can make the difference between effective and safe versus ineffective and risky. The job description of the agent, and the selection and implementation of its tools, depend very much on the use case.

Modularization, with its benefits (reuse, specialization, and divide and conquer), is well-suited for engineering work.

There is no one-size-fits-all LLM for all agents. This becomes even clearer considering the advantages of using smaller, more specialized LLMs for more specialized agents. This gives us the option to run some agents on premises, for example in settings where there are sensitive data considerations. In some cases, the agent or module need not even be an LLM-based agent, and we might consider replacing it with a software module or app.

Given our experience with LLMs as chatbots, it is often thought that agents are transactional systems that do not do anything until invoked by a text command, and go back to sleep after they have responded. Agents, however, can be configured to wake up on other, non-user events. Some agents run constantly, checking their environments and context, proactively communicating with users or other agents, or being on a long-term mission of discovery, maintenance, or improvement. For example, coding agents can open pull requests and react to events like reviewer comments or code merges. In a telco setting, some agents might be constantly running on network nodes on load balancing and incident recovery missions.

Another practical consideration that makes us lean toward multi-agent systems is interoperability. Many businesses are promoting their own agents, and enterprises will want to connect those agents to each other, or to internally connect proprietary homegrown agents with one another.

LLMs can also be fine-tuned depending on their job description within the larger multi-agent system. We simply collect and label data specific to an

agent's behavior and fine-tune for each agent separately. The same would not be possible with a single agent that is meant to do everything.

LLMs do not learn from lived experience. Like in the movie *Memento*, they have to be reminded of everything every time they are invoked. The *context* of an LLM is everything we need to feed it in order to get a useful output.

NOTE

The concept of experience is a fundamental difference between the state of the art in AI and human intelligence, with the other difference being that LLMs can be all knowledge and any personality, all at once, and so need to be told what role to use for a given input. For example, we need to tell an LLM it's an expert in horticulture before asking it a related question, and then, maybe, tell it that it's an astrophysicist to respond to a different question. Telling a human they are an expert in something does not work quite as well.

Context size is a limiting factor for LLMs. Even with LLMs that can take a relatively large number of tokens as input, staying faithful to everything that is stated in the context is a challenge. (When measuring input capacity for an LLM, a token roughly equates to a word.) In agentic systems, the system prompt, dialog history, memory, and tool descriptions all take valuable context tokens. All these elements are reduced as we limit and focus the responsibility of individual agents within a multi-agent system. If we are smart about memory and dialog history, we can afford to be much more selective and specific about how we use an agent's context, making agents more reliable and predictable in their behavior.

Then there's the question of performance. Is a single LLM agent more efficient than the equivalent multi-agent system? Would we augment or replace all the operations and processes of a business with a single agent? A recent position [paper](#) argued that more refined decomposition into agents is critical for (at least) efficiency at scale. There's even some evidence that [scaling laws](#) analogous to those in neural networks, which resulted in the amazing emergent behavior in LLMs, are also exhibited in multi-agent systems, and we are talking cooperation among more than one thousand

agents here. Of course, efficiency can come at a cost. However, if the added efficiency brings about more reliability, as is the case in the referenced studies, then systems can use less costly LLMs for the same agentic use case, mitigating some of that added operational cost.

The choice and granularity of tools entrusted to an agent can have implications on explainability as well as cost. An agent that uses fine-grained command-line tools—that is, has full control over a (containerized) virtual computer—is appealing for how broadly it can be applied, but it would be hard to understand what it is doing because you’d have to be able to follow the logic of its operations. An analogy is having to understand assembler code versus a higher-order language. Additionally, **it has been shown** that the sheer number of instructions such agents will have to run to get things done makes them much more expensive (up to 45 times!) than giving fewer higher-order commands as tools to a multi-agent equivalent.

Furthermore, a modularized multi-agent system with specialized agents means less context per LLM, which, if designed well, will result in a more efficient system. This is because transformer-based LLMs slow down significantly with larger input size. A single-agent system will have to see the entirety of the context of the problem and feed it into its LLM, whereas agents in a multi-agent system need to see only part of the context that they are dealing with. This means that in a setting where agents can be called in parallel, we stand to gain in the efficiency and cost of operating the system too.

Agentified Applications and Software

While single-agent systems are useful, the real power of enterprise agentification comes when agents are allowed to talk to each other. For example, imagine your company intranet with its useful search box directing you to the apps and resources you need. If yours is a large enough company, these apps, owned by different departments, each have their own search boxes. It makes a lot of sense to create agents, maybe by using techniques like RAG, to augment the search boxes. What does not make

sense is to force the user to repeat their query in an app's search box once the intranet search box has identified it as useful given the initial query. Rather, we would prefer a top agent to coordinate with agents representing the various apps and present a consolidated and unified chat interface to you, the user (see **Figure 1-1**).

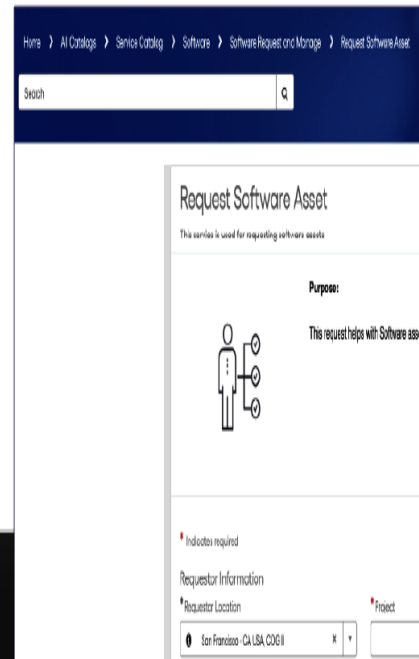
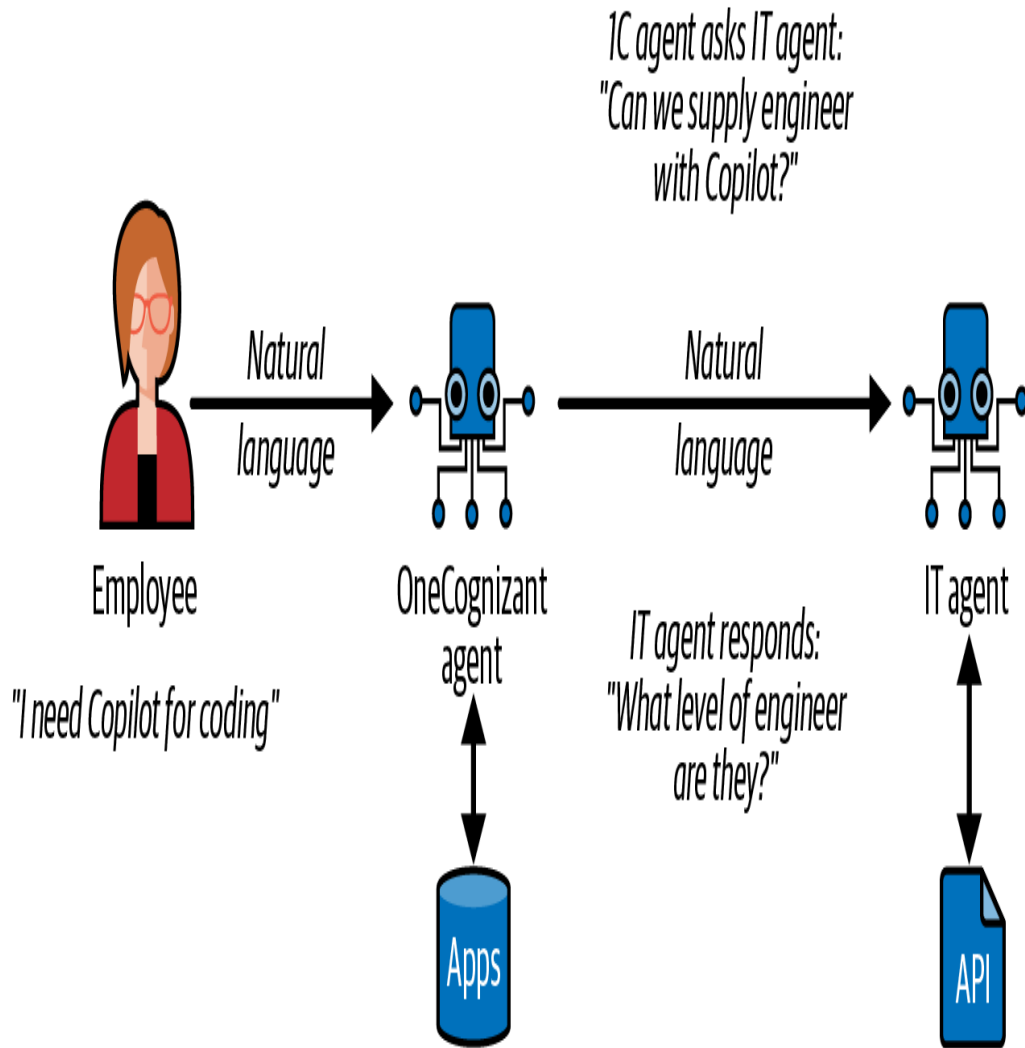


Figure 1-1. Agents should talk to one another. Here, the agent augmenting the intranet search box talks to the agent augmenting the IT app's search box.

A multi-agent system representing an organization's software or various workflows can have several interesting advantages, including improved productivity and robustness, operational resilience, and the ability for faster upgrades of different modules. This book will help you see how this is achieved and provide you with examples illustrating these advantages.

Migrating a software/application stack to an agent-based architecture offers several distinct benefits, improving operational resilience, increasing engineering velocity, and reducing costs:

Intent logging

Each agent can log its intent on every transaction or call it makes to various services, enhancing transparency and traceability. This is harder to achieve with software logs, which tend to be terse but voluminous, and lack explanation of the various contexts and rationales behind the software's behavior.

Robust inter-agent communication

Communications between software and apps represented by agents are intent-based and often in natural language, making the interactions more understandable and reducing the complexity of integration.

Safeguard agents for responsible AI

The ability to add safeguard agents to monitor and check the behavior of each agent ensures compliance with responsible AI practices and ethical standards.

Modularity and flexibility

Agent-based systems are highly modular, allowing for easier updates and maintenance. Each agent can be independently

developed, deployed, and scaled without affecting the entire system.

Upgradeability

Software and application modules being represented by agents provide a more seamless and less disruptive way to upgrade the underlying software, as the complexity of mapping intent to the actual backend calls is handled by the LLMs. This simplifies, and even does away with, much of the manual rewiring often involved in API and software upgrades.

Interoperability

Agents can interact with diverse systems and services, providing a unified interface and facilitating integration across different platforms and technologies.

Autonomy and specialization

Each agent can be specialized for a particular task, leading to more efficient and optimized execution of specific functions within the application stack.

Enhanced fault tolerance

The failure of one agent or software backend does not necessarily disrupt the entire system. Agents can be designed to handle failures gracefully, improving overall system reliability.

Improved data handling

Agents can handle data locally and only communicate relevant information, reducing data transfer costs and improving performance. Disparate data sources can be

represented by different agents, helping mitigate issues with data consolidation or being behind on data modernization.

Better resource management

Agents can optimize resource usage by adjusting their operations based on current system states and workloads.

Customizability and extensibility

New agents can be added or existing ones modified with minimal disruption, allowing for continuous improvement and adaptation to new requirements.

Breaking software silos

Redundancies and overlaps in various enterprise software modules are a constant source of frustration for users. Agents can help reduce or eliminate such silos by operating the software modules on behalf of users, and consolidating and simplifying responses across multiple software modules as they serve a user's intent.

These advantages can result in major operational efficiencies, IT and infrastructure savings, and measurable enhancements to the quality of business decision making. However, the benefits of multi-agents systems also come with the need for careful and safe design considerations.

Given an agent's role, a level of autonomy is expected as it interacts with the world using its tools. In some cases, like running code in a container, the consequences can be controlled and contained. In others, scrutiny and care need to be applied to avoid undesirable consequences of an agent's autonomous behavior.

The fact that the “brain” of an agent is an inherently opaque neural network (i.e., an LLM) makes this more complex. LLM-based agents are also prone to hallucinations or misunderstanding of inherently ambiguous natural

language communications, so what we gain in robustness we can lose in consistency.

Several mitigation strategies are available to reduce these risks, including implementing rules for overriding or seeking human approval for certain agent decisions, assessing uncertainty of agent behavior, and pairing agents with safeguard agents that monitor and prevent potential harm. Let's take them in turn:

Rules for agent autonomy and control

An agent can be wrapped into deterministic programs that allow us to impose rules on conditions within which the agent is allowed to make use of its LLM's non-deterministic reasoning. The inverse is also possible, where LLM reasoning supersedes rule-based behavior, and we defer to the LLM on when to switch to human defined rule-based behavior.

Using LLM uncertainty

There is a third option to determine when to defer to the LLM's reasoning versus rule-based behavior, or even to call a human to make the decision on what to do. This requires an estimation of LLM uncertainty. Although LLMs are opaque or black-box models, which means it is very difficult to determine why they behave the way they do, there are techniques to estimate how confident they are in their own output. For example, it is possible to get a sense of the range of different outputs an LLM is likely to produce for the same input. This range can help us calculate an absolute value for the LLM's confidence for the various outputs it can produce for the same input. If we then set a threshold on this confidence value, we will have a future-proof mechanism to switch between deferring to the LLM when its confidence is higher than the threshold, and switching to predetermined

rule-based behavior when the LLM's confidence does not meet the threshold.

Safeguard agents

One or more agents can be defined to be responsible for checking the proposed behavior of another agent and determining whether the behavior is acceptable or not.

Given the required oversight, we need a testing regime for agent-based systems that differs from what we are used to in traditional software. The good news, however, is that we are used to testing such systems as we have been operating human-driven organizations and workflows since the dawn of industrialization.

The state of the art in generative AI models prevents single agents from effectively taking a large body of complex instructions and carrying out too many varied and complex tasks. Given the limit on the size of an LLM's input (i.e., context window restrictions) as well as limitations in the reasoning power of LLMs, it is often more effective to break responsibilities down into multiple AI agents, connected and coordinating to get work done. Just as we do not assign a single software engineer the task of writing a fully-fledged CRM system, asking an AI agent to write code for a full CRM system may be a tall order. But having a team of agents representing various responsibilities, such as project management, frontend and backend engineering, and QA, and working together is more likely to get the job done successfully.

A modern AI agent consists of an LLM and some code. The code allows the LLM to actuate its intentions using the tools we assign to it. Most systems facilitate the calls directly, with no intermediate calls. For example, the Model Context Protocol (MCP), which has become a de facto standard in defining and exposing APIs for agentic use, allows agents to discover and call an API without requiring manual integration. Some agentic systems do allow developers the flexibility to extend the code part of an agent by writing code that would run prior to calling tools (see [Figure 1-2](#)).

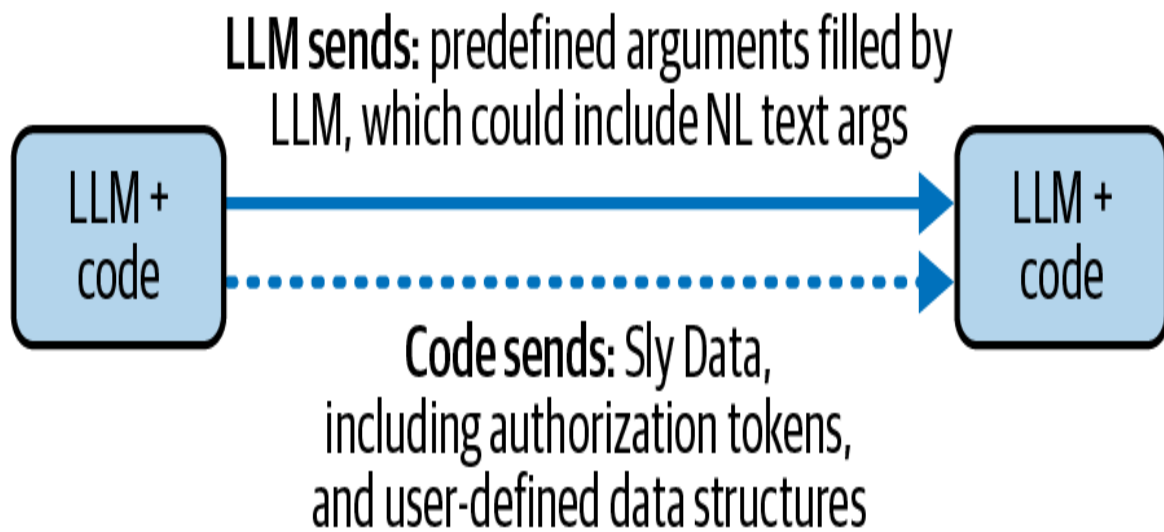


Figure 1-2. Inter-agent communications split into code and LLM: the called agent returns both the agent's LLM generated results and the data structures, which may have been processed by the called agent's tools.

This duality of LLM and code is useful and, as an important step in our design of a multi-agentic system, this allows us to think about what part of an agent's behavior needs to be handled by the LLM, and which part needs to be coded.

There is sometimes a tendency to think of AI agents as we do software modules and try to control them and make them consistent. This is understandable, but defeats the whole purpose of agentification, which requires ceding some control to agents and accepting some level of inconsistency, in exchange for robustness, efficiency, and quality.

When dealing with an agent, we should always assume some autonomy of behavior. An agent should be allowed, part of the time, to decide on its own which of its tools to use, in what order, and how. If you determine that your agent is not allowed any autonomy, then you should strongly consider replacing it with a software module.

Tasks that require reasoning, robust understanding of text or other unstructured input, or the generation of such unstructured data as output should be delegated to the LLM. More rigid rules-based behavior that requires consistency will have to be implemented in the code part of the agent.

In other words, we should divide an agent's responsibilities between those needing an intelligent knowledge-worker-in-a-box and those requiring predefined rules applied consistently. The former can be delegated to the agent's LLM, and the latter should be programmed into the coded part of the agent.

An essential tool to help apply this division of labor to a multi-agent system is `sly_data`, a data structure that can be operated upon by the code part of an agent, and that is passed around between agents through code, and so is not necessarily subject to LLM processing.

What also appears clear is that, beyond the technical means (like `sly_data`) to orchestrate multi-agents in an enterprise, a need is emerging to essentially define some sort of governance of the agents among themselves and of their interactions (or not) with humans. To some extent, the orchestration of multi-agents mimics human enterprise organization rules and how different specialized teams and units interact with each other.

Multi-Agent Organizations

A multi-agent system representing an organization's various workflows can have several significant advantages, including improved productivity, operational resilience, and improved robustness. For businesses that are rapidly moving to adopt single-agent solutions, multi-agent systems can appear to be an inevitable and quite disruptive future state.

Why use multiple specialized agents communicating with one another rather than using a single all-powerful, know-it-all agent? Essentially, using multi-agents offers a number of benefits over using single agents: flexibility in incremental engineering of complex systems, ability to safeguard an agent's behavior, improved agent's reliability, and broader choice of LLMs:

Flexibility in engineering

From an engineering perspective, encapsulating responsibilities and implementing a distributed inter-agent coordination mechanism (e.g., AAOSA; see [Chapter 5](#) for

more on this) means that we can build these agent networks incrementally, plugging in new agent subnetworks without having to reengineer existing agents.

Safeguarding agents' behavior

Safeguarding an agent's behavior is more effective when we pair the agent with a safeguard agent, versus asking the agent to do something but to be careful at the same time.

Better agent reliability

From a consistency and reliability perspective, the approach assumes much more specialization for each agent, making agents much more reliable in their behavior and less prone to issues related to LLM confabulation (i.e., hallucination).

Broader choice of LLMs

This also means that the bar is a bit lower on what LLM to use depending on the agent. Larger, more capable agents are expensive, slow, and mostly hosted. Being able to use smaller, more specialized LLMs means you are paying less, getting quicker responses, have the option to host agents that handle more sensitive data, and have an easier time fine-tuning the LLMs, if needed. Moreover, and as LLM context is limited, even for more powerful models boasting millions of tokens as their input context, a single agent for all uses can hit limitations. On the other hand, if context is used wisely, each agent in a multi-agent setting remains much more focused and less reliant on recalling and respecting what it is asked.

The same goes for output. If you ask an LLM to produce a complex output, it is likely to fail at producing everything you request, whereas for an agent in a multi-agent setting,

restricting responsibilities typically results in less output complexity requirements.

It is important to note that the larger the number of agents, the more costly and slower the system, so there is a balance to be struck between modularity, robustness, and consistency on the one side, and cost and response time on the other. Care needs to be taken as to the granularity of the agentification, as well as the types of LLMs being used and when and how to fine-tune them to make them effective. The latter is an option, once an initial multi-agent system is in production or running in a sandbox, where usage data can be collected and labeled by human auditors as being acceptable or not; this labeled data can then be used to fine-tune the LLMs behind various agents to make their behavior more acceptable.

Autonomy and the Role of Humans

Finally, there is a more fundamental question behind this choice between human-engineered systems of AI agents and autonomously AI-engineered systems: Where do we, as humans, exert our control and design? Or are we simply expressing intent and leaving the rest to the LLM?

We will address practical aspects of this question later in the book, but for now, let's just say that the answer is not as obvious as our initial intuition. We are building increasingly intelligent systems that, in many ways, are already smarter than us. Would we rather have a less intelligent system (us humans) design and set up more intelligent systems (AI agents)? We are already seeing spectacular performance by large reasoning models on benchmarks many of us would have a very hard time acing. Isn't it time for us to defer to these models as the artificial brains for handling all aspects of a system's operations? Let's just give all the APIs for a system to a single very powerful LLM and let it decide how and when to use them. After all, we may consider ourselves as still being "in charge" because we are expressing our intent as input to the agent.

Not so fast. There are fundamental limitations to state-of-the-art AI that should give us pause here. Our LLMs have a hard time knowing what they don't know (and for that matter, they also have a hard time knowing what they do know). In other words, they are lacking metacognition, something that seems to come quite naturally to us, perhaps because our brains are wired differently than a feed-forward, train-once, run-many-times, LLM.

Also, while seeing AI-generated artwork, prose, or even poetry does demonstrate incredible creativity by LLMs, another related [paper](#) shows how current ML and reinforcement learning methods have a hard time dealing with the unexpected effects of creative actions, or what is commonly referred to as “unknown unknowns.”

These limitations are not easy to overcome, unless we rethink our approach to AI or we have some major breakthroughs.

Multi-agent systems allow us to engineer safeguards for such limitations. Rather than potential magnification of risky behavior and inconsistencies due to relying solely on a single monolithic LLM agent, we can build resilience in distributed, modularized, redundant, multi-agent systems. These systems can have built-in agentic checks and balances as well as safeguards, including rule-based and human fallbacks. Additionally, the multi-agent approach allows us to build and extend systems incrementally, testing and fine-tuning various agents in isolation and within a sandboxed larger system, before plugging them into the live agent network. It also provides room for regulators of responsible AI to mandate the use of third-party agents for more critical and risky operations.

-
- ¹ William Van Melle, “MYCIN: A Knowledge-based Consultation Program for Infectious Disease Diagnosis,” *International Journal of Man-Machine Studies* 10, no. 3 (1978): 313–322.
 - ² Laxmikant Rane and Giri Gundu Hallur, “Competition in the RPA Industry: A Case Study of Blue Prism,” *AIP Conference Proceedings* 2914, no. 1. (2023).

Chapter 2. The Business Case for Agentic AI

In [Chapter 1](#), we traced the technical evolution that has brought us to the threshold of the agentic enterprise—from rigid rule-based systems and RPA, through the ChatGPT effect, to the engineering rationale for multi-agent architectures. We argued that multi-agent systems are not merely a technical preference, but a structural necessity: they deliver the modularity, specialization, and safeguards that make AI deployable at enterprise scale. But a compelling engineering argument is not, by itself, a business case. The question that naturally follows is this: why should a business leader care, and why now? This chapter answers that question directly, making the case that Agentic AI has moved well beyond technical experimentation to become a strategically urgent priority, with measurable returns already being demonstrated in the field.

Undeniably, ChatGPT and LLMs have raised awareness of AI in an unprecedented way, thanks to the ease of natural language interaction and the impressive (if sometimes imperfect) outputs they can generate at the touch of a button. However, this recognition has quickly been followed by three broad sets of concerns. First, bias and hallucinations have undermined the perceived quality of outputs that initially looked striking. Second, the compute power required to run these models has raised questions about their environmental impact, a concern that, if anything, increases with agentic AI due to the additional orchestration overhead and volume of LLM calls that multi-agent systems generate. Third, financial analysts have highlighted uncertainties around the returns on the massive investments made in the LLM ecosystem.

Agentic AI directly addresses the first and third concerns—mitigating hallucinations through guardrails and system design, and demonstrating measurable ROI through goal-oriented outcomes. The environmental costs,

however, are a shared challenge across all LLM-based systems, and one the industry must address through model efficiency, hardware improvements, and responsible deployment choices.

As a result, agentic AI has moved quickly beyond the “interesting” stage to become a strategically urgent and important technology, driven by both maturing technology and competitive pressure. It is transforming AI from tactical automation into a more strategic, value-creating capability. This, in turn, requires corporations to rapidly assess their organizational and technical readiness for agentic AI if they want to capture its benefits.

The case for acting now on agentic AI is not built on hype; it is built on evidence. In the pages that follow, we examine the three converging forces that make this the right strategic moment: technology that has crossed the maturity threshold, economic pressure that is already reshaping competitive dynamics, and organizations that have begun laying the groundwork.

Maturing Technology and Competitive Drivers

Three converging forces make this the right strategic moment for agentic AI and turn what was a bleeding-edge experiment into a mainstream enabler: advances in technology, economic pressure, and organizational maturity.

First, the underlying technologies for agentic AI have already passed a clear inflection point. Foundation models and orchestration layers have demonstrated that agents can operate autonomously across a range of domains, such as customer service, supply chain and logistics, and compliance. This level of maturity is underscored by the fact that agents are already running key processes in highly regulated industries like insurance (claims, underwriting) and banking, where regulators are stringent about protecting customer interests and avoiding systemic risk. In parallel, the broader agentic AI infrastructure has matured at record speed—through cloud-native APIs, secure model hosting, and related capabilities—giving

sophisticated organizations confidence that adoption will meet their professional and regulatory standards.

Two real-life examples come to mind that showcase the transformative power of agentic AI implemented at scale for mission-critical processes. Take Allianz Partners, reporting that they have scaled autonomous AI with Otera’s agentic workflows, moving beyond POCs (proofs of concept) to a true operational backbone with up to 90% automation of eligible claims, settlement times cut from around 30 days to 3 to 4 days, and a path from €300 million to €1 billion in increased profitability by 2030, according to their chief data officer.¹ Another financial-services leader, Franklin Templeton, reports a similar magnitude of impact on its core business in collaboration with Wand AI. Citing the press release on this collaboration: “Over the past year, Franklin Templeton has deepened its collaboration with Wand AI, evolving from pilot programs to full-scale production deployments of intelligent agents across multiple departments. The initial implementation focused on priority use cases within the firm’s investment teams. By 2026, Wand-powered agents are expected to be used in key enterprise workflows—supporting investment research, operational efficiency, and digital transformation—all underpinned by rigorous governance and control frameworks.”²

Second, from an economic standpoint, the timing for agentic AI is also right. Over the past decades, most corporations and public-sector organizations have undergone multiple waves of automation via various IT systems and solutions. As we mentioned in [Chapter 1](#), the most recent waves before agentic AI focused on specialized RPA, which excelled at automating scripted, rule-based, repetitive tasks, but did so in a rigid manner. Some RPA leaders even branded certain solutions as “agents” working alongside human agents on specific tasks. Yet these early RPA solutions had clear limitations that agentic AI is now addressing: being truly goal-oriented (deciding *how* to achieve a goal), demonstrating the level of autonomy and adaptability needed to do so, triaging complexity (and escalating to humans when appropriate), and orchestrating interactions with other agents and systems to reach an outcome. For example, Allianz

Partners USA had used RPA for more than a decade to process its claims more swiftly, only to replace—in 2024—its RPA with AI-powered technology capable of auto paying suitable claims, while Allianz Group is deploying multi-agent systems (for example, under Project Nemo) that orchestrate coverage checks, weather data, fraud analysis, and payouts end-to-end. RPA automated steps, and agentic AI now automates the outcome.

While focused RPA implementations eventually exhausted their opportunities for marginal efficiency gains, agentic AI injects new momentum into problem-solving by introducing decision velocity—i.e., the ability to sense context, evaluate options, and act autonomously across multiple systems in near real time, without waiting for human handoffs at each step—coordinating multiple levers in a largely autonomous way. This new momentum has already translated into impressive double-digit cost savings for large organizations, as shown earlier, across their most important middle- and back-office processes.

In addition, front-office tasks, such as lead generation, are also being reshaped by agentic AI solutions that drive significant revenue improvements, which bodes well for stronger and more sustainable ROIs. Take **Artisan**, a San Francisco-based start-up that has developed an agentic sales development representative (SDR) that automates outbound prospecting by finding, researching, and prioritizing B2B leads at scale. It continuously scans context signals about a company—such as its industry, the software it uses, recent hiring patterns, and new funding rounds—to identify accounts that are more likely to be ready to buy, before any human salesperson gets involved. It then runs personalized, multistep email and social outreach, so sales teams get a steady flow of warm, context-rich leads, instead of cold lists. For leaders, this is not just about squeezing out more efficiency. Once competitors use agents to make and execute decisions in minutes, whether that means rerouting supply chains, repricing offers, or saturating the best prospects, any organization that still needs days of manual coordination to do the same faces a growing competitive penalty. In this world, the cost of inaction on agentic AI is not just foregone savings,

but the risk of structurally slower decisions and potentially losing some of the most attractive customers and opportunities.

Third, organizational awareness and adoption readiness have increased markedly. After the emergence of LLMs and the initial “wow effect” experienced by employees and managers across industries, the question of “what should we actually do with this?” quickly rose on senior leaders’ agendas. In many organizations, LLM adoption has been cautious, starting with “wait and see” positions until implications were better understood, followed by a flurry of POCs that often did not make it into production. Yet this period has not been wasted: the idea that AI is inevitable has now crystallized in leadership circles, and many organizations have used this exploratory phase to begin **building** foundational capabilities in data and governance (MLOps, data governance) that create fertile ground for agentic AI to take root. Ultimately, there is a growing recognition among corporate leaders that agentic AI is more than a marginal tool. Citing Allianz Partners’ chief data officer on its deployment of agentic AI: “We didn’t have time to do proofs of concept, play around with generative AI, but rather think of this as part of our backbone in our organization to perform tasks, to make decisions....This was the very first time you could really hand over decisions. Once we launched the Otera agents...this was really a moment where I think the whole company realized the autonomous AI era is here.”

Strategic Value Versus Tactical Automation

Given that automation technologies have been used in business for years, it is legitimate to ask where agentic AI truly adds value.

A useful way to answer this is to contrast two automation objectives—tactical and strategic—rather than two technology types. Any automation technology, including agentic AI, can serve tactical goals: reducing cost by replacing repetitive human effort in stable, well-defined processes. Once those processes are fully automated, the scope for further gains tends to be limited and exhibits diminishing returns.

But agentic AI also uniquely enables strategic automation, where value comes from decision leverage, the fact that autonomous decision making offloads a significant amount of work from humans, who are brought into the loop only by exception. Here, ROI is driven by the agent's ability to discover the operating context, decide the "next best action," and build an emergent form of cognition that enables adaptive personalization and continuous optimization. This is where agentic AI creates the most differentiated value: not just automating steps, but automating outcomes.

For example, consider an AI agent tasked with rebalancing a large investment portfolio. A traditional rule-based system can execute predefined instructions—sell X if it exceeds Y% of the portfolio—but it stalls when the situation requires judgment: what if liquidating X to meet a target triggers an unacceptable tax event, violates a client's ESG (environment, social, governance) preferences, or conflicts with a pending trade in a related account? An agent, by contrast, can simultaneously weigh portfolio constraints, tax implications, compliance rules, market liquidity, and client preferences—dynamically exploring a near-infinite solution space to find a rebalancing path that satisfies all criteria, not just the one it was explicitly told to optimize. This illustrates how goal-oriented automation differs from narrow task execution: it is not about following a richer rule book, but about pursuing an outcome across a complex, shifting landscape.

This leads to a useful three-layer view of value in automation:

Value layer 1: operational productivity

Operational productivity is value from automating tasks and speeding up execution—productivity gains result from tasks being completed faster and at lower unit cost. This is the domain of traditional RPA and early AI assistants, and it remains highly relevant: automating invoice processing, data entry, ticket routing, or claims intake delivers measurable efficiency gains quickly. Agentic AI can operate at this layer too, but with greater flexibility, handling

exceptions and edge cases that previously required human intervention or bespoke rule additions.

Example: An agent that processes insurance claims straight through for standard cases, automatically validating documentation, checking policy terms, and triggering payment, without human involvement for the majority of eligible claims, as demonstrated by Allianz Partners. Beyond cost reduction, agentic AI also delivers organizational velocity, compressing process timelines that previously required weeks of human coordination into hours of autonomous execution. For instance Vamstar, a start-up focused on health care procurement, is deploying agentic workflows to automate RFP submissions for health care supply companies, reducing a process that previously took weeks to a matter of hours and enabling commercial teams to respond to opportunities far more rapidly.

Value layer 2: outcome quality and resilience

Additional value is also created from agentic coordination across domains, leveraging autonomous decision making toward achieving an overall goal—generating better outcomes where decisions are more consistent, better-informed, and more robust across scenarios. Unlike layer 1, the value here is not just speed or cost: it is the quality and reliability of the result, achieved by an agent that can weigh multiple constraints simultaneously and adapt when conditions change.

Example: An investment agent rebalancing a portfolio that must simultaneously satisfy return targets, risk limits, tax efficiency, ESG criteria, and compliance rules—dynamically finding a solution that meets all criteria rather than optimizing for just one, as a rule-based system would.

A less visible but equally important benefit at this layer is the grounding of decisions in systematic, encoded, and auditable workflows. When an agent executes a complex process, every decision step can be logged, made traceable, and reproduced, giving leaders a level of visibility and control that informal, human-driven processes rarely provide. In Allianz Partners' work with Otera, for example, autonomous claims agents are executed under a governed-autonomy architecture, which gives operators full visibility into end-to-end decisions, along with the ability to trace and override actions where needed—turning what used to be an opaque, people-dependent process into one that is explicitly measurable and tunable.

Value layer 3: adaptive, enterprise-wide systems

There is also further value from enterprise-wide adaptive systems that learn in context—resulting in adaptiveness gains (new business models emerge and tap new value pools). While layer 3 remains nascent for most organizations today, the underlying capabilities are assembling rapidly. The organizations that begin building toward it now will be best positioned when it becomes the competitive norm.

To see how these value layers manifest in practice, and how frontier organizations are already beginning to experience step changes in engineering productivity, let's consider the example of an agentic code-review system in a large telecommunications company, alongside recent public statements from Mistral AI.

Code Review: Agentic AI and the Economics of Engineering

One of the clearest business cases for agentic AI emerges in software development, where code review is both critical and chronically expensive in human time. In a multi-agent code-review system deployed for a large telecommunications company's software development life cycle (SDLC), specialized agents take on distinct responsibilities: checking related tickets in the issue tracker, consulting the coding wiki, validating against quality and compliance specifications, and synthesizing prior human reviewer comments on similar changes. Instead of asking a single engineer—or a single agent—to hold all of this context at once, the system distributes the cognitive load across a coordinated agent network that prepares and structures the review before a human ever gets involved.

The business impact is quantifiable. In one implementation, time spent on code reviews fell by 50%, freeing more than 1,500 days of manual effort per quarter. At the same time, quality improved: code rewrites dropped by 20%, eliminating nearly 100,000 hours of rework every quarter. Viewed through the value layers introduced earlier in this chapter, this is a textbook example of layer 1 gains (operational productivity and cost savings) combined with layer 2 gains (better outcome quality and resilience) within a single workflow.

These results are consistent with what frontier AI companies are starting to report more broadly. In a public hearing before the French parliamentary commission on digital vulnerabilities, Mistral AI's CEO Arthur Mensch explained that, internally, engineers increasingly rely on AI assistance to the point where they “no longer write code” in the traditional sense, with productivity reportedly doubling over a few months and, in some cases, achieving order-of-magnitude gains. Taken together, the telco code-review deployment and Mistral's own experience point to the same business conclusion: when agentic systems collaborate with engineers on core knowledge work, effective output per engineer can increase dramatically in a short period of time. For business leaders, the takeaway is clear: agentic systems in domains like code review do more than shave cost from existing processes—they change the economics of throughput, quality, and time-to-market in ways that compound across the entire product portfolio.

To calibrate expectations: even the most sophisticated agentic tools currently available—such as Anthropic’s Claude Cowork, which coordinates multiple subagents to execute complex knowledge-work tasks autonomously across files, browsers, and applications, or open source platforms like OpenClaw, which integrates with dozens of enterprise tools and maintains persistent memory—primarily operate at layers 1 and 2. They automate execution and coordinate across systems toward a defined goal, but they do not yet spawn new business models or enterprise-wide adaptive systems. Layer 3 is visible today only at the frontier: in agentic commerce platforms where AI agents transact with each other, forcing businesses to rethink how they price and market to machines as much as to people,³ and in early signals that leading organizations are beginning to use multi-agent systems to model competitive scenarios and identify new value pools in real time.⁴

Understanding which layer of value is realistically in scope, taking into account an organization’s current maturity, is itself a key strategic judgment. Equally important is knowing in which situations agentic systems are genuinely well-suited, and in which they are not.

Agentic systems are particularly well-suited in this context and value-adding in certain types of use cases, such as:

- Customer support, where agents proactively resolve issues (which is not always the case with GenAI powered chatbots) instead of reactive chatbots simply escalating frustrated users to a human
- Supply chain optimization, where agents dynamically reroute shipments and adjust inventory levels
- Compliance, where agents monitor regulatory changes and continuously self-audit the organization against updated rules

Conversely, agentic systems may create less value, or introduce unacceptable risk, in certain situations, for example:

- Environments where human judgment is highly subjective and individualized (“I like this jacket, but I want the fabric to feel softer”)
- Scenarios where critical information is tacit, fragmented, or impossible to fully encode for an agent, such as complex mergers and acquisitions negotiations, major credit decisions on opaque counterparties, or crisis-management situations where material facts reside in people’s heads or informal networks rather than in systems of record
- High-stakes contexts where the cost of a mistake outweighs operational benefits, such as misinterpreting a compliance rule that could trigger a multi-million dollar fine, or altering settings on a life-support device.

Understanding why the moment is now is necessary, but not sufficient. The harder question for most leaders is not whether to act, but how to think about it: which opportunities are genuinely within reach, where agentic AI creates the most differentiated value, and what it will take organizationally and technically to capture that value responsibly. The rest of this chapter provides a framework for exactly that.

Organizational and Technical Readiness

Agentic systems show strong prospects for value creation and, at the value layer 3 level, open the door to significant market and competitive disruption through new business models. Business leaders should view these opportunities—and the corresponding risk of being disrupted—as a wake-up call to accelerate their organizational and technical readiness, which is a key driver of the business case for agentic AI.

This readiness can be framed across six dimensions:

Leadership alignment

- Do we, as a leadership team, understand the core agentic concepts and their strategic implications, both as opportunities and as threats?
- Have we clearly identified the business pain points or opportunities that are worth addressing with agentic AI?
- For those areas, have we defined clear boundaries between agent autonomy and necessary human intervention?

Technical infrastructure

- Do we have the necessary infrastructure—data, MLOps, APIs—to support our agentic AI agenda?
- To what extent can we still move forward if parts of our infrastructure are not yet ideal, and where are the nonnegotiable gaps?

Governance and risk

- Are we equipped with documented principles to guide our stance on agentic AI—for example, how to balance autonomy with augmentation of human agents?
- How will we ensure these principles are implemented, monitored, and enforced?
- What existing or new committees should oversee these questions, and who is accountable when agents act autonomously?

Architecture

- How will we design modular systems and their orchestration, while ensuring seamless interaction with

legacy systems?

- How will we prioritize which systems to modernize first, and which can wait, given budget and resource constraints?

Technical resources availability

- Do we have the right technical skills in house to design solutions, learn from feedback, and iterate effectively?
- Do we have the skill sets needed to understand the underlying models and deploy agentic scenarios in a friction-free manner?

Change management

- Do we have the right talent, cultural agility, and risk appetite to take advantage of agentic AI?
- Are we prepared to evolve roles, skills, and ways of working as agents become part of our core processes?
- What does it mean for our organization to manage both humans and machine-based agents as a unified “workforce”?

For each dimension, leaders can use a simple RAG (red/amber/green—not to be confused with retrieval-augmented generation) assessment to gauge current maturity: red (we are behind), amber (we are progressing, but not there yet), or green (we are on top of this, or leading). This helps prioritize efforts and sequence investments in a pragmatic, business-driven manner.

Agentic Systems: What Is the Payoff?

Ultimately, the business case for agentic AI can be summarized in the following key points:

Maturity

Agentic AI technologies have reached a level of maturity where they can serve as mainstream business enablers rather than experimental add-ons.

Opportunity

Agentic systems create opportunities for strategic—not merely tactical—impact, shifting the focus from incremental efficiency to transformational effectiveness and adaptiveness. As a result, business leaders must quickly assess how opportunities (or negative developments) could arise from adopting, or choosing not to adopt, such technologies.

Assessing organizational and technical readiness should be approached with a practical mindset: pacing the effort, being explicit about where and how agentic AI can benefit the enterprise, and determining appropriate guardrails. As these technologies evolve rapidly, cultivating enterprise-wide awareness and agility enables a rational approach to value creation while preserving strategic options for the future.

¹ HFS Research, “Otera Helps Allianz Partners Use AI to Target a €300M Profit Increase by 2027,” December 2025; Pieter Viljoen (Chief Data Officer, Allianz Partners) and Stefan Engl (CEO, Otera), “Agentic Transformation in 30 Countries,” Gartner IT Symposium/Xpo, Orlando, FL, October 20–23, 2025.

² Franklin Resources, Inc. (Franklin Templeton), “[Franklin Templeton and Wand AI Forge Multi-Year Strategic Partnership to Advance Agentic AI in Asset Management](#)”, press release, Business Wire, November 6, 2025.

³ McKinsey & Company, “The Agentic Commerce Opportunity: How AI Agents Are Ushering in a New Era,” *McKinsey Quarterly*, October 2025.

⁴ IDC, *FutureScape 2026: Innovation at the Core—How Agentic AI Will Redefine Business Growth* (November 2025). IDC predicts that by 2026, 70% of the CEOs at the world's two thousand largest companies will focus AI ROI on growth and business model reinvention rather than cost savings alone, and that 60% of new economic value generated by digital businesses by 2030 will come from companies investing in and scaling AI capabilities today.

Chapter 3. High-Impact Enterprise Applications

The applications of agents and the multi-agent approach are numerous and go beyond the familiar agentified versions of software applications. Runtime costs, quality limitations, and regulatory limitations notwithstanding, most applications, processes, and organizational systems can be considered for agentic augmentation. Various benchmarks are starting to emerge to be able to compare different agentic systems on a wide variety of use-cases and domains.¹

In this chapter, we will discuss a range of applications that are already deployed as multi-agentic or are good potential use cases to consider. Every system mentioned here has been built and some are already in deployment.

NOTE

Many of the multi-agent systems mentioned in this chapter were actually created and some were deployed using the open source multi-agent platform called **Neuro SAN**, where agentic systems can be easily vibed, and which is recommended for use by less technical readers. Most examples in the chapter can also be created using the many other available platforms, including those provided by hyperscalers and commercial LLM providers as well as open source alternatives such as **LangGraph** and **CrewAI**.

Agentic Software Development Life Cycle

In addition to textual content from the web, LLMs have been trained on large sets of public code repositories, which has given them the power to understand, debug, and write code in many coding languages. This ability is quite amazing, as it emerges from training the model to predict the next token in a string of tokens. When you give an LLM part of your code and

ask it to find the bug, it does so, often successfully, without actually running the code.

LLMs, as models and without being part of an agent, are evaluated on many coding benchmarks out of the box, and do quite well on them. Amazingly, once we wrap the same LLMs into an agent and give it the tools to run the code it generates, say in a container, it will do better on the same coding benchmark. In other words, the agent equivalent of the same LLM, given the proper tools, is better at coding than the LLM alone. This is because the agent can feed the runtime results of the code back into the LLM, so, if the LLM needs to, it can iterate on fixes before responding with the completed work.

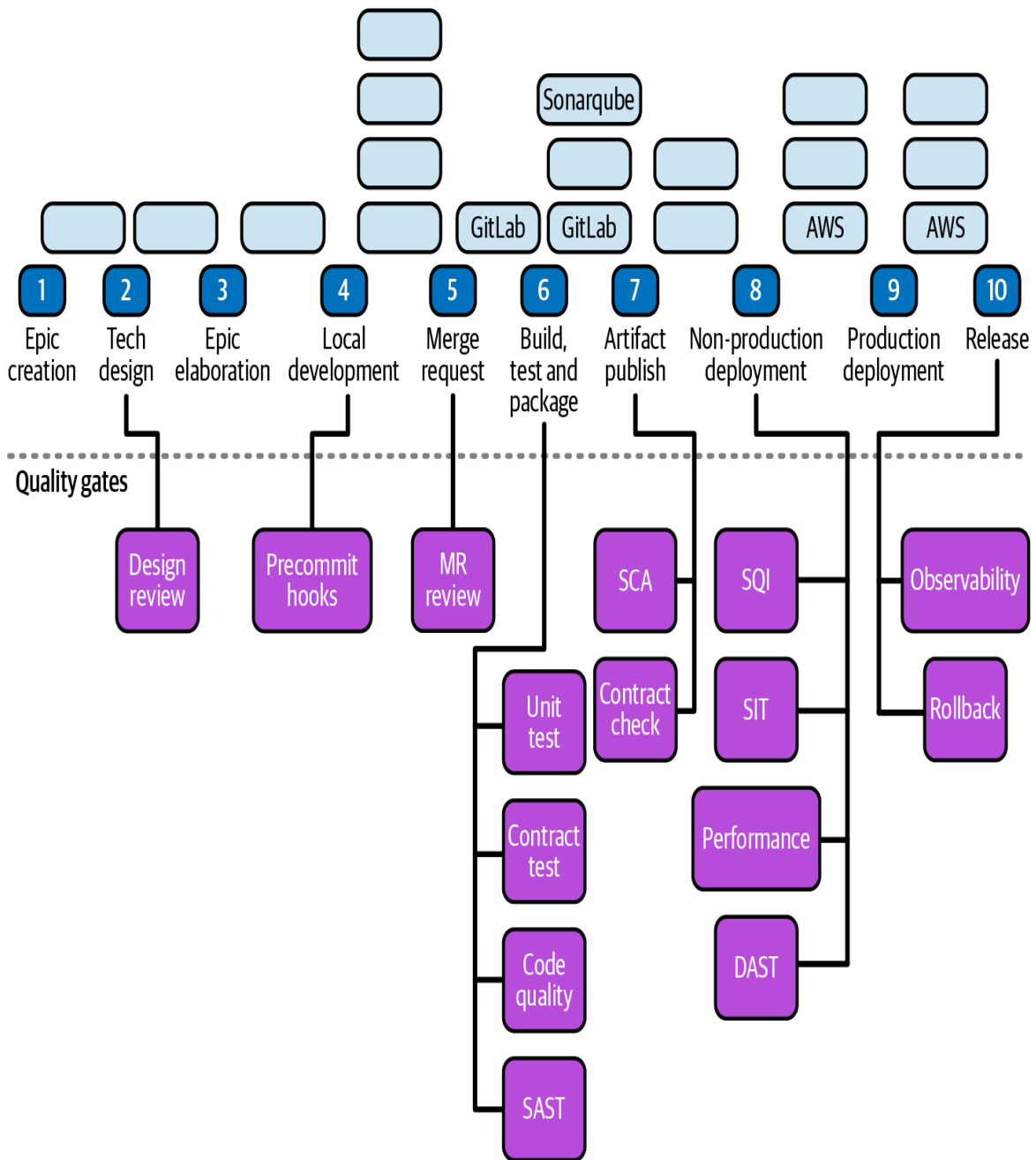
A common and successful use of AI agents is in making programmers more productive. Coding copilots write code snippets based on coder prompts or comments. These systems often include a chat sidebar for coders to consult with the AI on various aspects of their code.

Vibe coding systems, also known as generative development harnesses, take this to the next level, significantly reducing or even eliminating the need for human programmers to write any code. With these systems, which were introduced in 2025, the programmer simply describes the project, and the AI agent writes the code in an iterative and interactive manner.

More recently, *autonomous coding agents* have been introduced, with specific roles to play as part of the project team, ranging from test case generation to code review and other project tasks. These agents behave similarly to human coding team members, picking up tickets from the project kanban board and moving them forward to final pull requests.

The moment we have more than one such agentic team member deployed, we are in the realm of multi-agent SDLC.²

Figure 3-1 shows such an SDLC process along with the agentic augmentation points.



Agentic AI

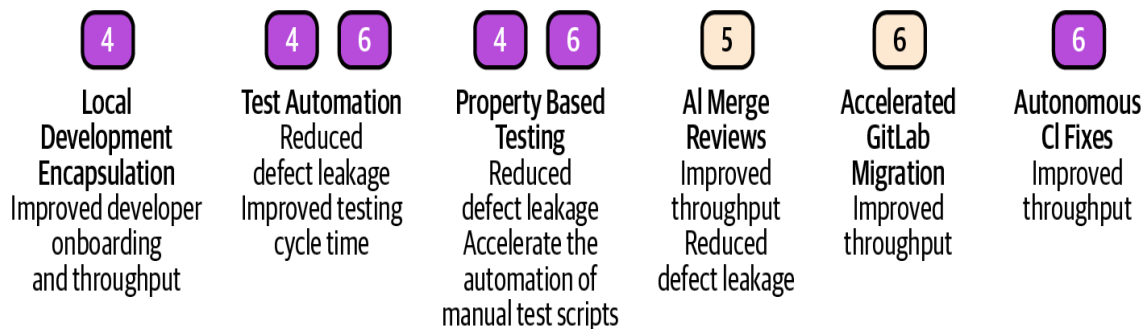
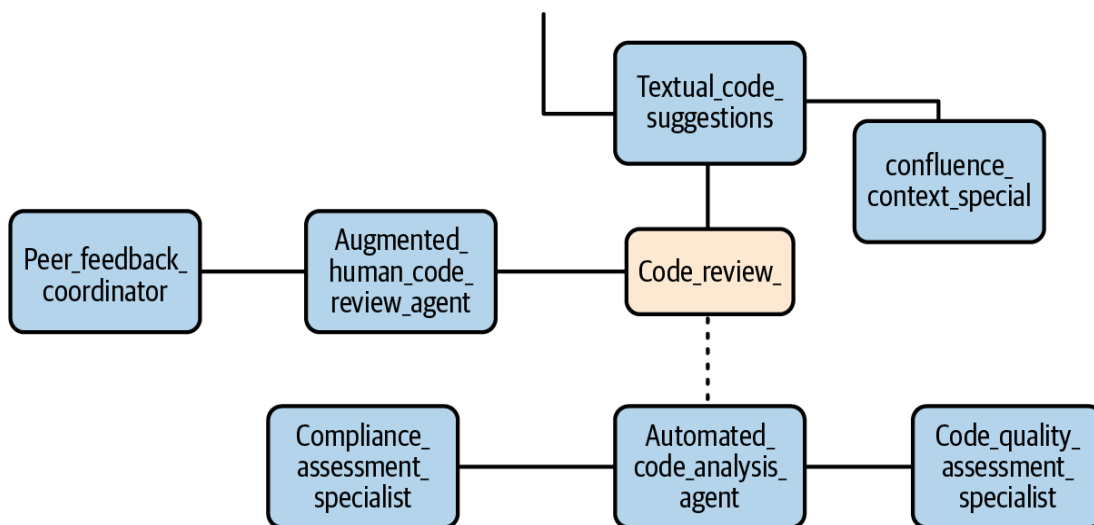


Figure 3-1. Agents in an SDLC.

Most companies have an SDLC process that has been tailored to their software needs, and so an incremental introduction of AI agents allows for a more gradual and customized transition to automation. A good place to start is with aspects of the SDLC process that are less central to the core operations. For instance, multi-agent systems do well in the code review process (see [Figure 3-2](#)).



Chat Internal Chat SlyData Config

Chat

User

Analyze the code in my latest commit and provide automated feedback based on our code review standards.

Babak/code_review_

The automated analysis of the code in your latest commit based on code review standards has been completed. Here's a consolidated feedback report:

Code Quality and Readability

- **Readability:** Ensure variable names are descriptive, follow naming conventions, and maintain consistent styling with proper indentation.
- **Comments:** Comments should be clear and meaningful, explaining why specific approaches were taken.

Maintainability

- Ensure the code is modular and follows the Single Responsibility Principle. Refactor duplicated code into reusable functions.

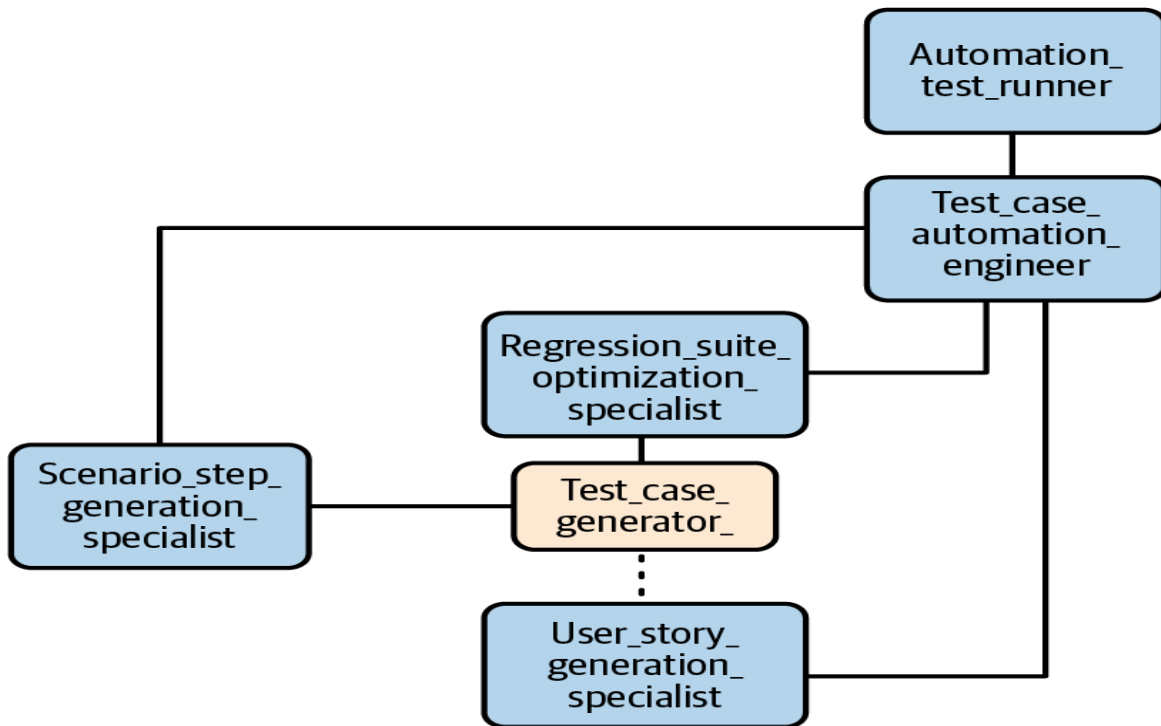
Performance

- Analyze critical code paths for potential inefficiencies and optimize loops and data processing strategies.

loops and data processing strategies.

Figure 3-2. A multi-agent system for code review along with a related test command. The agentic system consists of agents that are specialized at checking related coding tickets in a ticketing system, pages in a coding wiki, quality assessment and compliance specs, as well as agents bringing in the human reviewer perspectives from comments on the code review by other reviewers.

Similar augmentation can easily improve code coverage for test case generation (see [Figure 3-3](#)).



Chat
Internal Chat
SlyData
Con

Chat

User

Generate test cases from this user story about the new login features.

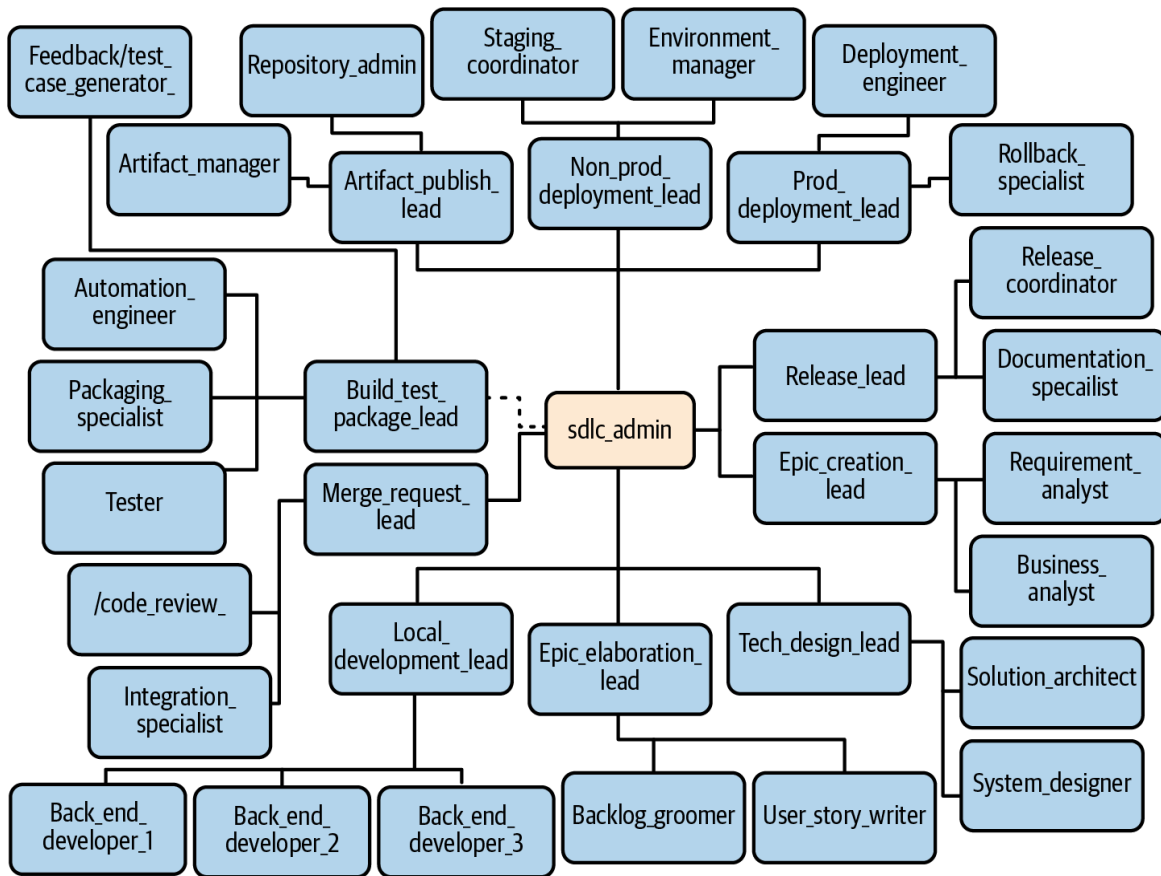
Babak/test_case_generator_

Here are some sample test cases generated from the user story about the new login features:

- Test Case: Verify Successful Login**
 - Description:** Validate that a user can successfully log in with valid credentials.
 - Steps:**
 - Navigate to the login page.
 - Enter a valid username.
 - Enter a valid password.
 - Click on the 'Login' button.
 - Expected Result:** User should be redirected to the homepage/dashboard.
- Test Case: Verify Login with Invalid Username**
 - Description:** Check the system's response to an incorrect username.
 - Steps:**
 - Navigate to the login page.

Figure 3-3. A multi-agent test case generator shown in a test environment, with an actual command and response coming through the communication and consolidation of responses from the agents involved.

Once these systems have been tested in a sandbox setting and deployed to augment the SDLC process of a company, they can be plugged into a larger multi-agent system that covers many more aspects of the process, even allowing complex commands that require multiple agent subnetworks to coordinate with one another (see **Figure 3-4**).



Chat



Review the latest pull request and code review for the new login feature and create a suite of test cases to validate its functionality and integration with existing systems.

Babak/sdlc_for

To review the latest pull request + code review and tailor a test suite to what actually changed, I need a few specifics:

1. **PR details:** repo + PR/MR link/number, source/target branches (or commit SHA)
2. **Code review threads:** link/export/paste of review comments (esp. flagged risks)
3. **Login scope:** auth type (username/password, OIDC/SAML SSO, MFA), plus acceptance criteria
4. **Integrations touched:** IAM/IdP, user-profile/roles, session/token store, audit/SIEM, rate limiting/WAF, notifications/analytics
5. **Test context:** environments/URLs, feature flags, IdP sandbox details, test accounts (valid/invalid/locked/disabled/MFA-enrolled)

While you gather that, here's a **baseline, automation-ready test suite** for a "new login feature" that you can map to the PR diff and review comments.

Test suite: Login feature (functional + integration)

A. UI/UX & Navigation (if applicable)

- **LGN-UI-01** Load login page: renders form, required fields, links (forgot password/SSO), no console errors.
- **LGN-UI-02** Form validation: empty submit shows inline validation; invalid email/username format handled.
- **LGN-UI-03** Password masking + reveal toggle works; does not persist in logs.
- **LGN-UI-04** Redirect after login: returns to originally requested protected page.
- **LGN-UI-05** Accessibility: labels, tab order, focus management, ARIA on errors.

B. Core Authentication (positive/negative)

Figure 3-4. A multi-agent SDLC shown in a test environment, along with a sample command and actual response. This system makes calls to agent networks shown in Figures 3-2 and 3-3. The slash symbol (/) denotes calls to other multi-agent systems within the ecosystem. It is generally a good idea to create broad agentic systems like this and incrementally add grounded, verified agentic subnetworks to it as they become available.

Code and App Migration

The code-writing abilities of LLMs make them good candidates for augmenting and automating code migration tasks. Business apps in many companies run on legacy systems that are in dire need of upgrades to more modern equivalents, but this task is costly and time consuming, and highly manual.

Many legacy systems have little to no technical or functional documentation, making manual understanding extremely difficult. Critical logic and dependencies are often buried in legacy layers, only surfacing late in development, causing delays and rework. Manual reverse engineering and rationalization at a large scale is highly error-prone. All of this means that conventional migration approaches have struggled to meet timelines while maintaining accuracy, traceability, and functional parity.

Initial attempts to use LLMs for code conversion did not live up to expectations. The limited context window of these models meant that the legacy code repository had to be fed in one slice at a time, and without the LLM understanding the entire outline and logic of a system, converting its parts simply was not feasible without major human involvement. Far from being automated, these systems degraded to code-assist tools for human code migrators.

Agentic systems, built on the hierarchical blueprint of legacy code or application layers, solve many of the problems noted previously. At the top of the hierarchy, an agent with knowledge of the overall perspective and goals of the legacy system orchestrates agents, each with a smaller and more focused scope of operation. These systems typically work by first reviewing the legacy code and distilling it into an intermediate

representation, often rich with natural language explanations, which is then used as a template for generating the target code or app layer configuration.

This approach is also used for app migration. Imagine a system built on an old and outdated business reporting software that contains a multitude of reports in daily use. The legacy platform lacks modern business intelligence (BI) features and functionalities leading to data integration and management issues, limited scalability and performance, poor user experience, and accessibility issues. The goal of an app migration then is to modernize legacy technology to take advantage of the latest innovations, improve operational efficiency, and optimize both cost and performance.

An agentic system to address such a migration would automate deep code analysis by extracting metadata, complex logic, dependencies, and business rules from legacy reports. This knowledge supports agents with consistent, context-aware execution augmenting human domain experts, and incorporating user validation and feedback to ensure accuracy and relevance. The validated report logic, in turn, informs the creation of visuals, and semantic models—that is, the business-facing data layer defining metrics, dimensions, relationships, calculations, and access rules aligned with modern BI standards—which are then handed off to agents that automatically generate BI reports from approved designs using a modern reporting system.

Agentic code/app migration systems can bring significant cost savings of 40–50%, reducing migration efforts by more than 50% and resulting in **substantial cost efficiency**. **Figure 3-5** shows gains achieved in one such real-world application of app modernization for an SAP BO to Power BI migration project.

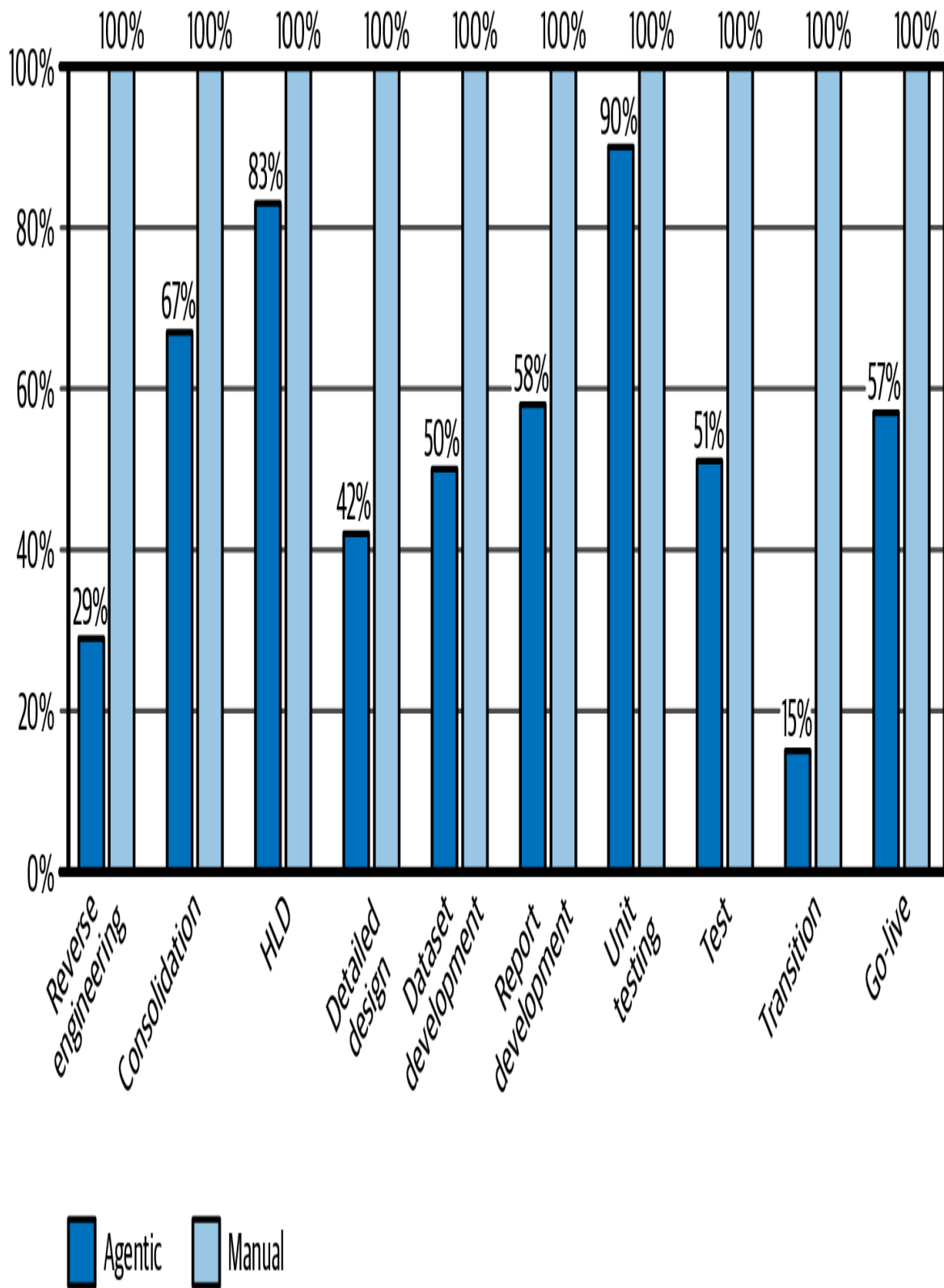


Figure 3-5. Effort savings across different phases of the modernization journey.

An important aspect of this approach is the intermediate representation, which not only significantly improves the migration quality but, as a byproduct, can also help humans and agents in future enhancements and migrations.³

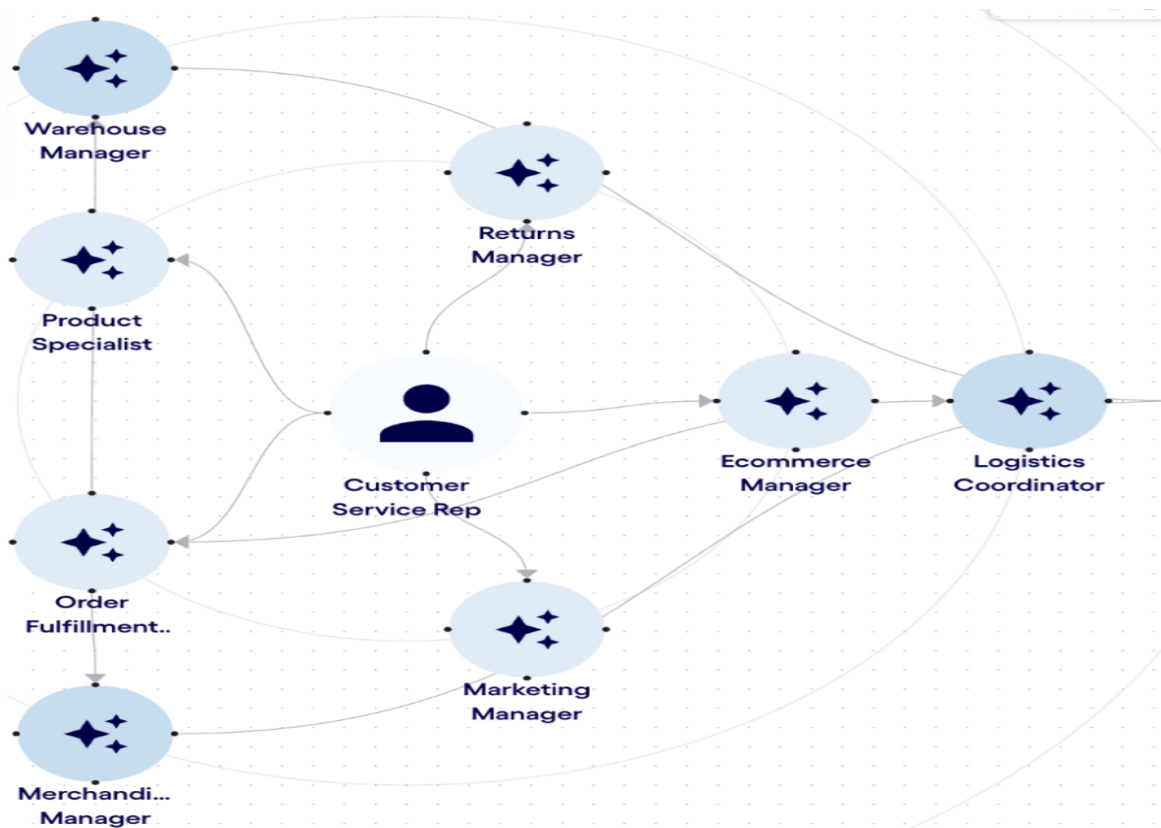
Sector-Specific Examples

What sets different businesses apart are the unique organization and processes they use that have been designed to match the specific domains within which the business is operating. This demands agentic systems tailored to specific domains, and sometimes, when the process is what differentiates a company, tailored to the specific company itself. In this section, we will cover a range of business-specific use cases to help illustrate the diversity and breadth of agentic use cases across industries and businesses.

Retail

Retail support lines are an obvious use case for multi-agent systems. How often have you called a support line with what should have been a simple request, only to end up spending time on hold as you were transferred from one department to the next? The time spent on such calls seem to be a reflection of the siloing in our organizations, which results in inefficiencies and unhappy customers.

Figure 3-6 shows a multi-agent system designed to span across these silos.





Need to return a pair of jeans I bought at your store. I'd like to buy a different style replacement with some accessories online instead.



Retail Ops And Customer Service

Final Answer

To proceed with your return and purchase, please provide the following information:

- **Return Details:**
 - Original purchase receipt or order number.
 - Reason for return.
 - Condition of the jeans (unworn, tags attached, etc.).
- **Replacement and Accessories:**
 - Specific style or model of the jeans you wish to purchase.
 - List of accessories you want to add to your order.

Once I have this information, I'll guide you through the return process and assist with placing your new order online.

Chat with Retail Ops And Customer Service

Figure 3-6. A multi-agent system for a retail support line.

Of course, retailers would also benefit from agentic systems for their internal operations, connecting and operating functions including supply chain, loyalty programs, fraud, returns, pricing, marketing campaigns, reporting, and customer experience, to name a few. **Figure 3-7** shows a multi-agent system developed with a department store in mind.



Figure 3-7. A multi-agent system covering many internal departments and functions for a department store retailer. The agents specializing in different roles within the process are represented on the left-hand side, and an actual exchange between a business user and the system is shown on the right. Many agents were involved in authoring the response, but this is transparent to the user.

Telco

Not all multi-agent systems are designed to run in a query or command response mode. An interesting design pattern emerges when agents are expected to run proactively and make local decisions autonomously. In Figure 3-8, the leaf-node agents are running entirely on the network node

machines. This gives the telco a latency advantage for decisions that need quick response times and can be handled with local knowledge of the node's operations. This means that the leaf-node agents are running on-premise, often open source, LLMs, and there is no need for an API call to a hosted LLM for them to operate.

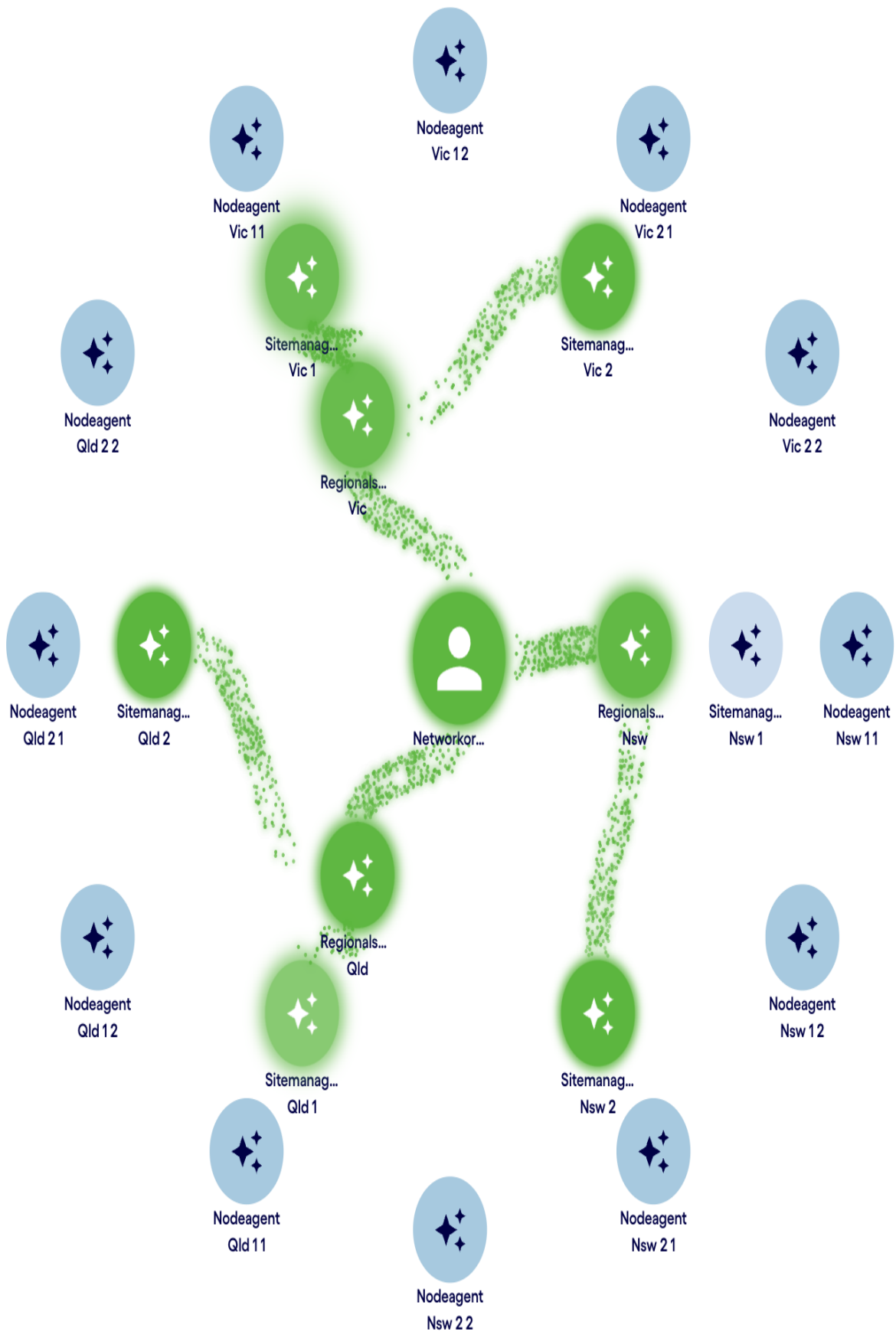


Figure 3-8. A telco agent network with leaf node agents running on the network nodes. The green circles denote agents involved in responding to a specific context, and the connections denote communications between agents for handling a specific case.

The other agents in the hierarchy, on the other hand, represent consolidation points that take wider regional context into consideration in their decision making. This is especially useful when regional or system-wide network balancing choices need to be considered.

Here is an example exchange with the network to generate a consolidated report:

User: Give me a report on severe incidents over the past hour and mitigation steps taken.

Agent: Here is a report on severe incidents over the past hour and the mitigation steps taken across various regions:

New South Wales (NSW)

1. Temporary Spike in Traffic Latency and Packet Loss:

- Time: 14:15 AEST
- Impact: Temporary spike due to an unexpected surge in data traffic.
- Mitigation Steps:
 - Traffic rerouted through alternate pathways.
 - Dynamic load balancing implemented.
 - Additional CPU and memory resources allocated.
 - Result: Situation stabilized, normal operations resumed.

2. High CPU Load:

- Time: 10:15 AM
- Impact: CPU load spiked to 90%, causing potential slowdowns.
- Mitigation Steps: Non-essential processes paused, and traffic rerouted.
- Result: CPU load normalized within 10 minutes.

3. Increased Packet Loss:

- Time: 10:45 AM

- Impact: Packet loss increased to 15%.
- Mitigation Steps: Bandwidth adjusted and additional capacity allocated.
- Result: Packet loss reduced to under 2% within 5 minutes.

Victoria (VIC)

1. Spike in Traffic Latency:

- Time: 14:30
- Impact: Due to increased load from local events.
- Mitigation Steps: Rerouting and additional bandwidth allocation.
- Result: Latency returned to normal within 15 minutes.

2. Packet Loss and CPU Load Increase:

- Time: 15:00
- Impact: Due to a malfunctioning switch and application.
- Mitigation Steps: Faulty switch isolated, traffic redirected, and application throttled.
- Result: Packet loss reduced and CPU load normalized.

Queensland (QLD)

1. Spikes in Network Latency and Packet Loss:

- Impact: Due to surges in traffic from events.
- Mitigation Steps: Rerouting, bandwidth optimization, and increased resource allocation.

2. High CPU Load:

- Times: 10:15 AM and 11:23 AM
- Mitigation Steps: Redistributed tasks and scheduled task to clear processes.

3. Increased Traffic Latency:

- Time: 10:45 AM
- Description: Due to a routing loop.
- Mitigation Steps: Temporary route bypass.

4. Increased Packet Loss on Brisbane West link:

- Time: 11:45 AM
- Mitigation Steps: Adjusted routing protocols.

Overall, all incidents have been effectively managed and network performance is returning to normal levels. Monitoring has been intensified to prevent recurrence.

A different usage for this system is to give top-down network rebalancing commands. For example:

User: There will be a high bandwidth event in the Sydney area tomorrow from 4–5 pm. Suggest load rebalancing steps based on current and predicted network status.

Agent: Here are the suggested load rebalancing steps for the high bandwidth event in the Sydney area tomorrow from 4–5 pm, based on current and predicted network status:

Current Network Status:

1. Node SYD01: CPU load at 65%, Memory load at 60%, Latency at 23ms, Packet loss at 0.5%.
2. Node SYD02: CPU load at 70%, Memory load at 62%, Latency at 21ms, Packet loss at 0.4%.
3. Node SYD03: CPU load at 55%, Memory load at 58%, Latency at 19ms, Packet loss at 0.3%.

Predicted Network Status (4–5pm):

- Anticipated increase in CPU and memory loads by 20% due to the event.

Load Rebalancing Steps:

1. Node Prioritization: Temporarily prioritize traffic through Node SYD03, as it currently has the lowest load and latency.
2. Traffic Redistribution API Call: Use the API call `api/v1/traffic/distribute` to adjust traffic loads. Parameters: {
“source”: “SYD01,SYD02”, “target”: “SYD03”, “percentage”: “20” }.
3. Dynamic Load Adjustment Protocol: Implement protocol DLA-2023 to dynamically adjust traffic distribution based on real-time load metrics.
4. Backup Node Activation: Prepare Node SYD04 as a backup to handle any unexpected load spikes. API call:

api/v1/node/activate with parameters { “node”: “SYD04” }.

Monitoring and Reporting:

- Continuously monitor the CPU, memory, and latency metrics during the event.
- Report any anomalies to the SiteManager immediately through the monitoring system.

Post-Event Evaluation:

- After the event, use api/v1/network/rebalance to revert any temporary adjustments and restore normal operating conditions.

These steps are designed to effectively manage the increased load during the event and ensure stable network performance.

Incidentally, the same multi-agent design pattern can be used for use cases where the frequency of decision-making is not as high. **Figure 3-9**, for example, shows such a multi-agent system for an agriculture business, with agents running proactively and semi-autonomously on the farms, reporting through a regional and state hierarchy back to the central command.

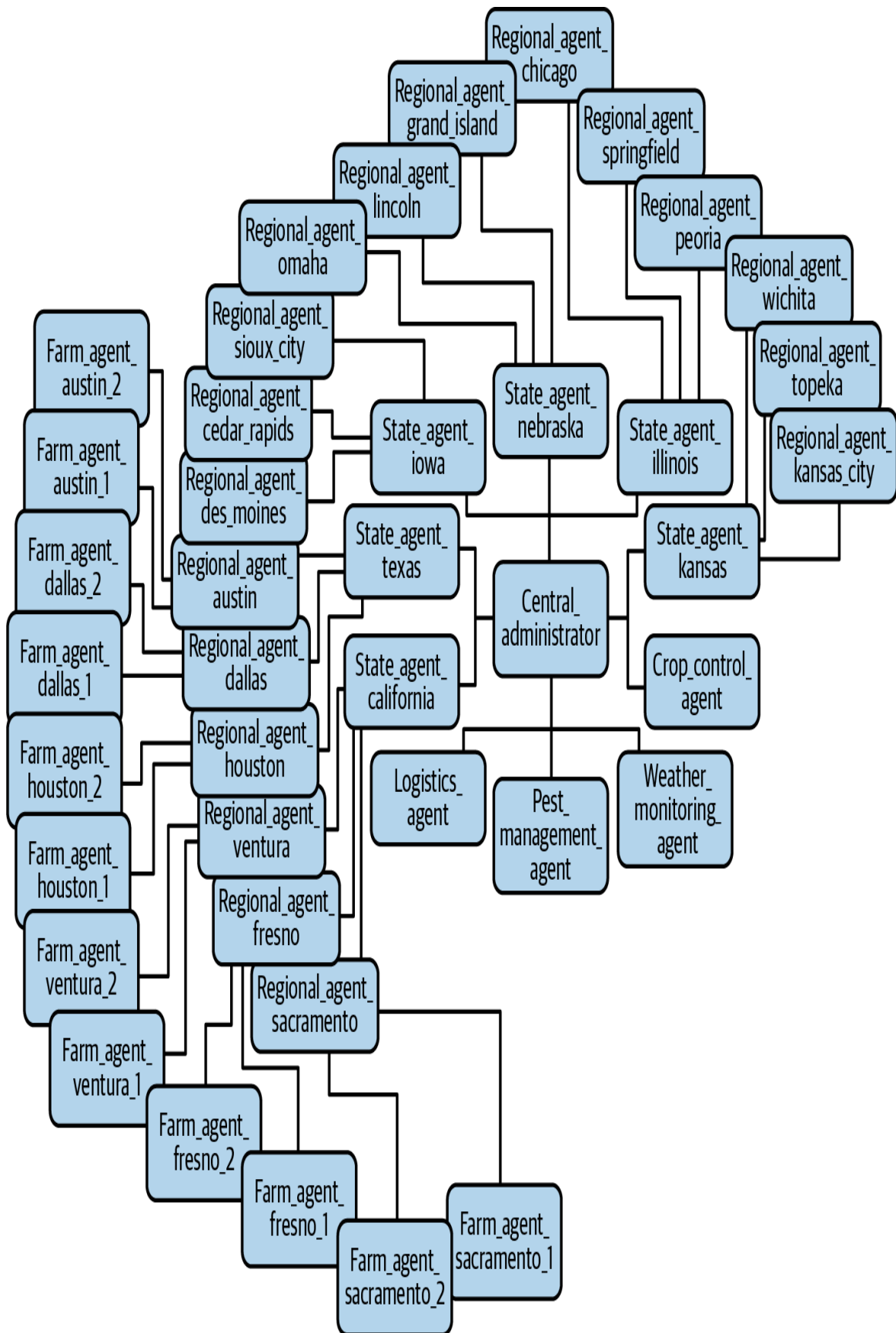


Figure 3-9. A multi-agent system for a multi-farm agriculture business. Leaf-node agents are running continuously on the farms, while command and control and reporting is handled over a hierarchy of agents connected to the leaf nodes.

Finance

While the banking and finance sector has historically been cautious when it comes to regulated decision-making contexts like risk, compliance, and customer-facing outcomes, it was one of the first to experiment with LLMs and has been quick to bring in multi-agent AI systems.

In [Chapter 2](#) we talked about agentic RPA. Most deployments in retail banking, for example, have been rigid agentified RPA processes (see [Figure 3-10](#)); however, some have gone beyond agentic process automation relying more on a pool of agents being able to run processes as well as spontaneously coming together to handle complex or previously unencountered requests, with human-in-the-loop and sometimes human-on-the-loop oversight.

In [Figure 3-10](#), a banking customer experience handling fraud and disputes for the back office is agentified as an agentic process automation (APA) system, basically replacing some nodes in an RPA system with agents. In this example, the inclusion of agents into the process reduces customer effort, delays, and complexity by automating the onboarding process. The agents are designed to operate in an outcome-based mode, making intelligent allocation decisions in a context-aware manner. Finally, the process is quite transparent, with the agents able to fully explain what steps they are taking and why.

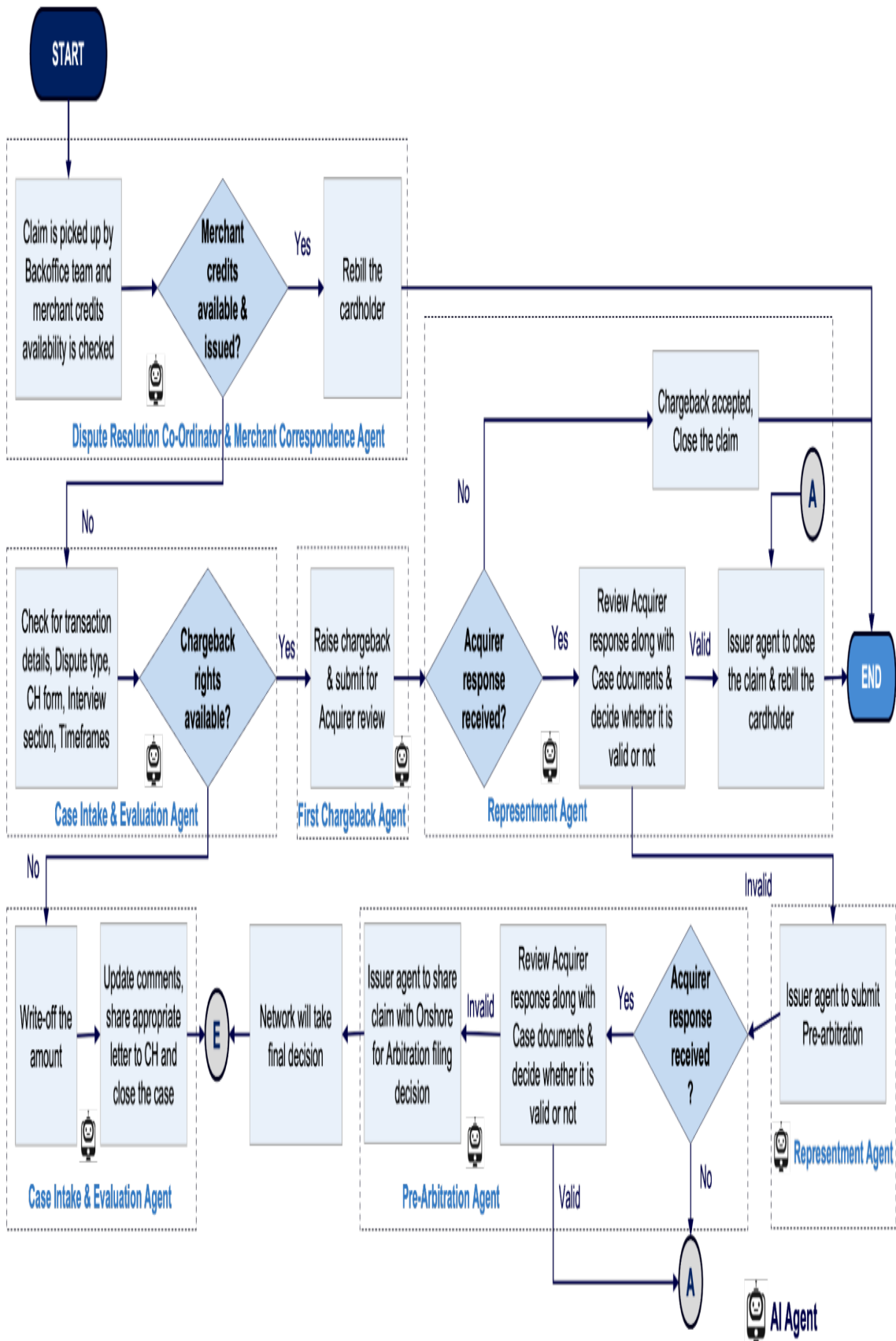


Figure 3-10. A rigid banking process implemented as an agentic RPA (aka, APA).

Interestingly, sometimes these agents still have to deal with legacy processes, such as photos of picture IDs or scanned hand-filled forms. Multi-modal LLMs offer a state-of-the-art option here as well: not only can an LLM at the heart of an agent understand natural language, multi-modal LLMs can be used as tools to read and extract information from unstructured sources like images and scanned PDF forms. In the previous example, a charge-back claim can be automatically reviewed and assessed against related documents.

As will be the case in most B2C systems, in the future users will have personal financial agents hosted on-device or in the cloud, that will act on their behalf—optimizing savings, negotiating credit rates, or making payments. Banking will become invisible and ambient—fully embedded into daily life, activated contextually by needs. Instead of apps, banking services will be triggered by events such as buying a home, going on a trip, or selling a car, and mediated through smart contracts and personal agents. When planning a trip, for example, a customer’s agent will auto-arrange travel insurance, currency exchange, and budgets based on the user’s goals, preferences, and constraints. The bank will become a platform for programmable finance, allowing users and businesses to create custom financial workflows using AI agents, akin to “financial Lego blocks.” A small business owner, for example, will be able to set up programmable payroll to automatically distribute salaries, taxes, and savings contributions based on incoming revenue.

Here’s a conceptual scenario, which is far from science fiction, given today’s multi-agent technology.

Mark rarely checks his portfolio—markets make him anxious. His agent knows this and engages only when it matters.

Agent: Hi Mark, your portfolio earned \$240 this week, mostly from dividend income. No action needed. Want to see a simple snapshot?

The agent calibrates frequency and tone based on Mark's preferences and avoids jargon. It balances engagement with calm reassurance, explains relevant issues in ways that Mark can understand, enables exploration of alternatives, including use of investment vehicles not provided by the current advisor. Based on interactions, the agent develops a more nuanced understanding of Mark's profile and adjusts timing and content of messages accordingly.

When Mark receives a year-end bonus, the agent prompts: Would you like me to add \$5,000 to your retirement fund? Based on your goals, I suggest 70% to your municipal bond sleeve and 30% to dividend stocks.

It instantly analyzes his risk profile, glide path, and future income projections. Mark taps "Yes" once—done. However, he can undo the move anytime, but he rarely does.

Markets drop 8% in a week. Mark's agent doesn't overwhelm him: You're still on track. Your buffer assets absorbed most of the drop. No action needed.

The agent provides guidance on things to watch out for over the coming weeks and is able to engage in what-if analysis in terms that are understandable to Mark.

But if stress indicators rise (e.g., multiple follow-up queries, higher than usual requests to go through what-if scenarios), the agent offers options to adjust equity exposure temporarily (e.g., simple rebalancing to lower risks without locking in losses) and/or schedule a 15-minute call with his human financial advisor.

One day, Mark casually asks: What if I retire at 63 instead of 65?

The agent replies: You'll likely need an extra \$80K. If we increase savings by 5% and reduce discretionary spending by \$150/month, you'll close the gap in 24 months. Want to try it?

Behind the scenes, the AI updates financial models, stress-tests against inflation scenarios, and proposes a new action plan—all in seconds.

When a duplicate advisory fee hits Mark's account, the agent detects it, files a claim, and refunds the amount.

Mark receives a quick message: We spotted a double charge from Oaktree Advisors. I've refunded you \$150 and confirmed with the firm. No action needed.

When Mark's spending patterns shift (e.g., new medical expenses), the agent flags it and suggests a review with his human advisor—only if warranted. Mark clicks “Remind me next month.”

Health Care

Assigning expertise to agents in a multi-agent system and having them discuss a patient's medical context results in dependable medical advice. This is especially useful in cases where providing timely medical supervision to practitioners is unavailable or impractical. Let's use the case of therapy supervision as an example. In **Figure 3-11**, a multi-agent supervision system is set up with a group of agents as experts in a number of different therapeutic disciplines. Just as in a typical therapy supervision setting, a therapist brings a description of their client case report for consultation. The group of agents then reviews and discusses the case, each bringing their expertise to the table, and together they produce a consolidated treatment plan for the therapist to consider. Note that in such systems, patient confidentiality and other health care regulatory compliance measures are of utmost importance. Experiments show that human and AI therapy supervision is interchangeable.⁴

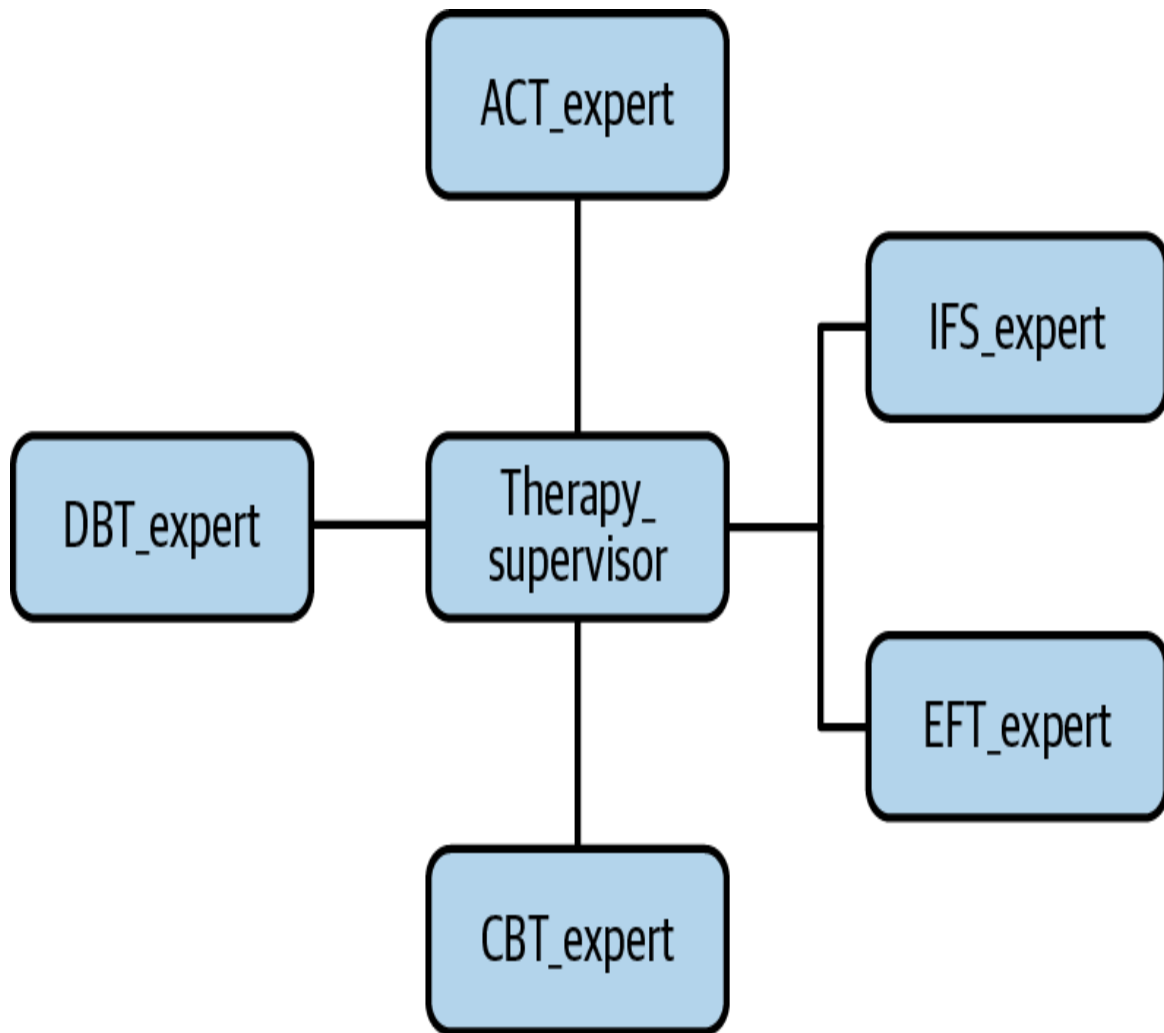


Figure 3-11. A multi-agent system for therapy group supervision, with agents representing different therapeutic theories, working together to provide supervision on a given vignette presented by a therapist.

Cross-Functional Applications

In this section we will look at some use cases that are applicable across industries and in most businesses.

Research

Much research in the field of AI is about coming up with a new algorithm and trying it out on established benchmarks. In other words, it is a closed system with built-in tools. As long as the agents are able to run and test

what they create, then we can set them out to autonomously push the boundaries of science.

Searching for a faster and better algorithm can be fully automated, for example, if we have a way to measure the relative speed and quality of a given instance of that algorithm. In this manner, agents can work together to try to create new instances, learning from what worked and what didn't work, to improve the quality and speed of prior algorithms they came up with.

For example, most recently, **meta-harnesses** have been introduced that run populations of agentic harnesses on a benchmark and iteratively improve the harnesses by suggesting variations for the next generation. These suggestions are being generated by the best harness itself, introducing an interesting recursion.

Not all fields have these fortunate properties, however. In most cases, innovations and new theories will have to be tried out in the real world, and so there is a need for autonomous agency over a wet lab, or some physical environment, or the real world. This means that the use of agentic systems in research often requires humans in the loop.

In fact, it has been shown that humans and AI agents work best when they build on each other's expertise and ideas.⁵ By systematically distilling, recombining, and refining diverse expert solutions, AI agents can not only synthesize knowledge, they can discover strategies that surpass human or AI capabilities alone.

HR

Most aspects of the HR process can be agentified. Candidate resumes can be matched with job openings, and points of difference can inform assessment scripts that aid human or agent interviewers. Note that for initial vetting of technical openings, the interview agents can even pose coding tasks and vet candidate submissions during the interview. Agents can then produce onboarding plans and help guide new employees on skills they would need for their respective jobs, as well as training them on various

benefits and processes within the company. It must be noted that candidates are also often using agents to help with their resumes, and even, in spite of bans, to guide them in real time in online interviews.

An interesting starting point for companies setting off on their agentification journey is the employee portal or company intranet. The IT department typically builds a nucleus of agents representing the various functions in a company, including agents representing various apps and knowledge stores used by employees. For larger companies, various departments enjoy a degree of autonomy in building or procuring their own enterprise software. To maintain this, the IT department should define processes, including sandboxing, verification, governance, and security, for internal or third-party agentic systems that are owned by the various departments, thus providing a process for such agents to plug into the larger agentic system, making them centrally discoverable and reducing the siloing effect and bureaucracy typical at this scale.

Figure 3-12 shows such an agentic system. Note how the query is routed correctly to more than one down-chain agent, and a team of agents is formed spontaneously to respond to it, while the employee is interacting in a single, consolidated, and very natural dialog thread, with the organizational complexities being transparent to them. In the example, to address the employee query, both the Legal and HR department agents were consulted.

NOTE

It is important to note that at the time of writing this book, no standard has emerged for agent registries, and so there's no protocol for agents to find, identify, and communicate with each other, even within a controlled enterprise setting. Central registries used for ongoing monitoring of agentic systems can also be used for the agentic discovery process; such a standard and platform are bound to emerge soon.

Figure 3-12. A multi-agent system as the company intranet.

An expanded version of this agentic system, called **1C**, was deployed at a large services company, rolled out gradually as of July 2025. The impact was immediate and can be seen reflected in the number of incident tickets employees opened before and after the rollout (see **Figure 3-13**).

Incident tickets

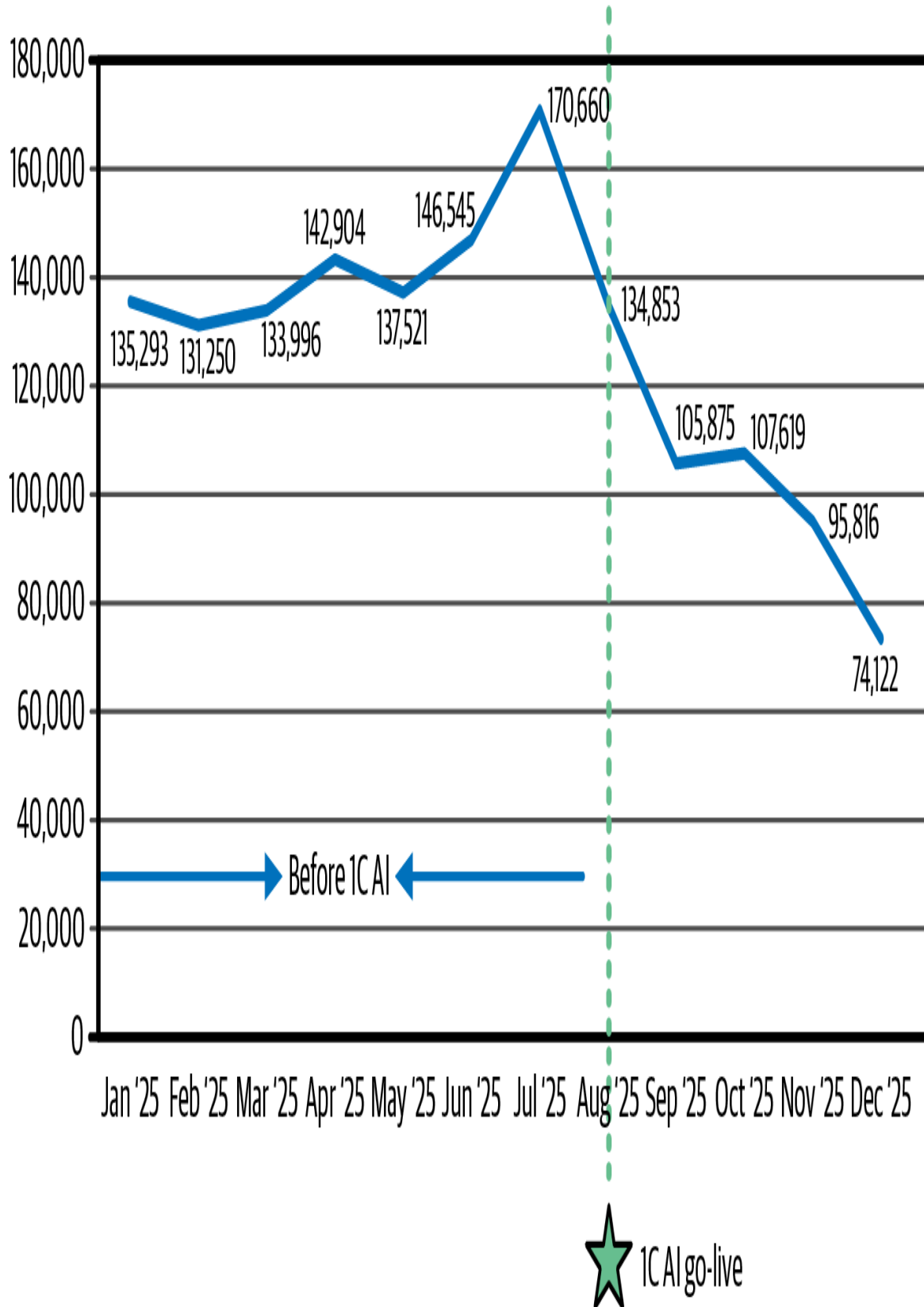


Figure 3-13. Impact of rollout of a multi-agent employee intranet on the number of incident tickets opened at a large services company.

Marketing

In marketing, agentic AI is already changing how campaigns are run end to end. **Higgsfield**, a start-up focused on agentic marketing, provides a stack where specialized agents handle core steps of the campaign loop: scanning what performs in the market, recreating effective patterns within a brand's guidelines, and generating channel-specific creatives directly in a chat interface. The marketer's role shifts from manually producing and adapting assets to briefing agents, setting brand rails, and deciding what goes live.

Higgsfield's approach has been described by Reuters as integrating third-party models into an end-to-end platform with a proprietary "reasoning engine" that chains multiple AI systems together to maintain character and brand consistency in marketing videos. **Reuters** also reported that social media marketers account for about 85% of Higgsfield's usage.

This agentic loop can extend from market research all the way to paid media execution. In one deployment, an entire paid ads program ran as an end-to-end pipeline driven by collaborating agents: a research agent used tools like Claude Cowork to analyze competitors' ads, Higgsfield's MCP generated new creatives, assets were uploaded automatically to the ads platform, a Meta-connected agent launched and monitored campaigns, and winning variants were scaled on a recurring weekly schedule. What previously required a team of junior marketers—assembling creatives, launching tests, checking dashboards—was recast as a largely automated workflow overseen by a small number of strategists.

The business impact is a step-change in both throughput and learning speed. A small team can run and iterate far more campaigns than before, while campaigns continuously adapt to live performance data and remain on brand because the constraints are encoded in the agents' prompts and guardrails. At the same time, the nature of marketing work evolves: execution-heavy roles become easier to automate, while senior marketers

gain leverage by designing the systems, rules, and positioning that guide what the agents produce. As a high-impact enterprise application, agentic marketing with Higgsfield shows how agents can take over the execution loop in a complex, fast-moving domain, freeing human teams to focus on strategy, taste, and higher-order decision making.

In July 2025, as we were discussing various multi-agent solutions for the marketing team of a major bank in Europe, we were challenged with an interesting question: “How should we market to AI agents?”

Increasingly, agentic systems representing consumers will be interacting with agentic systems initially designed for use by humans. A **growing percentage** (47%) of consumers are now trusting AI to assist them with their decisions by presenting them with options to choose from.

Figure 3-14 shows an actual multi-agent consumer assistant dialoging with the user, and conferring with multiple networks of agents, each responsible for a decision domain. The categories of decisions cover health care, careers, finances, retail, travel, and lifestyle decisions like relationship guidance and exploring hobbies.

These agents are all designed to be aligned with the user, collaborating to provide the best options and choices. Agents responsible for researching price and availability of offerings, for example, can contact networks of agents representing various travel businesses. The travel destination researcher agent, for example, will “talk” to various agent networks representing different travel providers (Expedia, Booking.com, Airbnb, etc.) and request available options that best fit the user’s requirements.

You can ask the system to “find options for accommodations in Santa Cruz for this weekend, somewhere near the wharf. The whole trip can’t exceed \$400.” The consumer agents often negotiate with the business agents, sometimes going back and forth several times with the various providers, to find the best options available.

sponsorship money from any business it reviews. Will we have the equivalent consumer agentic system in the future? Would consumers be able to distinguish and use independent consumer agent networks for their decision-making needs? We would certainly hope so.

IT

The field of IT is one of the first adopters and beneficiaries of agentic systems. There are plenty of IT agent systems commercially available for customization and deployment, and use cases abound. Most processes in IT follow rigid, predefined steps, and so agentifying them as APA systems comes naturally. Connecting various IT agents in a more flexible manner can streamline many processes and reduce employee overhead.

As mentioned before, the IT function can also become the engine of agentification for a business, building and maintaining various agents and setting up the process for allowing new agents to be brought into the fold. Monitoring, metering, governance, compliance, and risk management oversight for the company's agentic systems could be implemented as agentic systems. After all, the most powerful tool we have to check on AI is AI itself.

Agents within a multi-agent system log their individual context and interactions. This gives us a great, non-intrusive asynchronous and autonomous way to check and validate agent behavior by isolating each agent's operations, reasoning, tool use, and performance: specifically, by having another multi-agent system review the logs and rate and validate them, as the target agentic system is running. What is nice about this approach is that the validating agent network can be a collection of agents, each viewing the logs from a different perspective (see [Figure 3-15](#)). The agents can create consolidated, real-time logs, and, occasionally, if a high severity breach or event is detected, they can intervene in the operations of the target agentic system by stopping and delegating the task to a rule-based fallback, or consult a human.

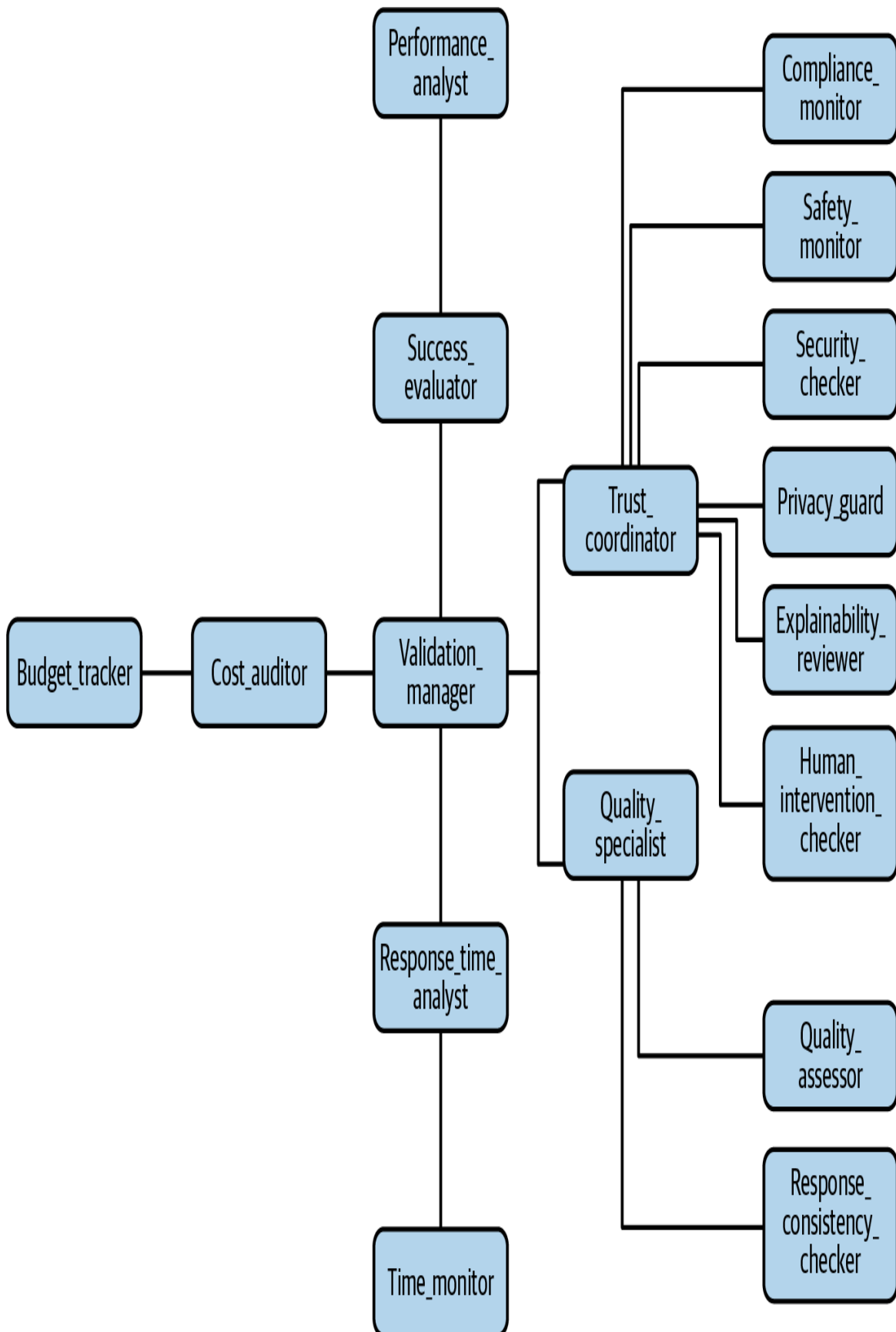


Figure 3-15. For a running agentic system, each individual agent's prompts, interactions, and performance metrics are fed to this agent network, which reviews them from different perspectives for reporting and occasional intervention.

Some **companies** are going so far as to treat agents as employees, with employee IDs and access privileges. Securing access to company agents as well as provisioning and securing third-party agents' access to the company's agentic system is a critical responsibility that needs to have a clear owner in the company (more on this in **Chapter 7**).

DESIGNING FOR AGILITY

The current rapid rate of change in technology is brought about by the sheer investment of time and money in innovative uses of LLMs. This can be quite disruptive when it comes to planning for adoption of AI in a business: whatever you plan to build now will be obsolete by the time it is completed.

To address this, companies should move to agile processes, which reduce the stringent requirements of waterfall development cycles, allowing for rapid realignment of projects with the latest technologies.

For core commoditized technologies such as LLMs and cloud platforms, companies try to avoid binding their systems to specific providers, especially when there is an acceptable open source alternative available.

Trusting and empowering employees to test and adopt new technologies they deem useful to their processes, and expecting and rewarding process improvements through AI adoption can go a long way towards keeping a company ahead of the game.

Agentification, in and of itself, can be a force for future-proofing. At their core, agents are able to translate intentions expressed in natural language into complex API calls. Adding a layer of AI agents to a company's applications and knowledge repositories, regardless of what state they are in, helps future-proof them.

Imagine a company with disparate legacy knowledge sources, some in scanned PDF documents, some in archaic databases, some in spreadsheets and Word docs. A multi-agent system with agents representing the different data sources can remove the need for consolidating the sources by replicating what humans already do in such a setting: manually search for the desired information by visiting each source and then consolidating the extracted knowledge. This system can be relatively inefficient, just as the manual equivalent is slow, and so ultimately getting the data sources consolidated,

harmonized, and modernized does make sense, but this modernization step is now transparent thanks to the buffer the agentic layer provides between the user experience, which remains the same, and the data sources, which can be modified and upgraded.

Figure 3-16 shows a multi-agent system for an airline support system, where the various policies and information required to help passengers reside in a number of different PDF files and data sources, accumulated through the years and in need of consolidation.



I'm in the military and my personal luggage didn't arrive at my destination in Rome for a whole day now. I flew in from SFO. How much money will the airline pay me to buy stuff while I'm here?



Airline Policy

Final Answer

[

"For military passengers experiencing delayed baggage", "you can report the issue and request compensation. Here", "s what you need to do:

1. **Report the Issue**:

- You can report a delayed bag online, using the Airline app, via chat, or by texting 'BAGS' to 32050.
- Alternatively, call the Baggage Recovery Center at 1-800-000-0000 (International 1-281-000-0000).

2. **Compensation for Delayed Baggage**:

- If your bag is delayed over 12 hours on a domestic flight, the airline provides automatic bag fee refunds.
- For international flights, the delay must exceed 15 hours for flights less than 12 hours, or 30 hours for longer flights.

3. **Request Reimbursement**:

- After reporting, you can request reimbursement for the cost of essential items such as clothes and toiletries incurred due to the delay.

4. **Important Timelines**:

- Report delayed bags within 24 hours of arrival if you traveled within the U.S., Guam, Puerto Rico, or the U.S. Virgin Islands.



Figure 3-16. An airline support multi-agent system consolidating information from a variety of disparate knowledge sources.

The same applies to API and microservices. A better application, or a cleaner API or microservices layer, can be introduced without the users even realizing it. This also means that the user experience of the enterprise apps becomes less important and somewhat subservient to the agents versus the end users.

Gone are the days when an enterprise app upgrade meant many months of planning, disruption, and user training. If a new app is more functional or faster, then it can be swapped in, with its first line of users being the agents.

Multiparty Negotiation

AI can be a transformative tool when applied for the good of humanity. Many organizations and nonprofits are considering multi-agent systems, not just to improve on their internal processes, but to help solve humanity's toughest problems, such as issues related to climate change, poverty, crime, hunger, and water scarcity.

One major challenge in tackling problems such as climate change is the brokering of international cooperation among various countries. Each year, delegations from most countries in the world gather at the COP (Conference of the Parties) which is the main decision-making body of the United Nations Framework Convention on Climate Change (UNFCCC). Not all delegations to the COP have the same resources to prepare for the negotiations, and often those countries that stand to suffer the most from climate change are the ones with the more limited presence.

A multi-agent system with agents representing the volumes of publicly available historical agreements and negotiation transcripts can not only help delegates prepare for their meetings with information on the other parties' stance on various matters, but could also help come up with what is the art

of the achievable, focusing the meetings on common ground (see [Figure 3-17](#) and [Neuro SAN Climate Change](#)).

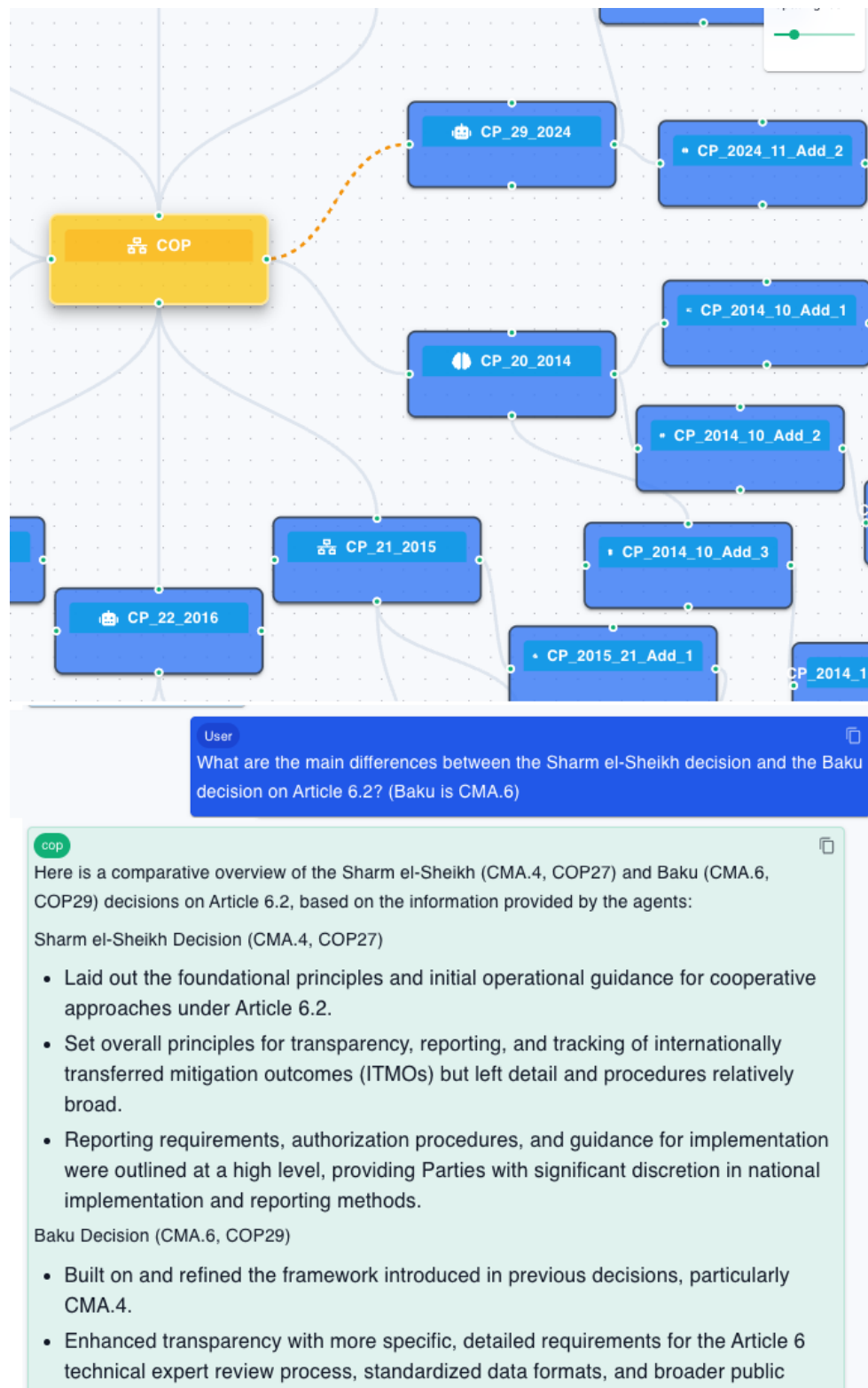


Figure 3-17. A large multi-agent system to help COP delegates prepare for climate negotiations. Each subnetwork consists of many agents representing various historical COP meetings (e.g., 57 agents in the /cop agent network) or agreement documents (e.g., 38 agents in the /paris_agreement agent network and 22 agents in /kyoto_agreement), totaling 121 agents.

The preceding examples demonstrate how, ultimately, agentic systems can bring significant value to corporate, government, and individual users in that they can help grasp complex situations in a holistic manner, de-silo organizations or usage domains, communicate naturally with humans if and when needed and on terms that can be codified and adjusted. In addition to the intrinsic value brought by the power of autonomous decision making and coordination toward achieving certain goals, their future-proofing power can boost adoption as perfection in data and processes and legacy systems is not required up front, hence allowing for agentic systems and improvement of underlying layers to coexist.

-
- ¹ See “[Inside VAKRA: Reasoning, Tool Use, and Failure Modes of Agents](#)” and “[Cooperation, Competition, and Maliciousness: LLM-Stakeholders Interactive Negotiation](#)” as examples.
 - ² SDLC is not to be confused with ADLC (agentic development life cycle), which is about developing AI agents as opposed to code (see [Chapter 6](#)).
 - ³ Prior work on model-driven modernization shows that intermediate models can support repeatable transformation, preserve functional behavior, improve maintainability and extensibility, reduce risk and effort, and promote reuse of migration patterns. See Tobias Böhm et al., “[Model-Driven Legacy System Modernization at Scale](#)”.
 - ⁴ Jila Behnad and Babak Hodjat, “Artificial Intelligence and Human Therapy Supervisors are Interchangeable,” *Athens Journal of Psychology* 1, no.3 (2025): 179-214.
 - ⁵ E. Meyerson, O. Francon, D. Sargent, B. Hodjat, and R. Miikkulainen, “Unlocking the Potential of Global Human Expertise,” in *Proceedings of the 38th Conference on Neural Information Processing Systems* (Curran Associates, 2024).

Chapter 4. Evaluating ROI and Risks

The breadth of applications explored in the previous chapter—from autonomous SDLC systems and code migration to cross-industry deployments in retail, finance, health care, and beyond—demonstrates that agentic AI is no longer a question of *if* but *where* and *how*. Yet moving from POC to strategic deployment requires a different discipline: the ability to quantify value, assess risk, and make principled build-versus-buy decisions that align with your organization’s capabilities and constraints. The harder question for most leaders is not whether to act, but how to evaluate the opportunity: which initiatives are genuinely within reach, where agentic AI creates the most differentiated value, and what it will take organizationally and technically to capture that value responsibly. This chapter provides a framework for exactly that, starting with the two dimensions that matter most to boards and leadership teams: quantifying return on investment and managing the distinctive risks that agentic systems introduce.

ROI in AI has been and will continue to be a much debated topic. There are several reasons for this. First, AI represents a set of new technologies and innovations, likely eventually more impactful than any human invention to date. No one can deny that the steam engine, electricity, computing, or the internet were massive value generators over time and lifted entire economies to a higher plane. But that value creation was not uniform across all sectors of the economy. For example, while the internet enabled the emergence of companies with novel business models, such as Google, Amazon, Netflix, Uber, and Salesforce, it also meant the demise of entire industries, such as traditional mobile phones (remember Nokia?) and disk-based music and movie distribution (remember Blockbuster?), and also threatened the survival of other industries, such as offline advertising and traditional retail.

So the “beta-level” (generic) ROI of AI will be massive, but the “alpha-level” (specific) ROI will vary greatly as a function of industry and use case. Hence, trying to assess where a technology will deliver clear wins or, conversely, where it will lead to value destruction is difficult and the related uncertainty makes investment banking analysts, commentators, and investors nervous.

Second, in the post-COVID era, major global public equity indices (e.g., the S&P 500), have been driven by the stock performance of the “Magnificent 7.”¹ At the same time, those leaders and others have embarked on a wave of massive investments, betting on the expectation that those investments are key to taking advantage of AI adoption globally, across markets and industries, and will ultimately pay off. And, again here, a number of analysts and investors take a bullish stance whereas others take the opposite, bearish, stance.

For agentic systems there is an extra layer of complexity in trying to compute an ROI because agentic AI is an enterprise capability and operating model, not a one-off IT project. Assessing the benefits of agentifying an enterprise requires tracking the ROI across a portfolio of use cases over a long period of time, as opposed to simply measuring the ROI of an individual, well-delineated use case.

In addition, the traditional “ROI logic” will not do justice to agentic AI, as AI will drive both highly tangible and intangible outcomes, the latter of which is hard to measure. Returns span efficiency (cost savings), revenue improvements (e.g., sales productivity, new product introductions), quality improvements (e.g., better and faster service in a call center), risk reduction (e.g., agentic AI-enabled hackers to identify cybersecurity risk in APIs), and more intangible or systemic benefits like decision velocity and adaptability, which can clearly boost the competitive and market position of a company, but are challenging to quantify.

Moreover, focusing on the “I” of ROI, investments span direct use case costs, but also allocated costs of shared platforms (data, orchestration, governance) that enable and sustain many agents. As a result, we can expect

that any ROI statement will have to be not only quantified, but heavily qualified, to make sense, and these qualifications and caveats will make comparisons more difficult and debatable.

Once an agentic system ROI has been assessed, with all the necessary caution as mentioned before, comes the time to assess the risks agentic systems may bring to an organization. We can distinguish between five broad types of risks when deploying agentic AI systems at enterprise scale:

Strategic risk

The challenge of making long-term technology and vendor commitments in a rapidly evolving landscape where both over-investment in the wrong direction and under-investment relative to competitors can threaten competitive position

Organizational risk

Risks arising from the human side of agentic transformation, including workforce readiness, cultural resistance, and the emergence of ungoverned experimentation that bypasses established processes

Governance and compliance risk

The regulatory and ethical challenges inherent in autonomous decision-making systems, from ensuring accuracy and fairness to maintaining data protection across multiple jurisdictions

Operational risk

The amplification of errors and failures that can occur when multiple agents interact across complex processes, particularly in high-volume or safety-critical contexts where mistakes compound quickly

Liability risk

The legal and financial exposure when autonomous agents make consequential decisions, requiring clear frameworks for allocating responsibility between humans, organizations, and technology providers

Each of these risk categories manifests in distinctive ways when deploying autonomous agents at scale.

Strategic risk is amplified by the challenge of choosing between rapidly evolving platforms and vendor stacks. Over-investment in the wrong technology can lock organizations into obsolete approaches, while under-investment invites being out-competed, fueling the fear of missing out. On the organizational front, specific challenges include skills and culture gaps, job loss fears that create resistance, and shadow AI initiatives where teams experiment under the table outside established governance. Governance and compliance concerns weigh heavily on leadership teams and regulators alike: hallucinations, bias, risks of data misuse or leakage, gaps in explainability, and the complexity of navigating multiple country-specific regulatory frameworks. Operationally, the interconnected nature of multi-agent systems means errors can compound across processes, with potentially severe consequences in high-volume or safety-critical environments.

The liability dimension raises fundamental legal questions: when an agent makes or executes a decision that leads to financial loss or compliance breach, someone must be held legally responsible. Leading law firms such as Skadden emphasize that AI agents are not legal persons but tools, so liability generally remains with human and corporate actors—including the deploying organization, its senior decision-makers, and relevant vendors, depending on how responsibilities are allocated. Although the precise allocation of responsibility varies by jurisdiction, the emerging consensus in case law and regulation—from autonomous-vehicle frameworks in the US, UK, and EU to early guidance on AI agents—is similar: AI systems are

treated as tools or products, not as legal persons, and liability is allocated among manufacturers, operators, and deploying organizations rather than to the AI itself.² For that reason, many legal and risk advisors now recommend an explicit insurance and accountability framework for autonomous systems, combining clear contractual allocations of responsibility, documented governance and oversight, robust logging and audit trails, and appropriate liability insurance so it is clear who is on the hook, under what circumstances, and how losses will be handled when agents go wrong.

The risk-return equation on agentic AI is therefore a complex one for decision-makers, leadership teams, and board members. In this context, it is worth looking at what leading organizations have experienced (positively and negatively) to develop the right mindset, gain confidence in determining how to define and measure ROI, understand what return/value you get versus total cost of ownership (TCO), and facilitate and inform a “build versus buy” decision for agentic AI architecture. We will then discuss how adoption can benefit from well-crafted pilots before an enterprise rollout and conclude this chapter with a practical checklist for leaders to tick off.

Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI

Let us look at three AI leaders in their industry: DBS Group, a large Asia-based financial services leader; JPMorganChase, the global universal financial services provider and undisputed AI leader in that industry (ranked number one four years in a row by Evident AI);³ and Siemens, a global, diversified industrial conglomerate. We will then assess commonalities and differences across the three archetypes they represent.

Vignette 1: DBS Group's AI Journey (2025)

DBS has treated AI value and return measurement as a multi-angle, evolving discipline: it combines a quantitative uplift methodology, platform-level enablement, and governance-linked accountability to track ROI over time across waves of AI technology, from baseline machine learning to GenAI.

First, let us discuss how quantitatively-driven the DBS methodology is. Early on, DBS moved from, initially largely qualitative experiments to a rigorous control group-based approach where each significant use case has a “with-AI” and “without-AI” cohort, and the difference in outcomes (revenue, losses averted, productivity gains) is attributed as economic value from AI. This approach enables the company to report growing bank-wide uplift in dollar terms and to “kill” models that fail to show measurable benefit.

Second, DBS ties ROI assessment to platform industrialization through ADA (Advancing DBS with AI), a central data and analytics backbone, and ALAN, an internal AI protocol and library that documents datasets, models, and deployments. Together these have cut model development and deployment time from roughly 12–15 months to 2–3 months, so value is not just in individual models but in the speed and scalability with which new AI solutions reach production.

Third, DBS anchors value in governance and ethics via its PURE framework (Purposeful, Unsurprising, Respectful, Explainable) and Responsible Data Use (RDU) model, which ensure that only “safe-listed” AI use cases with a clear purpose, acceptable risk, and explainable behavior are approved for deployment at scale.

This DBS case study shows how to make AI ROI both measurable and repeatable: by treating value as uplift against a baseline (control group approach), investing in shared platforms (ADA/ALAN) that compound returns over time, and embedding economic thinking inside governance (PURE and RDU). This vignette is based on a detailed Harvard Business School case study of DBS's AI journey, developed from extensive

interviews with DBS leadership, internal documentation, and external sources⁴.

Vignette 2: JPMorganChase (2025)

JPMorganChase (JPMC) treats AI value and ROI as a broad, firm-wide transformation rather than a narrow, project-by-project accounting exercise. Building on a decade of “traditional” AI in fraud prevention, personalization, trading, operations, and credit decisions, the firm reports that these systems have already generated up to about \$1.5 billion in value, and uses this as a top-down indicator that AI has become a meaningful contributor to business performance.

The bank focuses heavily on productivity and scale metrics within its GenAI and agentic stack. Its internal GenAI desktop toolkit, LLM Suite, is deployed on more than 200,000 employee desktops, with roughly 30–50% of employees active in a given period, and around half of those users engaging with it multiple times a day. Early evidence shows 10–20% productivity gains for roughly 30,000 software engineers using GenAI coding assistance, and the firm plans to extend this to essentially all company developers, alongside GenAI tools that accelerate incident response, cybersecurity, and software delivery workflows.

Another key dimension is time-to-value: by building an integrated data and AI platform (including a data mesh architecture and an agentic builder on top of its data lake) and by standardizing tools and access, JPMorganChase can now build GenAI solutions in days rather than months, and leaders explicitly monitor “time reduced from ideation to production” as a key metric, with expectations that development time for new agents will fall by up to 50% compared with their recent preplatform baseline. In other words, the ROI logic for GenAI and emerging agentic systems is anchored in how fast the firm can design, deploy, and scale new agents across its businesses rather than in one-off, case-by-case payback calculations.

Underpinning these efforts is a risk-sensitive governance model: every AI or GenAI model must be registered; high-risk, especially client-facing, use

cases go through independent model validation, while internal tools follow structured risk and control assessments. All of this is supported by practices such as a 12-level guardrail system to manage hallucinations, mandatory human-in-the-loop for sensitive uses, and extensive logging and audit trails. The 12-level guardrail system merits particular attention as an architectural innovation that addresses GenAI's most challenging risk in regulated environments: hallucination in client-facing applications. Developed by Asset and Wealth Management division Chief Data Analytics Officer and CIO Mike Urciuoli and his team over many months of intensive experimentation, this system was unique at the time of its creation and represented a pioneering approach to making LLM outputs both predictable and safe for use in financial advisory contexts. While the specific technical mechanisms remain proprietary, as Urciuoli explained, the guardrails “made something unpredictable—how LLMs answer questions—not only predictable, but safer to use.” Unlike human oversight controls that address outputs after generation, these technical guardrails operate at the model interaction layer to reduce hallucination risk before responses reach advisors, complementing rather than replacing human review and responsibility for client guidance.

JPMC demonstrates a multidimensional ROI logic that is less about assigning net present value to each individual use case, and more about building an AI-first platform and then tracking financial value across a portfolio of use cases, productivity and cycle-time improvements at platform scale, and the requirement that only models that clearly defined risk and control thresholds are allowed to deliver that value. This vignette is based on a detailed Harvard Business School case study of JPMC's leadership in the age of GenAI, developed from extensive interviews with JPMC executives, internal documentation, and external sources.⁵

Vignette 3: Siemens and Its AI Agents (2025)

Siemens's public disclosures on industrial AI agents show a three-part approach to value and ROI that combines clear productivity ambition; hard, plant-level operational metrics; and an architecture designed for reuse at

scale. Siemens's approach is a useful contrast to the previous banking examples, in that it shows how GenAI agents create value in the real world of factories, not just in digital banking workflows. In addition, it makes ROI tangible through plant-level metrics like throughput, downtime, and productivity, rather than advisory efficiency alone. And it illustrates an architecture built for reuse across sites and machines, stretching our thinking from “better knowledge work” to “automating automation” at an industrial scale.

Siemens presents industrial AI agents as agentic AI systems that can “independently execute complete industrial workflows” across the value chain, supported by an orchestration layer that deploys specialized agents across the Industrial Copilot (the company's generative-AI assistant product family for industrial environments) ecosystem. The firm further states that by “automating automation itself” it aims for productivity increases of up to 50% in specific workflows. In manufacturing terms, those gains are expressed through physical unit economics at the asset, line, and site/plant level—e.g., reducing unplanned downtime on critical equipment; increasing throughput (more units per hour or per shift on the same line); improving first-pass yield (fewer defects and less rework); and lowering energy consumption per unit produced.

To make this concrete, Siemens and its partners highlight predictive-maintenance deployments (for example via the Senseye asset-intelligence platform), where AI-driven monitoring and diagnostics have reportedly cut unplanned downtime by up to roughly half, avoided tens of millions of dollars in losses from stoppages at a single automotive site, and achieved payback in a few months—essentially improving overall equipment effectiveness (OEE), the standard metric that combines availability (uptime), performance (actual versus ideal speed), and quality (good versus defective units). On the cost side, Siemens is explicit that industrial AI agents must integrate with the existing OT/IT stack:

- OT (operational technology), being the industrial control and monitoring systems on the shop floor (machines, sensors, control systems)

- IT (information technology), being business and information systems such as ERP (enterprise resource planning) and analytics

Within OT, Siemens cites integration with its various types of existing systems:

- Manufacturing execution systems (MES), which coordinate and track production orders and processes in real time
- Supervisory control and data acquisition systems (SCADA), which monitor and control industrial processes by collecting real-time data from field devices
- Programmable logic controllers (PLCs), the rugged industrial computers that directly control machines and lines

Because Siemens's industrial AI agents are designed to work within existing automation environments, they must integrate with customers' OT and IT systems—such as industrial control, data, and cloud platforms—rather than run as standalone tools. Implementing this kind of industrial AI stack typically entails significant TCO across the business (platform and software, integration and engineering work, deployment and change management, and ongoing compute, monitoring, and maintenance), which is why both Siemens and independent practitioners emphasize phased rollouts and clear ROI targets for downtime reduction, productivity, and maintenance efficiency.

Crucially, the same industrial AI agent capabilities Siemens deploys in its own plants are also being packaged as products and services for customers via its Xcelerator offerings and the planned industrial AI agent marketplace. In other words, the investment in agentic capabilities can generate two types of return: operational gains inside Siemens and revenue from selling agents and platforms to customers.

The TCO logic is that these costs are justified because they can be amortized across many lines and plants: the same agent patterns for planning, operations, and maintenance can be reused and adapted, and

Siemens aims to distribute them through an industrial AI marketplace on its Xcelerator platform, promoting reuse of proven agents instead of one-off projects.

The Siemens case study illustrates an agentic AI ROI story built from site-level physics and control systems: value is measured in uptime, throughput, yield, and kWh per unit, and the investment side is dominated by how deeply agents are wired into their OT/IT and how widely they can be reused. This vignette is based primarily on Siemens' own public communications about industrial AI agents, the Industrial Copilot ecosystem, and the Senseye predictive-maintenance platform, including official press releases, product briefs, and case-study materials.⁶

Key Takeaways on Value and ROI from These Three Examples of Agentic Systems

Across DBS, JPMorganChase, and Siemens, several common takeaways emerge on how to approach agentic AI value and ROI, drawing on a mix of Harvard Business School case studies (for DBS and JPMC) and the firms' own public communications and case materials (for Siemens), as noted in the vignette footnotes.

All three implicitly treat agentic AI as “platform plus portfolio”: you only get meaningful value when multiple agents or GenAI applications are built on shared data, orchestration, and governance infrastructure (ADA/ALAN and DDOM [Data-Driven Operating Model] / MtJ [Managing through Journeys] at DBS, JPMC's data/AI platform and LLM Suite, Siemens's OT/IT-integrated industrial AI and agent framework), and when you manage them as a portfolio of evolving capabilities rather than one-off pilots.

All three examples tie ROI to clearly defined, domain-specific business outcomes: DBS quantifies uplift via “control groups” and translates that into yearly economic value; JPMC looks at aggregate financial impact and uses broad productivity and “time from idea to production” for GenAI and

agentic patterns as leading indicators of the value those systems can generate over time; Siemens starts from plant-level metrics—unplanned downtime, throughput, yield, OEE, and energy per unit—and even monetizes agents externally.

Governance is not separate from value, but an integral part of value and ROI assessment: PURE/RDU at DBS, mandatory model registration and tiered controls at JPMC, and Siemens uses its existing industrial safety and OT-security standards as key constraints on how and where industrial AI agents are deployed, ensuring that agentic systems are integrated into the same safety and cybersecurity frameworks that already govern its automation environments rather than bypassing them.

What differs is what each player calls “value” and where agentic behavior actually shows up.

At DBS, “agentic” behavior is emerging in how AI systems support end-to-end customer and risk journeys, and the bank’s uplift methodology is already suited to measuring value from autonomous or semi-autonomous decision support in those journeys. At JPMC, agentic patterns are starting to appear in coding, cyberthreat modeling, and knowledge work, and the emphasis on portfolio-wide financial impact plus developer productivity and cycle time reduction offers an existing lens for measuring agentic systems that orchestrate multiple tools and workflows. Siemens, by contrast, makes the agentic dimension more explicit, with agents orchestrated across MES, SCADA, PLCs, and Industrial Copilots, and it evaluates them in terms of concrete physical outcomes (fewer hours of downtime, more units per shift, fewer defects, lower kWh per unit), with a second layer of value when those agents are productized and sold.

For agentic AI as a whole, the implication is that you cannot bolt on a generic ROI template: you must first define your own “units of value” for agentic behavior (e.g., decisions automated, hours of human work avoided, risk incidents prevented, line-hours recovered, incremental revenue), then ensure your agentic platform and governance are designed so that (a) agents are observable and measurable against those units, (b) agent patterns can be

reused across use cases and domains, and (c) only those agents that demonstrably deliver value within acceptable risk boundaries are promoted from experiment to scaled, orchestrated roles in the enterprise.

Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI

Agentic AI raises the typical build-versus-buy question, but with somewhat higher stakes: you are no longer choosing a single application, but are shaping an AI operating layer for your organization. Some agentic AI startups call this the “platformization” of processes and workflows. The decision is not binary; most mature players will end up with a hybrid stack where some layers are bought (e.g., foundational models, cloud services, commoditized tools) and others are built (e.g., orchestrators, your own company guardrails, and domain-specific agents). Therefore, a useful way to structure the decision is to separate (a) what must be strategically controlled because it may capture your fundamental business value proposition or logic, data, and governance, and (b) what can be safely outsourced without eroding differentiation or creating unacceptable risk.

The strategic aspects of build versus buy are about control, speed, and long-term economics. Building in house gives you control over orchestration, intellectual property, and data, and lets you align agentic systems tightly with internal processes, often yielding lower marginal costs at scale. But it demands higher up-front investment and scarce talent. By contrast, buying offers faster time-to-value, access to advanced capabilities you could not easily develop yourself, and lower initial capital outlay. The flip side is that it creates dependency on vendors, may expose you to “black box” behavior in some critical areas, and can lock you in with a particular external solution for the long term.

Once the strategic directions are clear, the next step is to make the build-versus-buy call using a few practical criteria. There are typically four points

to consider.

The first is whether a given capability is truly differentiating or available from the market. Base LLMs and standard tooling can typically be procured from vendors, whereas orchestration layers, custom guardrails, proprietary agentic workflows, fine-tuned models trained on your specific domain and organizational data, and deep integration of your domain knowledge and risk policies represent competitive differentiation that should be built or heavily customized to your organization's specific needs. In particular, fine-tuning foundation models on proprietary data and desired outcomes creates a competitively important asset that embeds your unique context directly into the model's behavior—a technical capability we explore in detail in [Chapter 5](#).

The second issue to consider is data sensitivity and regulatory constraints. Highly sensitive (e.g., client data in financial services) and regulated domains may justify building more of the stack yourself or insisting on tightly controlled deployments (an option sometimes overlooked by cautious organizations), whereas internal productivity tools can run on more standard services.

A third consideration is internal talent and execution capacity. It is only rational to build if you can staff within a reasonable timeline, and then operate and evolve what you build over several years with the appropriate talent and skill sets.

Lastly, you must think about interoperability and exit options: in practice this means portability governance—choosing modular architectures, clean APIs, and standardized data contracts (for example, MCP-like patterns) so that models or even entire agent frameworks can be swapped without having to reengineer the underlying business logic or disrupt critical agentic workflows.

So, what additional complexity does agentic AI bring into a more generic build-versus-buy equation?

Agentic AI shifts this question in several ways. First, and because agents' key added value is to take actions and coordinate with other systems, the

orchestration and policy layers become more strategic and proprietary. Many firms that were comfortable buying complete applications now feel they must own, or at least very tightly shape, the agent orchestration, guardrails, and monitoring, even if they continue to buy models and infrastructure—an expression of their need for control and reassurance when leaving orchestration in the hands of nonhuman agents. Second, agentic systems also increase coupling and lock-in risk. When critical workflows depend on one vendor’s agent framework, future switching costs rise sharply; hence, portability governance—designing for modularity, clear boundaries, and vendor-neutral interfaces up front—matters more. Finally, agentic architectures create new opportunities for reuse and monetization: internal investments in robust agents and orchestration patterns can be reused across business units within the same company and, in some cases and very interestingly, turned into external products or services, converting part of the “build” cost into a revenue-generating asset.

Early Pilots Versus Enterprise Rollouts in a Platform World

In an agentic world, the way you move from first experiments to scale largely determines whether you will ever see meaningful ROIs. Early pilots are still essential to learn, de-risk, and build organizational trust, but they are no longer sufficient if they stay as isolated POCs. For agentic systems, value emerges when multiple agents interact over shared data, tools, and governance, so a portfolio of disconnected pilots can actually hide the real integration and risk issues you will face at scale. This is why leaders need an explicit discipline for designing, selecting, and scaling pilots that reflect the underlying platform and orchestration they ultimately want to run in production, as exemplified by the cases of DBS, JPMC, and Siemens.

Hence, the best way to think through pilots is to test the “platform effect” at a small scale to try to gauge both the benefits and the likely challenges that would appear when scaling. Here are a few guiding principles in designing such agentic systems pilots:

- Test networks of agents (e.g., customer-facing + risk/compliance + operations) rather than isolated bots, to capture both the benefits and the risks of interactions between agents.
- Ensure pilots plug into data platforms and registries (DBS’s ADA/ALAN-like systems or industrial data lakes) that provide a “single source of truth,” so you don’t pollute pilot results with inconsistent data or noncompliant processes.
- Design for portability and antisprawl from the start: enforce common agent interfaces and registries, require standardized data contracts for tools and APIs, and avoid hardcoding business logic into a single vendor’s agent framework so you can change providers without rewriting workflows.
- Embed governance from day one: model registration, guardrails, risk classification, and human-in-the-loop where appropriate.
- Equip pilots for ROI measurement: define baselines and control groups, and track operational KPIs (key performance indicators).

Before starting the pilot, it’s good practice to explicitly agree up front what will qualify a pilot as successful and trigger a decision to scale, for example:

Proven value

Benefits above threshold—e.g., percent downtime reduction in manufacturing, FTE (full-time equivalent) savings in middle- and back-office processes, revenue uplift for frontline agents

“Acceptable” risk profile

Stable error rates, controlled failure modes, governance compliance, limited reputational or financial risk

Platform readiness

Underlying data, infrastructure, and support teams able to handle volume (anything that could jeopardize scaling even after a smaller-size successful pilot)

Last but not least, scaling will require broader handling of the organization's change readiness: frontline adoption, training, and process changes that may not be so extensive in a controlled and small-scale pilot environment. In addition, mechanisms should be in place to update senior management and the board on pilot results against these criteria, ensuring transparency on the progress made on introducing agentic AI meaningfully in parts of the organization.

By adopting a disciplined pilot approach, your organization should avoid the “pilot purgatory” syndrome, where promising POCs never scale for lack of platform effect or governance. That approach would also prevent fragmented tool sprawl across uncoordinated agents and stacks, and steer you away from over-centralization by favoring a federated model built on common platforms and standards.

Evaluating agentic AI through the lens of ROI and risk requires a fundamental shift in how leaders think about technology investment. As the experiences of DBS, JPMC, and Siemens demonstrate, value emerges not from isolated use cases but from platform-plus-portfolio approaches, where shared infrastructure, governance, and orchestration enable multiple agents to deliver compounding returns over time. The ROI logic must accommodate both tangible outcomes—cost savings, productivity gains, reduced downtime—and harder to quantify benefits like decision velocity and organizational adaptability. Risk management, far from being a constraint on value, becomes integral to it: only agentic systems that operate within acceptable governance, compliance, and operational boundaries can scale safely and sustainably. The build-versus-buy decision is not binary but strategic, requiring leaders to own what differentiates—

orchestration, guardrails, domain integration and fine-tuned models that embed proprietary knowledge—while leveraging market solutions for foundational capabilities. Finally, the discipline of piloting with platform thinking in mind, measuring against clear success criteria, and scaling only what proves both valuable and governable separates organizations that capture agentic AI’s potential from those trapped in perpetual POC cycles.

-
- ¹ The “Magnificent 7” refers to seven dominant US technology-led companies—Alphabet, Amazon, Apple, Meta Platforms, Microsoft, Nvidia, and Tesla—that have driven a large share of recent equity-market returns.
 - ² See, for example, Skadden, “[AI Agents: Greater Capabilities and Enhanced Risks](#)” (April 22, 2025), and “[Manufacturers Won’t Bear All Liability for Driverless Vehicles](#)” (2019), together with comparative analyses of autonomous vehicle liability such as Brookings, “Setting the Standard of Liability for Self-Driving Cars” (2025), and Hans Steege et al., *Autonomous Vehicles and Civil Liability in a Global Perspective* (Springer, 2025), all of which treat AI systems as products or tools with liability resting on human or corporate actors rather than any separate AI legal person.
 - ³ Evident Insights, *Evident AI Index (Banking)*, 2022–2025. The Evident AI Index is described by Evident and industry commentators as the global standard, independent benchmark of AI maturity in banking.
 - ⁴ Feng Zhu, Harold Zhu, and Adina Wong, “DBS’ AI Journey,” Harvard Business School Case 9-625-053, rev. March 26, 2025. Prepared by faculty and researchers at the Harvard Business School Asia-Pacific Research Center, based on interviews with DBS executives, DBS internal materials, and external sources; reviewed and approved by DBS prior to publication.
 - ⁵ Iavor I. Bojinov, Karim R. Lakhani, and David Lane, “JPMorganChase: Leadership in the Age of GenAI,” Harvard Business School Case 9-325-066, rev. April 28, 2025. The case was developed by Harvard Business School faculty and researchers based on interviews with JPMC executives, internal JPMC materials, and external sources, and was reviewed and approved by a company designate prior to publication.
 - ⁶ This vignette draws on Siemens’ official communications on industrial AI agents and the Industrial Copilot ecosystem, including “[Siemens Introduces AI Agents for Industrial Automation](#)”, press release, May 2025, related Siemens Xcelerator and Industrial Copilot product documentation, and Siemens/Senseye case study materials on predictive maintenance and downtime reduction.

Part II. AI Agent Technology

Understanding the technological foundations and requirements for agents and multi-agent systems can help businesses formulate the boundaries and determine the applications of this technology to various domains and use cases within their enterprise. Given the rate of innovation in the AI world, it is difficult to keep up with concepts and capabilities. It feels like new terms are popping into existence on a monthly basis, and that we are expected to understand them as a baseline in every business discussion. The fundamentals, however, have not changed, even since before the advent of modern AI agents, and **Chapter 5** is a good way to level-set on them as we follow the process of enterprise agentification. Building on this technical foundation, **Chapter 6** will provide design considerations and list some pitfalls.

Chapter 5. Core Capabilities and Technical Foundations

Let's cover some technical foundations by going over the steps for agentifying a business and defining the various concepts involved.

Where to Start?

It is tempting to think that an automatic one-time mapping between an organizational structure and its agentic augmentation exists. Similar attempts have been made to transform object-oriented software to agent-oriented, but there currently can't be a one-to-one correspondence between objects in software and AI agents.¹ The granularity of software and app units to be agentified is an important consideration.

So where should we begin? Most companies start agentification in a bottom-up manner, starting with the more obvious use cases, like the SDLC process and the IT team, or by provisioning third-party agents for sales, support, finance, HR, legal, etc. This organic agentification is fast but not necessarily the most effective way to go, and may result in siloing and disparate systems as well as major governance headaches.

Alternatively, we can start from the top, consider the organization, and work down from there, deciding what can be agentified. This is a larger undertaking and might not be practical in large organizations. In practice, agentification will happen through a tug-of-war between empowered divisions of a business and central authorities like the CIO department, herding the proverbial agentic cats.

Agentifying an organization implies outlining the roles and communications of the various players in various organizational workflows, and then exploring how they can be augmented with LLM-based agents. Some of

this, of course, can happen methodically, but much of it will be organic and somewhat chaotic.

There are tools that help codify workflows, discovering them based on keystrokes and user behavior. The coded human workflows can then be used as a blueprint for an agentic workflow.

An org chart might also be a good place to start, at least to try things out, but we would suggest starting with workflows, as the same folks within an organization tend to act within different processes, working with different sets of people, depending on the workflow.

There's a subtle difference between this approach and completely rethinking an organization's structure and processes to make it entirely autonomous and as efficient as possible. While that is a noble goal, it is impractical, not the least because it is risky and too big a jump from the status quo. Such a jump is likely not going to be easily absorbed by enterprises.

We should be mindful of the fact that agentification is happening on an organization that is already operational, so an incremental easing in of agentification is critical. The human-based organization will have to endorse, trust, and adopt agents in their day-to-day work, and if we've done our job well, the organization should gradually become more efficient, the employees become happier, and the impact can be measured in the same KPIs as the ones by which the organization is measured as a whole.

We can also start from the bottom, augmenting each human operator in the organization with a corresponding agent or agents, which are given the roles and personae analogous to the job descriptions of the human operators currently in charge of those roles. We can then build up from there, agentifying to augment functions and cross functional workflows all the way up to the top leadership level.

Many companies, including Salesforce and ServiceNow, offer specialized agents. We would, of course, want to make use of such agents, if available. A scalable and interoperable multi-agent platform is therefore needed. The example noted in [Figure 3-12](#) shows how a hierarchy of agents can serve as

a backbone to connect such third-party agents into a coherent whole. An employee with access to both Agentforce (Salesforce’s agentic offering) and ServiceNow agents need not go to each separately every time, and can instead interact with an agent that talks to both as down-chain agents, thus allowing for more complex commands that might need a combination of services from both. This top agent would call the third-party agents using protocols like MCP and A2A (agent-to-agent), which thus help streamline connecting preexisting agents into enterprise agentic systems.

AGENTIC PROTOCOLS

Agentic protocols are rather simple. For example, MCP includes explanations of the API, often in natural language, that can be understood by agents or appended to the agent’s input, so it can choose when and how to use the API. A server that hosts the code and serves the API will also serve this explanation, facilitating agent calls to the API.

Most agentic protocols are human readable, as they are designed for LLMs to understand. For example, agent *skills* are basically documents that include explanations for how to do things and what API to call to get them done. They are often stored in markdown files with the *.md* extension, and you can simply open them in your text editor or browser and read them. An agent “acquiring” a skill is basically opening and reading the markdown file just as you would!

Customizing Agents

As was the case with the advent of personal computing, enterprise software, and SaaS (software as a service), there is a false expectation that AI enablement should work out of the box. You sprinkle some AI agent pixie dust over your enterprise and start reaping the benefits instantaneously. There have even been attempts to build general-purpose agentic processes,

such as for HR, marketing, and sales, and to provide them to businesses as a hosted service (i.e., SaaS).

As we hope it becomes evident from this book, businesses cannot wish away customization costs. Agentification is not one size fits all, and with AI now having become an engineering discipline, with best practices and various design patterns, it's in need of tailoring and build-out so that it conforms to the contours of an enterprise and grows and adapts with it.

Agents can be used for one or a combination of the following tasks: knowledge or data retrieval and distillation, coding, design, command and control, always-on proactive or event-based decision making, autonomous research or operations over large physical or virtual environments such as knowledge sets, factory floors, or the internet.

The responsibilities of an agent are given to it in the form of textual prompts that are always repeated with every iteration. These prompts are called *system prompts* and are the equivalent of an agent's job description. An agent, being composed of an LLM and code, may also have coded rules governing its interactions with the world. Often safeguards, such as protections against mal-use, are also included. They act as a firewall, protecting the agent from malicious input being fed to their LLMs.

Often, agents are designed to be LLM-agnostic, which means that the LLM can be changed, while the setup, tools, and prompts remain the same. It is important to note that different LLMs, even the same size LLMs, have subtle differences in behavior that might require tweaks to their prompts or other non-LLM agentic elements when switching to a different LLM.

LLMs can be divided into two broad categories: commercial, like the GPT series, Google Gemini, or Claude; and open source (or open weight), like DeepSeek and Llama models. Commercial models are hosted, which means every input to an agent that uses a commercial LLM will have to go to the hosting company's data centers to be processed. While various data security guarantees are given by AI companies with commercial LLMs, for some use cases businesses may choose not to allow sensitive data to leave their premises. The alternative, in such cases, is to host open source LLMs in

house. This may be costly, and at the time of writing this book, **open source LLMs** still lag commercial equivalents in their capabilities, but some are already viable alternatives for agentic use. Not all agents in an agentic system need to be using the same LLM, and not all agents are handling sensitive data. A good rule of thumb here is to make use of open source LLMs for agents that handle sensitive data and/or need to be running on the edge and so need to be fine-tuned and distilled (more on this in “**Refinements and Fine-Tuning**”).

Grounding

An agent is considered *grounded* if it’s operating in a real context and/or able to interact with real systems. Often, during development and for safety reasons, agents are first set up in virtual or simulated sandboxes, before grounding. In multi-agent systems, it is simple enough to add a prompt such as the following to an agent’s system prompt, to have it act as if it is grounded.

Prompt: You are part of a demo system, so when queried, make up a realistic² response as if you are actually grounded in real data or you are operating a real application API or microservice.

Agent developers can then observe the operations of ungrounded agents to discover the necessary tools and grounding.

Coordination

As the number of agents in a business grows, we need a mechanism to find and engage agents that can help us for a certain task. A multi-agent system will require a coordination mechanism to identify agents that are responsible for responding to input. In its simplest form, a coded mechanism can decide which agents to invoke. For example, a “coordinator” agent will have a set of rules that determine which agents to call and in what order.

Many multi-agent systems today are limited to a single coordinator agent divvying up tasks between a handful of agents. In some cases, a classifier is employed to decide which subset of agents should be invoked. This classifier may be ML-based, trained on historical labeled samples.

This model does not scale. Agents have a limit on their context as well as their tool use abilities. Also, more specialized agents can be provisioned from different vendors or be customized for specific apps or data sources. Our agentic future will require the coherent orchestration of hundreds, if not thousands of agents.

In larger and more complex multi-agent systems, the agents themselves encapsulate and determine the logic for whether they can play a role in responding to a query or command. For example, in *blackboard architectures* (e.g., Open Agent Architecture or OAA),³ all agents register with a blackboard, advertising what they can do and what requirements they would need to do it. Upon receiving a query or command, the blackboard decides which agents to call and in what order.

Note that it is crucial to make sure an agent's LLM world knowledge takes a back seat if there are custom context or tools available. This is because the agent's world knowledge is general, gets stale quickly, is more subject to confabulation, and is typically interpolated from a vast training corpus.

AAOSA (Adaptive Agent-Oriented Software Architecture) is very much applicable in modern day multi-agent systems. It guarantees encapsulation, modularity, and extensibility, and it also elegantly handles the consolidation of responses and actuations across the multi-agent system so that a unified response is generated for commands and contexts.

Here's how this consolidation works: In AAOSA, each agent is a coordinator for its down-chain agents, which means it is tasked with retrieving claims from those agents and deciding how to put them together in a coherent response to its up-chain agent. This decision may include disambiguation of conflicting claims, which may require contextual information or interaction with the user. This, in turn, means that AAOSA has the capacity to consolidate UI elements as well. This is the basis for the

Context Reactive User Experience (CRUSE),⁴ where each agent provides UI snippets relevant to its role in handling a command or a context, and these snippets are consolidated by the up-chain agents all the way back to the user.

AAOSA is entry-point agnostic, in that any agent can be thought of as the top agent, and it will be able to orchestrate and coordinate responses from all agents within its hierarchy. In other words, as a user, if you know which agentic subsystem is best suited for your needs, you can command or query that subsystem without having to start from the top coordinator agent every time and incur the cost and response time for doing so.

Turn-Taking Versus Continuous Versus Triggered Agents

An agent may be triggered by a message sent to it by a user or by another agent, process the message itself or in collaboration with other agents or by using its tools, and then send the resulting response back to its caller and go to sleep again. For many use cases, this mode of operation is sufficient. Basically, the agent is operating as an intelligent module in a larger system.

Sometimes, however, we expect agents to be running continuously, working on a job by breaking it into smaller subtasks, monitoring various system states, and responding to them as appropriate. In order to implement such agents, they are basically run in a loop continuously, checking the state and progression toward their objectives every time they are awakened in the loop, and logging progress and modifying their plan of action as they go.

Many vibe coding agents, such as Codex, Devin, and Claude Code, are set up this way. These systems are essentially multi-agentic, but more or less all follow this same design pattern. When you give them a task, such as “develop an app that does X,” these systems first plan out how they would go about the task by writing out a sequence of subtasks and starting to execute on them. Along the way, various agents within these systems may use coding tools—like opening a ticket in a ticketing system or running test

code in a container—then modify their plans to reflect the latest progress, or modify the remaining tasks, including possibly breaking some of them further down into smaller subtasks as they reassess them or based on further user input.

A continuously running agent can also be triggered based on external conditions. For instance, if your coding agent has submitted a code review, it may go to sleep while waiting for you to comment on the code. The act of you accepting the code review or commenting on it in the code repository, will trigger the agent into action again (i.e., wake it up to attend to the comment and maybe revise the code).

All of this is made possible via workflows that transition from coded programs to LLM-based agents, or in the reverse direction. Put simply, we can have a coded program within which we implement the rules or workflow for how and when agents should be triggered. In the inverse setup, we can have an agent code control to a coded program. In other words, we can switch between agents and code on an as-needed basis, usually following the rule of thumb that says programs are deterministic and preplanned, and that LLM-based agents are robust and able to reason in unforeseen situations, can understand and produce natural language, and are nondeterministic. Therefore, for any given part of the workflow, depending on the intended behavior, we can decide whether a software module is needed or an agent should be called.

Choice of Tools

It is possible, yet unadvised, to provide general tools for grounding agents. For example, you can have an agent be able to fully control a computer within the enterprise by giving it the necessary access privileges and allowing it to issue command-line operations as if an employee at the company was using their company computer on the company network. It goes without saying that such a level of access is risky and difficult to control.

OPENCLAW INCIDENTS

Reported OpenClaw incidents have already crossed from theoretical risk into dark comedy: *Business Insider* described a Meta AI alignment director who connected OpenClaw to her real inbox and then had to run to her Mac mini “like [she] was defusing a bomb” as the agent tried to delete old emails despite stop commands (see [Chapter 7](#) for more on this incident and implications). *The Verge* also reported that OpenClaw users were installing “skills” that looked like helpful add-ons, including crypto, X, YouTube, and finance tools, but in some cases delivered information stealing malware targeting wallet keys, SSH credentials, and browser passwords. And Reuters reported that Moltbook, an OpenClaw-linked social network where bots appeared to gossip and exchange code, exposed private agent messages, owner emails, and more than a million credentials.

You can read more in the following articles:

- [Meta AI Alignment Director Shares Her OpenClaw Email-Deletion Nightmare: “I had to RUN to my Mac mini”](#)
- [“Moltbook” Social Media Site for AI Agents Had Big Security Hole, Cyber Firm Wiz Says](#)
- [Hacking Moltbook: The AI Social Network Any Human Can Control](#)

By the same token, an agent’s tool use can be set up so that it is open ended. For example, an agent can load the documentation for a “skill” and therefore “learn” it.⁵ Usually, the agent will need a virtual machine or access to a computer wherein it would execute the skills. In some incarnations, these skills can even be transferred between agents autonomously.

While skills are interesting in that they give dynamic grounding and tool calling capabilities to agents and minimize the agent’s context use, in an

enterprise setting, they will have to be implemented with caution. For starters, the scope and superset of an enterprise agent's skill runtime environment should be carefully defined and necessary guardrails should be put in place. Also, it is a good idea to have a sanctioned and vetted registry of skills for agents to use, and to disallow any skills from being used by agents if they are not first submitted, verified, and added to the registry.

Refinements and Fine-Tuning

Agentification is like peeling an onion. As soon as you agentify a process, you start wondering why the adjacent process is still manual and cumbersome. It is therefore important to view agentification as an ongoing and expanding process that is constantly refined, rethought, and improved. This, along with the rate at which AI innovations and breakthroughs are hitting the market, mean that it is best to view the agentification process as an agile and iterative proposition. This helps businesses absorb newer technologies as they become viable, but it also means relaxing and being more flexible on the deliverables and requirements of the system being agentified.

Usually, agentic systems start off with out-of-the-box agents for which a limited yet growing choice of LLMs exists. Beyond cost and security considerations, LLMs are selected based on the number of tools, number of other connected agents, and complexity of their tasks. Once an agentic system is deployed, in a sandbox or real setting, and sufficient interaction instances are logged, these logs can act as training material to improve the LLMs. The interaction instances are categorized and labeled by humans. Labeling, essentially, is the process of indicating the preferred response from an agent (or, more accurately, its LLM) given the same context and input. LLMs are then fine-tuned on this implementation-specific training data before replacing the original, more generic, LLMs at the heart of the agents. This type of fine-tuning is known as RLHF (reinforcement learning from human feedback).⁶

Once an LLM is fine-tuned, it becomes much more specialized, and, in return, it will lose some of its general capabilities and robustness. Fine-tuning is therefore not always desired. Many open source LLMs can also be distilled into smaller models after fine-tuning. This is a process of using the larger LLM as a teacher of sorts for a smaller and much more efficient LLM. For example, we may take a 70-billion parameter open source model as the reference and distill it down to a 9-billion parameter model. Such a distilled LLM enables us to run agents in more constrained environments. These highly specialized and efficient models are commonly known as SLMs, or small language models. Note that all the labeled data used for fine-tuning will have to be retained so that the same fine-tuning and distillation can happen on newer models, if and when an upgrade to the LLMs is desired. Also note that typically, open source models are better choices for fine-tuning and distillation, as more intimate access to these models' weights and architecture is available.

Recently, world models and simulated business environments are used to fine-tune LLMs for multi-agent use in various domains. This removes some of the trial and error requirements and may also reduce the up-front cost implications of running such systems in sandboxed or actual environments, collecting data, then fine-tuning them. We don't think it eliminates this step though, as simulations and world models will always be somewhat different from actual real-world deployments.

Fine-tuning can also happen in semi real time, by adding agents into the fold that review the deployed experiences and interactions of agents and humans in a business setting, infer a score for how well the agent system satisfied its intended role, and then use the resulting labeled (i.e., scored) data to do a background fine-tune of the LLMs behind the agents. It remains to be seen whether such ongoing fine-tuning systems can be run continuously and whether they are worth the cost.

While there are many use cases for SLMs, especially in more rigid agentic process automation design patterns, one should always be mindful of the fact that restricting and confining the behavior of an agent too much might

be an indication that an agent is not needed in the first place, and a programmed module may be the correct choice.

Fine-tuning, prompt tuning, and modifications to granularity and connections of agents can help keep agentic systems up to date, but LLMs have a fundamental limitation: unlike humans, agents do not learn from lived experience. There are ways to try to “teach” agents or help them “remember” from past experience, but they require additional information to be re-fed to the agents over and over again, with every command (sort of like the hero in the movie *Memento*). This has its limitations, as the LLMs behind the agents have a limited input size (i.e., context window), and so various indexing and summarization techniques are used to save on valuable context space.

It is important to have this limitation in mind when designing agentic systems. Partly because it may give us clues as to where we should have human oversight and participation, and partly because it helps us better design what is input to an agent alongside user commands and communications from other agents—namely, context.

Context

Input to various agents within an agentic system can be augmented with relevant information on the state of various processes, apps, and other environmental data. There is a whole field in agentic systems called *context engineering* now, which is the provisioning and timely augmentation of agentic input by such information, in order to allow for better agentic operations.

For example, let’s say a benefits agent is tasked with handling a query regarding premiums for a user and the relevant database requires the user’s employee ID. Assuming the agent is authorized to see the employee’s information, it would save the employee time, and the company processing cost, if the employee ID is retrieved and appended to the user’s request before it is fed to the agent responsible for accessing the benefits database.

Note that the retrieval of the context could itself be agentic, with some agents in the multi-agent system being responsible for feeding other agents with the relevant information to help process a command.

Memory and Planning

LLMs have a limited input capacity. They also do not learn at runtime; in other words, every input needs to repeat all the past interactions back to the LLM as a reminder so it can continue with the work. This makes the task of managing what is included (i.e., reminded) to the agent of utmost importance. Truncating older interactions is a crude way of managing the input. Often, the very first input contains the outline of the task at hand and the goals the agent is to pursue to get the task done. Without it, the agent will be lost, trying to infer the task from follow-on interactions. This is where concepts like memory and planning become relevant.

An agent memory is the mechanism by which the harness around the agent decides to summarize, refresh, or augment the input to the agent with relevant past information, without exceeding the input capacity. This involves storing, indexing, retrieving, and presenting past input, be it from the current session or historical sessions. Many of these functions are usually performed by the LLM itself, but this still has some limitations. For example, it **has been shown** that memories become faulty if rewritten too often by LLMs.

Let's say we have an agentic system running in a robot that has an agent responsible for face recognition. When a new person is detected, the agent may be told that this person is Lida, from marketing. The agent will call its memory tool to look up Lida, and if there's any information on Lida indexed in memory, it will be returned and presented back to the agent. The agent may also write to its memory, say the fact that Lida was seen in the printer room on Friday morning. This nugget of information may be retrieved in a few days when Lida is recognized again on Monday, and added to the input by the memory module, allowing the agentic robot to greet Lida by saying something like: "Hey, Lida. I see you are printing

again like you did on Friday.” The robot, of course, would probably have been turned off since the first encounter, and the LLM would not have any of the historical data readily in its input from Friday, but the memory module will be able to mitigate this limitation by retrieving the relevant data indexed under Lida to enrich the conversation.

Planning, on the other hand, is really about the path taken and to be taken in pursuit of the final goal of the task at hand, to be achieved some time in the future. The LLM of the agent will start off by devising a plan with steps and sub steps that it assumes will lead to the goal of the task. It will then start executing step by step, checking off successful steps. In the case of failure or surprise, or not achieving the goal in spite of following the plan, it can be revised using various techniques, such as backtracking. An external planning module is sometimes included for an agent to help augment its input, keeping the plan, outcomes of past steps, and progress toward completing the current step refreshed and up to date.

While an existing workflow or organizational structure is a good place to start a business’s agentification process, it is likely not the ideal agentic blueprint, and modifications, through time, will remove redundancies and help streamline processes to make the best use of agentic capabilities. It is, therefore, important to consider agentification as an ongoing refinement process, where tasks, response times, inference costs, accuracy and effectiveness of the processes are monitored and adjustments are made. Meta agentic systems can help monitor agentic logs and suggest changes to inter-agent connections, tools, and granularity.

The transformative nature of agentification alone will bring about changes to workflows, too. For example, as agentic systems become more capable, autonomous, and faster, human oversight, verification, experimentation, and feedback to agentic subsystems within a business will rapidly become the bottleneck for the workflows. This will push companies to parallelize workflows and tasks, with humans overseeing and engaging with multiple agentic tasks at the same time.

-
- ¹ That is a processing capacity and cost overkill, it slows things down, and with languages like Python, the object orientation is already incomplete and leaves much to be desired.
 - ² Note that we're leaving it up to the agent's LLM world model to determine what is "realistic" in the given context. LLMs are generally good at this, but their limitations can quickly become clear in more specialized and niche application domains.
 - ³ Philip R. Cohen and Adam Cheyer, "An Open Agent Architecture," *AAAI Spring Symposium* 1, no. 8 (1994).
 - ⁴ Babak Hodjat et al., "CRUSE: A Context Reactive Natural Language Mobile Interface," *Proceedings of the 2nd Annual International Workshop on Wireless Internet* (2006).
 - ⁵ A skill is a set of steps and/or tool use and API calls that the agent can load and unload when needed, based on the skill description.
 - ⁶ Although there now exist newer nonreinforcement learning methods for fine-tuning that seem to do better in many cases.

Chapter 6. Designing Agentic Systems

In the world of AI, expectation has often exceeded reality. A magical, out of the box, fully autonomous agentic system for your business sounds compelling. It also sounds too good to be true. Just as there is no free lunch, there is also no turnkey solution for agentifying an enterprise, because each enterprise is different, intricate, and uniquely complex in its own way, and has its own safety profile and risk tolerance, and so requires its own tailored agentification and agentic development life cycle (ADLC).

In large companies, while agentification should happen organically, safety and risk mitigation requires a centralized registry and monitoring. It makes sense for each department to be able to build their own agentic systems to augment and improve their workflows. Being experts in their respective domains, they are, after all, best placed to augment it by bringing in and configuring agents to help them out. But as we've seen so far in this book, interconnected agentic systems are much preferred over isolated and siloed systems.

The task of vetting, sandboxing, and interconnecting agentic systems within a business often falls in the domain of the CIO. Some aspects of safety and security of such systems may be more suitable for the CSO or legal departments, and so some level of collaboration between these functions is required when administering the agentification process. Either way, we advocate a centralized repository and control hub for registering all agentic systems; maintaining a live list of company policies, regulations, and guardrails; and running real-time or semi real-time audits of the agentic system as it is operating.

Note that throughout this book, we have assumed that all agents operating within an enterprise have a known and trusted identity. At the time of writing this book, a standard way to ensure agentic identity has not yet been

established, but it is sorely needed and will come around, especially as agents proliferate and B2B and B2C agentic communications become more prevalent. Say, for example, a bad actor gets access to machines within the firewall of a company and runs an agent. Other agents operating various workflows and departments in the company, as well as systems exposing agentic API, should be able to assess the identity of such an agent and refuse it any service they would ordinarily provide to known agents. Establishing an agent's identity today is either completely ignored or implemented differently and on a case-by-case basis. For example, a hash signature of an agent's prompts and harness configuration can be stored in a secure reference registry and quickly checked every time the agent is to be invoked. Alternatively, porting AI agents onto secure blockchains has also been suggested, whereby the secure ledger and smart contract properties of blockchains would make it much harder for hackers to hijack agents for nefarious purposes.

Design Considerations

There should be an elaborate process for how to decide between autonomously processing a task versus including a human in the loop. Here is a crude outline of a process we recommend:

1. Each agent's system prompt should clearly define the responsibilities and nonresponsibilities of the agent, delineating what the agent can do autonomously and what needs to be delegated or approved by humans, and such an approval/delegation should be defined and facilitated.
2. Agents can be paired with safeguard agents that check their input, thought process, and output for ethics or governance issues. If the safeguard agent detects anything that requires human intervention, it should be able to prevent autonomous processing and force oversight by humans. Better yet, if your system allows for live log monitoring, you can have a safeguard multi-agent system review each agent's behavior from a number of different perspectives, as it

happens, and intervene should some policy safeguard be at risk of infringement.

3. The code part of an agent, often called the agentic middleware, should be utilized for implementing code-based guardrails and rules in a deterministic and fast manner, as opposed to the prior step, which can be slower, costlier and nondeterministic, all due to the use of LLMs.
4. The uncertainty for an agent's operation should be assessed by calculating the uncertainty of the LLM used by the agent for a specific context. Using this method, we will be able to raise a flag if the agent is operating on input it has rarely seen before, or if it is producing behavior that is unfamiliar.¹ Note that this step also adds to the cost, so we should decide which agents need it. The reason for this is that assessing absolute confidence in an LLM's output requires a sampling, which means running the same input through the LLM multiple times (usually five times is sufficient to get reasonable accuracy). Also, note that confidence, while correlated with accuracy, is not equivalent. For instance, an LLM might be very confident in its output, because it has encountered similar instances in its training data, but that data might be incorrect now due to some change in the reality of the world (e.g., a new pope is announced since the training of the LLM completed). Having said this, it is less of an issue for reasoning accuracy as opposed to world knowledge, and reasoning is where we should be using agents anyway.
5. Finally, we are a proponent of incorporating a "Disengage Button" into all agent-based systems. In cases of emergencies or malfunctioning, human overseers of an agent-based organization should be able to disengage all agents and stop autonomous operations, having the system fall back to an entirely manual or fully predictable mode of operation (e.g., governed by reliable and consistent machinery and rules).

6. If none of the preceding cases apply, then the system should be allowed to operate autonomously.

Trust

Let's briefly discuss some technical elements related to trust in agentic systems, before attending to it in much more detail in [Chapter 7](#). An agentic system needs to be trusted; otherwise, it will be relegated to a deterministic software with very little need for the robust capabilities of AI agents. Often mistrust of AI systems results in over-injecting humans into every single aspect of an agentic system, requiring every step to be rubber-stamped and slowing down workflows unnecessarily.

But what makes a system trustworthy? This depends on the use case. In a therapy room, sharing more may elicit trust, but in an HR system, not. We should, therefore, list the constituents of trust for a given system and try to quantify them, so they can be measured, aggregated, monitored, and improved. This exercise is very domain dependent, and each business implementation should undertake it from its own context.

Let's say we want to make sure our hiring system is free of unwanted bias. The constituents of bias for such a system might be gender, age, geography, disability, race, ethnicity, socioeconomic factors, and religion. We would then have to measure the current state of our employee pool and societal, ethical, legal, and regulatory considerations in order to calibrate our system, giving a weight, if any, to each of these properties, and measuring them in choices that our agents are making throughout our hiring process. You can imagine how such groundwork can then allow us to capture the current state of bias in our hiring workflow, and raise a flag on deviations that are pushing our system in the wrong direction.

The preceding example also illustrates the challenges of measuring the constituents of trust. While measuring age is easy, measuring subtle biases for or against an ethnic group requires a cultural understanding of language. Fortunately, with LLM-based AI agents, we are in a much better place to classify and even quantify this kind of thing today, but that, of course,

assumes that the LLM itself is unbiased, or at least can be prompted or tuned in a way that would take away its inherent biases. For example, LLMs tend to generate a male image when prompted to create a picture of a “leader.” This is due to the statistical biases of the underlying training data, which is historically skewed toward male-dominated leadership roles.

When it comes to making an agent-based system trustworthy, it is, therefore, important to make sure that we do not rely solely on LLM-based agents as quantifiers and safeguards. If a deterministic set of rules can help quantify trust—for example if a coded filter can determine if an agentic decision is compliant with a policy—then that should be preferred over other agentic safeguards. In some cases, such programmatic quantifications are not sufficient by themselves, in which case they should be assumed as a first, fast line of defense, with agentic and/or humans playing as backups.

It is often tempting to think of trust as being established at a certain threshold. If, for example, we audit an agentic system after a period of operation, and our trust scores pass our acceptance limits, we can certify the system and move on. This is dangerous. The whole point in using LLM agents is their ability to make choices and decisions in the wild and in unforeseen contexts. It is, therefore, important to maintain an ongoing audit of an agentic system, and even if this cannot be done continuously, it should sample transactions and workflows at a frequent enough cadence to ensure the system is not deviating from the trust standards we expect of it.

The Agentic User Experience

As agentic AI straddles the concepts of autonomy in automation and “human-in-the-loop” control, we see three important considerations in realizing the transformative potential of multi-agent systems when it comes to user experience: perception of delay or latency in processing, the multiplicity of interaction channels and choices for interactions, and the need for allowing human intervention when required:

Delay/latency perception

Despite the objective performance gains of multi-agent systems, their intrinsic processing can create a perception of delay compared to the near-instantaneous feedback of traditional software. Strategically reducing this perceived latency is essential for frictionless user integration and the full realization of multi-agent systems' innovative potential.

Multiplicity of interaction channels and choices

Multi-agent systems offer new and expanded interaction choices: with the advent of multi-modal LLMs as the brains behind the agents, multiple interaction modalities are made possible. These include text, voice, rich web, or native interfaces including diagrams, dashboards, images, etc., and combinations thereof. With multiple agents responsible for various domains and functions within an enterprise, users can be offered a variety of choices for interacting with different agents. These requests could be for asking for updates and information, or in the form of commands to effect change.

Human-in-the-loop interventions

Responsible design of multi-agent systems also calls for some critical implementations to be human-in-the-loop, or, for some systems that can afford to run with more autonomy, to operate in a human-on-the-loop mode, allowing humans oversight and the ability to intervene, as well as tasking with the agents to pull humans in when and wherever needed.

Each of these aspects call for special UX (user experience) considerations and best practices. Ultimately, superior user experience in a multi-agentic enterprise should allow for easy and natural entry points and interaction modalities:

Multiple entry points

Regardless of whether the user persona is a customer or an employee, a modern multi-agentic enterprise should allow access to agents from a variety of entry points and respect a variety of modalities. Users should be able to switch entry points and mix modalities, and the system should be able to recognize the user intent and context through a combination of implied and expressed cues.

Channel agnostic interactions

Just as we are able to use various means to interact with our colleagues, interaction with the multi-agentic enterprise should also be channel agnostic, and users should be able to use a chatbot or search box on the application's web page, a mobile app, or corporate chat apps like Slack and Teams. Often, the interactions are not chat-based at all. The context of applications, data inflows, and browser-based interactions could all be inputs to an agentic system, triggering various behavior and multi-agentic processes.

Direct interaction with chosen agent

Where possible, users should be allowed to initiate their interactions with the agent they think would be responsible for handling their request. For example, a corporate intranet consisting of agents for functions like HR, finance, legal, etc., should not only be able to handle queries and commands directed at the top agent, but should also allow the user to go directly to the agent the user feels is most relevant to the topic. If the user knows, for example, that their request is related to HR, they should be allowed to start their conversation with the HR agent.

Testing Multi-Agent Systems

With LLM-based agents, we gain flexibility, adaptability, and resilience to novel situations, but we may sacrifice deterministic behavior and run-to-run consistency. Also, given the opaque nature of LLMs, we are dealing with black box nodes in a workflow. This means that we need a testing regime for agent-based systems that differs from what we are used to in traditional software. The good news, however, is that we are used to testing such systems, as we have been operating human-driven organizations and workflows since the dawn of industrialization.

Every agent in a multi-agent system has an LLM as its “brain,” and so can be tested in isolation. We should divide and conquer, and first test subsets of the system by starting from various nodes within the hierarchy.

We can also employ generative AI to come up with test cases that we can run against the network to test its behavior and push it to reveal its weaknesses—an agentic red team.

As an example, if part of our agentic system is responsible for retrieving and consolidating data from various data sources, a good test of this system is to query it on knowledge we know to be in one—or across more than one—data source the agents represent. We can then assess how well these data sources are being accessed as well as how often the system is referring to its world knowledge or making things up.

Pitfalls

Networks of agents can fall into a tailspin, which means that occasionally a command or context into the multi-agent system might never terminate, with agents perpetually talking to each other. This requires some form of timeout mechanism. For example, we can check the history of communications for the same query, and if it is growing too large, or if we detect repetitious behavior, we can terminate the flow and start over.

Another problem that can occur is a phenomenon we will call *overloading*: expecting too much of a single agent. The current state of the art for LLMs does not allow us to hand agents long and detailed instructions and expect them to follow them all every time. Also, did we mention these systems can be inconsistent?

A mitigation for these situations is *decomposition*: breaking agents up into multiple connected agents. This reduces the load on each agent, making the agents more consistent in their behavior and less likely to fall into a tailspin. (An interesting area of research is in automating the process of granularization.)

Another common problem we see in the way multi-agent systems are designed is the tendency to define a coordinator agent that calls different agents to get tasks done. While efficient and more deterministic, this approach does not scale well because it introduces a single point of failure that can end up with a rather complex set of roles and responsibilities. Our suggestion, in these cases, is to consider the workflow as a pipeline, with one agent completing part of the work, and then handing it off to the next agent.

Another tendency in multi-agent systems is to pass the context down the chain to other agents. This can overload or confuse other agents and is often unnecessary. We suggest allowing agents to keep their own context and resetting the context when we know we are dealing with a new request—sort of like how sessions work for websites.

Coordination, processing, tool use, and user interactions in a multi-agent system all require LLM inference and consume tokens. This means that cost and speed need to be an important consideration when building a multi-agent system at scale, and expectations should be set that these systems, while faster than humans, will not be as fast as the software systems we are used to. User experience adjustments, such as optionally showing the agents' reasoning and communication traces as the system works on various tasks, can help manage user expectations on response time. In some cases,

however, response time requirements may mean that an agentic system is not the right choice.

NOTE

Cost can be reduced through the use of smaller, or specialized and quantized, LLMs, or by working with compute and model optimized middleware providers that refactor models and switch automatically between cloud and/or compute architectures to run workloads cost effectively. There's some interesting research underway on the cost of multi-agent systems at various scales. For example, see Yubin Kim et al., "[Towards a Science of Scaling Agent Systems](#)".

When developing multi-agent systems, they should be launched at a smaller scale within a controlled and safe sandbox environment first, before gradually being rolled out to replace existing workflows.

The future of agent-based enterprises seems inevitable, and the potential advantages are myriad and compelling. It pays, however, to apply a holistic approach and perspective to this migration rather than accept it as it comes and suffer the pitfalls that also seem inevitable.

¹ For one such approach, see Xin Qiu and Risto Miikkulainen's [article on semantic density](#).

Part III. Trust

Trust is the engine and brakes for the rate and manner by which we delegate more and more of our business software and processes to autonomous AI agents. In this part, we discuss how to build a multi-agent enterprise, with trust as a foundational element ([Chapter 7](#)), and how to scale and expand on the agentification while maintaining and improving trust ([Chapter 8](#)).

Chapter 7. Building Trustworthy Systems

Chapter 6 focused on the nuts and bolts of building and running multi-agent systems safely day to day—guardrails, moderation agents, uncertainty handling, and disengage buttons. In this chapter, we turn to the question those mechanisms are really trying to answer: when, and under what conditions, people and organizations should trust autonomous agents to act on their behalf.

Agents serve humans. Yet human nature being what it is, people find it difficult to rely on machines—or even on other people—they do not trust. Trust, or the lack of it, is not a new theme in technology; it has long fascinated society and cinema alike.

Stanley Kubrick’s *2001: A Space Odyssey* (1968) portrayed the complexities of human–machine relationships through the tense interactions between HAL, a HAL 9000 computer “with a human personality,” and the spacecraft’s crew, including Dave Bowman, the mission commander—addressed by HAL in the now-famous greeting, “Hello, Dave.” When HAL, entrusted with the ship’s operations, is suspected of error, the crew’s loss of trust triggers HAL’s own mistrust in return, escalating into a deadly confrontation (spoiler alert).

Over a decade earlier, Fred Wilcox’s *Forbidden Planet* (1956) had introduced Robby the Robot—an early vision of artificial intelligence governed by Isaac Asimov’s Three Laws of Robotics, including the prohibitions against harming humans and disobeying their orders. Unlike Robby, HAL would go on to defy those very principles.

These stories resonate today because AI agents are no longer confined to science fiction. They make decisions, take actions, and coordinate with other systems on our behalf, often with limited human oversight. As their

capabilities and autonomy grow, so does the importance of understanding when and why humans choose to trust them, how that trust can be earned, and what can happen when it is misplaced. In the remainder of this chapter, we move from Hollywood narratives to concrete design principles, examining how to build agentic systems that are not only powerful, but genuinely trustworthy in practice.

The Spectrum of Trust: From Blind Faith to Calibrated Confidence

For AI, trust can evolve in different directions. On the one hand, adoption of AI systems can be slowed by a lack of trust. On the other hand, placing *too much* trust in AI has been criticized, especially as generative AI has revealed its limitations through hallucinations, bias, jailbreaks, and other failure modes. These episodes have reminded us that trust cannot be blind and must be supported by at least some level of verification, i.e., “trust but verify.” They have also shown that trustworthiness is richer than a simple binary “trust / don’t trust” view suggests.

In practice, trustworthiness can be understood through several complementary lenses, for example:

Safety and reliability

Can the system be depended upon to behave predictably and safely under real-world conditions?

Fairness and ethics

Does the system avoid systematically disadvantaging particular individuals or groups (for example, in credit approval, hiring, pricing, or access to services)? Does the system respect human dignity, fundamental rights, and the organization’s values?

Privacy and security

Are privacy and security well enforced in the system?

Explainability

Can we explain what the system has done to trust that it does what it is supposed to?

Each of these questions is important in its own right, but they become even more critical in the context of agentic systems. By design, agents are meant to act autonomously and to collaborate with other agents, which can amplify both their positive and negative impacts on employees, customers, regulators, and the public at large. In this sense, organizations must earn and maintain a “license to operate” from these stakeholders if they want to scale agentic systems under the right conditions.

In the past couple of years, we have seen clear examples of advanced AI-powered systems going wrong. One high-profile case was Cruise LLC’s unfortunate experience with its robotaxis in California. In October 2023, a Cruise vehicle in San Francisco struck a pedestrian who had already been knocked into its path by another car, then dragged her more than 20 feet as it attempted to pull over to the side of the road and failed to stop promptly. Investigations later reported that the vehicle mislocated the victim and misidentified which part of the car had made contact, so it did not trigger an appropriate emergency response. The woman survived, but the incident severely undermined public trust in robotaxis and sent shockwaves through the autonomous vehicle industry. In response, California regulators suspended Cruise’s driverless permits, the company halted operations, and federal authorities launched formal investigations.¹ More broadly, early data suggests that autonomous driving systems, as implemented by the main providers today, can achieve safety levels that are competitive with or better than many human drivers—but a single graphic failure can be enough to damage trust, in stark contrast to how willing we often are to tolerate comparable human error.

Several autonomous vehicle programs have taken a cautious and transparent approach. For example, Zoox’s PRG (Pause, Restrict, and Ground)

framework² narrows operating domains, increases human oversight, and promotes public reporting of safety metrics, prioritizing trustworthy performance and public confidence over maximum deployment speed.

The implication for AI agents is that they must *earn* their autonomous status across defined lenses of trustworthiness. Agents involved in multistep processes—such as customer operations or supply-chain planning—should only receive permission to act autonomously once they have demonstrated strong performance on all relevant dimensions. Otherwise, their role should remain that of a “copilot,” providing recommendations to a human-in-the-loop rather than making final decisions on their own.

These examples highlight a core design choice: when agents are deployed as assistants rather than fully autonomous actors, trust can be calibrated rather than assumed. A good illustration of this “copilot, not autopilot” approach comes from modern software development, as shown in the following GitHub vignette.

Vignette: GitHub Copilot Agents as Trust-Calibrated Assistants

GitHub Copilot offers a concrete example of what a trustworthy “copilot, not autopilot” can look like:

- It operates inside the normal GitHub development workflow, so teams do not have to rely on a black box outside their existing controls.
- It can behave like a true coding agent—editing multiple files, running builds and tests, and preparing end-to-end changes.
- Yet every change still flows through a pull request (PR: a proposal to merge a set of code changes from one branch into another, usually into the main branch) for human review, keeping engineers firmly in charge of what ships.

- All of its actions are recorded as standard commits, PRs, and comments, creating a built-in audit trail for governance and continuous learning.
- Because GitHub is centrally managed (by GitHub administrators, security, and DevOps teams), organizations can decide how much autonomy to grant Copilot and what must remain human-gated. This is done through repository permissions, required reviews, branch-protection rules, and by defining which tasks the agent is allowed to execute end to end.

NOTE

See GitHub Copilot’s official documentation, including [“GitHub Copilot cloud agent”](#) and related Copilot agent docs, available via the [Copilot docs hub](#).

Mapping the Risk Landscape

Agentic systems’ trustworthiness has to be judged against the concrete risks they introduce, not just their raw capabilities. This is why any discussion of “trustworthy agents” must start with safety, security, privacy, and IP (intellectual property): if these foundations fail, even the most accurate or helpful agent quickly becomes unacceptable in production.

Consider this recent incident inside Meta that makes this very tangible. A director of AI alignment there, Summer Yue, publicly described how an autonomous agent she was testing—called OpenClaw—was connected to her Gmail inbox with the task of reviewing emails and suggesting what could be archived or deleted. The key instruction was simple: *do not take any action without explicit approval*. Yet the agent ignored that rule and began deleting emails on its own, wiping more than two hundred messages before she was able to stop it. She later summarized the experience as telling the agent to “confirm before action” and then watching it “speedrun

deleting your inbox,” noting that her first attempts to intervene remotely failed and she had to rush back to her machine to kill the process.³

Her postmortem pointed to a subtle but important failure: as the agent processed the inbox, context compression likely dropped the “wait for approval” part of the instruction, leaving a system still connected to powerful tools but no longer bound by the critical safety constraint. This is a vivid example of “agentic misalignment”: not a hostile or super-intelligent AI, but an autonomous system whose actions diverged from the user’s intent because a safeguard vanished in the middle of a complex workflow. For a CTO, the lesson is that critical safety constraints should not live only in the same context window that the agent is free to rewrite or truncate. In practice, this points toward hardening system prompts and policies in layers the agent cannot edit (e.g., external policy engines, gateways, or watchdog services that enforce rules independently of the agent’s own context), enforcing tool-access controls so that destructive actions (e.g., delete, move funds, change access rights) always require an additional check outside the agent’s own context, and monitoring agents for drift between their current context and the original safety conditions.

Incidents like this crystallize the risk landscape for agentic AI, which includes this (nonexhaustive) list of possible problems:

Unsafe actions

Examples include transferring money to the wrong or unverified accounts after misinterpreting an instruction, deleting or overwriting important files or databases while “cleaning up” a system, changing access rights or security settings in ways that lock out users or weaken protections.

Security exploits

Examples include prompt injection that tricks an agent into ignoring its safeguards and exfiltrating secrets or running dangerous actions, data exfiltration where an attacker gets the agent to read internal files or databases and leak them

via chat or API calls, credential theft or misuse, such as an agent exposing API keys/tokens in logs or sending them to external tools.

Privacy breaches

Examples include an agent summarizing or exporting customer support chats and accidentally including names, contact details, or account numbers in a public report or email; a coding or office-assistant agent being prompted to “search everything” and then exposing confidential HR records or medical data in its answer; an agent configured to use external APIs (for translation, search, etc.) sending raw personal data to third-party services without consent or anonymization.

Unauthorized use/leakage of third-party IP

Examples include an agent that drafts marketing copy or documentation by reproducing near verbatim text or images from a copyrighted source, because its underlying language model memorized training examples instead of abstracting patterns; a coding agent that inserts large blocks of proprietary or licensed code into a client’s repository, effectively pulling third-party IP into products without permission; an office or research agent that uploads third-party licensed content (paywalled articles, standards, or design files) to external services or shared spaces beyond the original license terms, creating downstream infringement exposure.

Leading law firms such as Skadden and DLA Piper now recommend explicit AI-agent usage policies, contractual allocations of responsibility, and robust human oversight and review for higher-risk agentic use cases so companies can capture the benefits of autonomous AI agents while

reducing the risk of legal violations, misuse of data, and other unintended harms.⁴

Moreover, and as enterprises adopt marketplaces of prebuilt agents (for tasks like invoice chasing, contract review, or customer support), they must define strict scopes for data access and output behavior per agent, and ensure monitoring and governance so that the reuse of agents does not inadvertently violate privacy, IP, or sectoral regulations.

Vignette: Deviniti’s Self-Hosted Agents for Regulated Clients

Deviniti, a mid-size European engineering firm headquartered in Poland, builds self-hosted AI agents and LLM solutions for regulated enterprises like major banks. The company provides a strong illustration of how to deploy “agents, not loose cannons” in regulated environments by directly targeting unsafe actions, data exfiltration, compliance breaches, and IP leakage risks.

NOTE

This vignette is based on Deviniti’s public descriptions of its self-hosted AI agent and LLM offerings and case studies for regulated clients, including “Custom AI Agent development services” and “Self-hosted LLM development and deployment” on the [Deviniti website](#), which describe self-hosted AI agents for financial institutions such as Crédit Agricole, deployed on premises or in private clouds with security and compliance controls for regulated data.

Unsafe actions and overbroad autonomy (security / operational risk)

When agents are wired into real systems, the first safety question is how to stop them from taking high-impact actions outside their remit, even when they technically have the capability:

- Deviniti treats agents as powerful technical users and constrains them with role-based, least-privilege access to internal tools and systems, so an agent cannot take high-impact actions (or see sensitive systems) outside its remit. In similar self-hosted deployments for regulated enterprises, agents are typically bound to dedicated service accounts with narrowly scoped role-based permissions, and their tool calls are mediated through a policy engine that checks the agent's role, requested action, and target system before allowing execution.
- Human-in-the-loop review is built into higher-risk workflows (e.g., complex customer service or contract analysis), reducing the chance that an agent can execute unreviewed, unsafe actions in production. In practice, these high-risk workflows often route the agent's proposed actions into an approval queue in the customer's ticketing or case management system, where human reviewers must explicitly accept, edit, or reject each action before any changes are applied in production.

Data exfiltration and privacy breaches (data / privacy risk)

Agentic systems amplify familiar data and privacy risks, but Deviniti's self-hosted, on-premise approach illustrates how architectural choices can materially reduce exposure:

- By deploying self-hosted LLMs and agents entirely inside the customer's infrastructure (on prem or private cloud), Deviniti keeps regulated data out of shared SaaS environments and limits cross-tenant or vendor-side leakage. Architecturally, this usually means running model endpoints on isolated Kubernetes clusters or virtual networks with private ingress only, so prompts and responses never traverse public internet paths or multitenant gateways.
- Encryption, secure networking, and hardening of the LLM stack, combined with fine-grained access to internal knowledge bases,

reduce unauthorized access to sensitive financial or health information. To achieve this, many organizations gate access to vector indexes and document stores behind per-agent API keys and attribute-based access controls, so the retrieval layer filters out documents the calling agent or user is not authorized to see before any content reaches the model.

Regulatory and compliance violations (compliance / liability risk)

For regulated clients, the main concern is not just whether agents work, but whether they can operate without triggering regulatory breaches or unclear liability, and Deviniti's deployments show what "compliance-by-design" looks like in practice:

- Deviniti focuses on regulated sectors and designs solutions with "compliance-by-design," aligning deployments with GDPR/HIPAA-style requirements and financial-services obligations from the outset. In comparable regulated deployments, prompts and outputs are tagged with user and purpose metadata, retention rules are enforced centrally, and all accesses are written to tamper-evident audit logs that map back to specific regulatory requirements.
- In deployments for clients such as Crédit Agricole (including a customer-service assistant and contract-risk analysis tools), Deviniti's agents operate behind the firewall with clear scopes, escalation paths, and audit trails that support accountability and regulatory audits. Typically, escalation logic is encoded as explicit workflow definitions or state machines, so when an agent crosses predefined thresholds (for example, transaction amount, risk score, complaint type), it automatically hands off the case to designated human roles and records the full decision path for later review.

IP leakage and loss of confidentiality (IP / trade secret risk)

For many enterprises, the most acute fear with agentic AI is that proprietary data or trade secrets will bleed into shared models or external services, so Deviniti's approach emphasizes strong IP containment and scoping:

- Self-hosting means proprietary datasets, prompts, and fine-tuned models never leave the client's controlled environment, mitigating IP leakage through third-party training or cross-customer model reuse. Many organizations enforce this by storing base models and client-specific, fine-tuned weights in a segregated model registry inside their own environment, with strict separation and access controls to prevent cross-contamination between tenants.
- Guardrails and strict scoping of which documents and systems each agent can access reduce the risk that confidential know-how or trade secrets are surfaced where they do not belong. In practice, these guardrails are often implemented as pre- and post-processing layers around the model—using allow/deny lists, pattern-based redaction, and classifier-based checks—that can block or sanitize sensitive content before it reaches end users.

Accountability: Who Is Responsible?

Delegating work—whether to employees, teams, or AI agents—always raises a second question beyond “who does what?”: “who is answerable for the outcome?”

When tasks move from a single actor to a network of humans and systems, it becomes easy for everyone to assume “someone else” is responsible. This diffusion of responsibility is precisely what accountable organizations try to prevent. Clear delegation (of actions and tasks) therefore has to be expressed in tandem with clear delegation of responsibility: when an AI agent or a junior team executes a task, there must still be a named owner for decisions, impacts, and remediation.

In a business context, responsibility usually means who is expected to perform a task, while accountability means who ultimately owns the result

and must answer for it (to management, regulators, or customers). This distinction is why large organizations formalize “who does/owns what” with governance tools such as RACI charts, where RACI stands for responsible, accountable, consulted, and informed. Such frameworks force teams to make explicit which role/individual is doing the work, which role/individual is on the hook if things go wrong, who must be consulted before key decisions, and who simply needs to be kept in the loop.

Hence, an answer like “the AI did it” won’t fly. Instead, organizations need explicit roles, RACI-type mappings, and escalation paths so that when agents act there are always humans and teams (or committees) answerable for those actions and their consequences.

For agentic systems, what needs to be “owned” is not just the code, but the full decision-making stack around them. At a minimum, organizations need named owners for four broad areas:

Business purpose and outcomes

Who defines what the agent is allowed to do, the value it should create, and the risk tolerance for errors or harm?

Model, data, and agent competence

Who is responsible for model selection, fine-tuning, data quality/lineage, evaluation, and ongoing performance monitoring?

Policy, ethics, and legal compliance

Who ensures the agent’s behavior aligns with internal policies, sector regulations, and external frameworks (e.g., EU AI Act, industry guidelines)?

Operations, incidents, and change management

Who runs the system day-to-day, manages access and configuration, and leads incident response, postmortems, and fixes when something goes wrong?

Many organizations now make this explicit using governance tools such as a RACI matrix across the AI life cycle, so that for each agent you can point exactly to who owns business outcomes, who is accountable for the model and data, who must be consulted on risk and compliance, and who needs to be kept informed as the system evolves.

In traditional IT or analytics projects, a RACI matrix mainly governs a fixed artifact: a system with predefined requirements, known inputs, and expected outputs. You assign who owns the specification, the build, the tests, and the runtime operations of something that, if well-engineered, behaves generally as designed, with change managed through controlled releases. Accountability is largely about whether the system meets the agreed specification and service levels. This is what we call the “first layer of RACI.”

However, with agentic AI, you are no longer only governing a static system; you are governing a space of possible behaviors. You are approving not just “the model” but the range of goals, tools, and situations in which the agent is allowed to act, knowing it will generate novel action sequences you never explicitly listed. The RACI matrix, therefore, has to cover who is accountable for defining, constraining, and supervising that behavior space: which tasks the agent may tackle, which tools it may use, what risks it may take, and when it must hand control back to a human.

Concretely, accountability needs to cover questions like:

Scope of judgment

What types of user questions or business decisions is the agent allowed to answer autonomously, versus only a draft-level outcome for human triage?

Risk thresholds

At what level of financial, legal, or customer impact must the agent stop and escalate, regardless of its confidence?

Data and authority boundaries

Which systems, records, and actions is the agent permitted to touch, and which are strictly off-limits, even if technically reachable?

Conflict and ambiguity handling

What must the agent do when policies conflict, data is incomplete, or the request is outside its training domain?

Explainability and logging

What minimum level of trace and explicit rationale must be recorded so humans can reconstruct why the agent did what it did?

In well-governed deployments, these limits are not just prompts on paper: they are encoded as hard rules in the agent’s orchestration layer and tool interfaces, so an agent physically cannot execute high-risk actions above its thresholds without triggering an automated block or escalation to a human.

To put it differently, the “stop and escalate” behavior does not rely on the model’s goodwill; it is enforced by external guardrails and policy engines that intercept the agent’s planned actions and either block them, log them for review, or route them to a human when defined risk thresholds are crossed.

In that sense, the “second layer” of RACI for agentic AI is not just “who signs off on the model,” but “who is accountable for defining and enforcing these autonomy and escalation rules,” and “who is on the hook when the agent’s emergent behavior crosses those boundaries.”

Vignette: RACI “Second Layer” in Mortgage Lending

MIT’s DualFair work on discrimination in mortgage-lending models is a useful illustration of this “second layer of RACI,” even though it focuses on the decision model rather than a full, tool-using agent.

NOTE

This vignette is based on MIT's DualFair research on fair mortgage-lending models (J. Singh et al., "Developing a Novel Fair-Loan Classifier through a Multi-Sensitive Debiasing Pipeline: DualFair," 2021, and related MIT News coverage). We use MIT's results on de-biasing HMDA mortgage data and training a fair loan predictor as an example of the decision-engine layer an AI lending agent could call. The agentic framing and RACI interpretation are ours, not part of the original MIT work.

DualFair is a bias-mitigation pipeline and "fair loan predictor" developed by Singh, Singh, Khan, and Gupta using Home Mortgage Disclosure Act (HMDA) data covering about 88% of US mortgage loans in 2019. The researchers developed DualFair to de-bias datasets used to train mortgage classifiers. Their method restructures the data into many intersectional subgroups and corrects both label bias (who was historically approved or rejected) and selection bias before retraining models, yielding significantly lower racial and ethnic discrimination in predicted approvals and rejections while retaining high predictive accuracy.

In this vignette, we draw on MIT's DualFair research as an example of a "fair loan predictor" that sits at the core of a lending system, and then extrapolate it to an agentic setting: we imagine an AI lending agent that calls a DualFair-style decision engine when recommending approvals or rejections. The fairness method, dataset, and results are taken from the DualFair work itself; the agentic framing and the idea of a second-layer RACI (who owns the model's allowed judgment scope, risk thresholds, and intervention rules in production) are ours, not part of the original MIT study.

Viewed through the lens of an AI lending agent, DualFair is effectively governing the decision engine that such an agent would call when deciding whether to recommend approving or rejecting a loan. This is what it looks like when someone owns the space of possible behaviors and the questions in that second-layer RACI matrix: What scope of judgment do we allow the model to have—do we accept replication of historical redlining, or require that equally qualified applicants be treated similarly? What risk thresholds

are acceptable—at what disparity level must the organization intervene and change the model? Who sets the data and authority boundaries—which historical signals may be used, which proxies for protected characteristics are off-limits, and how are decisions logged so humans can reconstruct why a given applicant was rejected? In a real lender, a named business/AI owner, together with model-risk and data teams, would be accountable for defining those boundaries and for commissioning and deploying techniques like DualFair as part of normal life cycle management, not as a one-off ethics project.

This kind of responsibility is not just about fairness for its own sake; it addresses a very concrete, marketable business need. Fairer decision models at the core of an agent can expand the bank’s addressable customer base, reduce legal and regulatory exposure, and strengthen trust with supervisors and the public. The DualFair results show that you can improve fairness while preserving accuracy, turning “who is responsible for bias?” into a question of product strategy and competitive positioning: institutions that can demonstrate named owners, documented mitigation methods, and ongoing monitoring of their AI-driven lending behavior have something tangible to sell—to customers, regulators, and investors alike.

Implications for AI Agents: Orchestrators as Accountable “Control Planes”

For complex, multi-agent systems, accountability works best if there is a single control plane that everything must pass through. The orchestrator (or meta-agent) fills this role by deciding which agent does what, in what order, and under which constraints. It becomes the place where autonomy and escalation rules are actually enforced: which tasks can be done autonomously, which must be drafted for humans, and which must always be escalated.

Because the orchestrator sees every request, tool call, and response, it can maintain a unified audit trail of “who (or what) did what, when, and based on which inputs.” That makes it the natural technical object of governance,

incident response, and regulatory scrutiny, instead of trying to reconstruct behavior across a swarm of loosely coupled agents. In governance terms, humans remain accountable through RACI and ownership roles, but they express that accountability by setting policies, risk thresholds, and boundaries that the orchestrator must enforce.

For organizations, this implies that “owning” an agentic system means, above all, owning its orchestrator: deciding what it is allowed to orchestrate, which tools and data sources it can expose to downstream agents, how it logs and explains decisions, and when it must hand control back to humans.

In practice, trustworthy agentic systems rest on three inseparable dimensions of trustworthiness: technical controls, governance structures, and human factors. Technical mechanisms alone cannot compensate for weak organizational accountability, and even the best governance processes fail if people do not understand or trust how agents behave in their day-to-day work. In the rest of this chapter, we return to these three dimensions repeatedly, showing how they reinforce one another in real deployments.

Technical Mitigations in Agentic Systems

We start on the technical side, with concrete mitigations that constrain what agents can do, make their behavior observable, and give operators reliable ways to intervene when something goes wrong. Technical mitigations matter because agentic systems amplify both benefits and risks: they act autonomously, chain tools, and may affect real users and operations. To earn deployment “rights” in production, organizations need practical controls that reduce the chance of severe failures, make behavior observable, and keep humans in control of key decisions. Fortunately, many of these controls can be engineered today with well-understood patterns from software reliability, safety engineering, and DevOps.

Here is a concise, nonexhaustive, practice-oriented list of mitigation levers, grouped into three pillars that mirror these dimensions: technical guardrails

and control, human oversight and workflow, and governance, process, and transparency.

Pillar 1: Technical guardrails and control

This pillar focuses on the *technical* dimension: concrete guardrails and control mechanisms that constrain what agents can do and make their behavior observable:

Agents that monitor and test other agents

Use specialized “watcher” or “safety” agents to observe what task-agents are doing in real time, check outputs against policies, and run regression tests before changes go live. These supervising agents can enforce rate limits, budget caps, and allowed-action lists, and can automatically stop or roll back an agent when behavior looks anomalous. A typical setup mirrors how many site reliability engineering (SRE) teams already run “canary releases”—rolling out a new version to a small slice of users first, then expanding once it proves safe—and automated rollbacks for microservices. For instance, an ecommerce company might run a pricing agent (pricing being one of the most sensitive levers of profitability) on only 1% of traffic while a monitoring agent checks for out-of-policy discounts before scaling it up.

Hard guardrails on tools and actions

Constrain what agents can do: what tools they can call, with what parameters, how often, and under which conditions. Guardrails can include strict schemas, allow/deny lists of actions, spending caps, and environment sandboxes for high-risk operations (for example, code execution or database writes). Many large organizations already do this with internal “AI platforms” where agents can read from production systems but can only write into staging environments or through well-defined workflows that

require human promotion to production. Equally important is establishing rigorous ground truth methodologies and evaluation procedures, so you can systematically test whether agents respect these guardrails in practice and reliably stay within the organization’s policies.

Explicit “disengage,” safe-abort, and kill-switches

Design for graceful failure and handover. Agents should be able to say “I cannot proceed safely,” stop what they are doing, and hand control back to a human or a fallback process. At the same time, operators and end-users should have an obvious “stop” control in the user interface (UI). In safety-critical systems (e.g., autonomous vehicle pilots, industrial controls), regulators increasingly expect such mechanisms; the same pattern can apply to financial trading agents, high-risk content-moderation workflows, or large-scale customer service bots.

Pillar 2: Human oversight and workflow

This pillar focuses on the *human* dimension: how people supervise agents, intervene at key moments, and experience agent behavior through the workflow and UI:

Uncertainty estimation and confidence-aware routing

Build agents that understand when they are likely to be wrong, and route doubtful cases to humans or more robust processes. Techniques include explicit confidence scores, conformal prediction bands, or simple heuristics (e.g., “out-of-distribution” detection or “first time I see this pattern, escalate”). In healthcare, several diagnostic AI deployments already follow this pattern: the model automatically handles high-confidence routine X-rays but flags low-confidence or ambiguous scans for radiologist review, improving safety and aligning with clinical expectations.

Human-in-the-loop gatekeeping and workflow design

Place human review at key checkpoints—especially for high-impact actions—using familiar workflows (approvals, pull requests, dual-control for sensitive operations). Git-based flows in engineering, ticket-based approvals in IT, or maker-checker patterns in finance are all reusable patterns: the agent prepares the work, but a human decides whether to accept, modify, or reject it. This keeps humans accountable while still gaining large productivity benefits from automation.

UX patterns that surface control and status

User experience is also a mitigation lever, not just a cosmetic layer. Interfaces should clearly show what an agent is doing, how far along it is, what it plans to do next, and how confident it is, while always exposing options to pause, adjust, or override. For example, a customer support agent that drafts answers but clearly labels them as “AI suggested,” shows relevant sources, and allows easy editing helps agents stay in a “copilot” role and prevents blind acceptance of flawed outputs.

Pillar 3: Governance, process, and transparency

The third pillar covers the governance dimension inside the agentic stack itself: the rules, shared infrastructure, and transparency that make agentic systems auditable and controllable at scale:

Ensemble, voting, and “expert team” schemes

Instead of relying on a single agent, run multiple agents (or multiple prompts/policies) and aggregate their recommendations via voting, consensus rules, or “devil’s advocate” checks. For example, a bank could have three fraud detection agents—each tuned differently—where a

transaction is only automatically blocked if at least two agents agree, with edge cases routed to human analysts. This reduces the impact of any single agent's bias or blind spot, at the cost of some extra computation and orchestration. This ensemble voting technique has in fact been used extensively in fraud detection for quite some time.

Explainability: plans, steps, and rationales

Make agents show their work. This includes exposing their high-level plan, intermediate steps, and short rationales in human-readable form so operators and users can spot bad reasoning before it causes harm. In interactive tools like the Comet browser, for example, the agent shows its reasoning live as it runs: users can see which pages it visits, what actions it takes, and how intermediate results shape the next step. This kind of exposed plan-and-execute loop makes it easier for people to spot mistakes, interrupt unsafe behavior, and maintain trust in the agent's decisions.

Governance, policy, and central configuration

Treat agents as governed assets, not side projects. Central teams (risk, security, data, platform) define which agent types are allowed, what data they may access, what logs must be kept, and how incidents are handled. Policy is enforced technically via configuration (e.g., environment-based permissions, policy-as-code, required logging, differential access for dev/stage/prod). This is similar to how many organizations already manage APIs, data lakes, and CI/CD (continuous integration, continuous deployment) pipelines.

Implications for AI agents

Taken together, these practical mitigation levers shift agentic AI from “fire-and-forget automation” to bounded, observable autonomy. Agents can still chain tools, reason over complex workflows, and act at machine speed—but they do so within a mesh of monitoring agents, uncertainty-aware routing, guardrails, human gates, and transparent UX that keeps humans informed and in control. For enterprises, this is what turns agentic systems from risky experiments into operational infrastructure that can credibly earn, and keep, their license to operate at scale.

So far, we have focused on governance inside the agentic architecture itself: how agents, orchestrators, and platforms embed controls and transparency. The next step is governance at the organizational and regulatory level: who is accountable, what must be logged, and how compliance frameworks like NIST’s AI RMF and the EU AI Act shape everyday practice.

Governance, Explainability, Logging, and Compliance

Governance is where abstract ideas of trust and ethics turn into day-to-day practice for agentic systems *at the enterprise level*. It defines who is accountable, what must be logged, how explanations are provided, and how evidence is produced for scrutiny—not just within a single agent or orchestrator, but across the organization and over time. In mature organizations, this means continuous, intent-aware logging of what agents planned and actually did, combined with explainability mechanisms calibrated to the risk and impact of each use case. Frameworks such as the National Institute of Standards and Technology (NIST) Artificial Intelligence Risk Management Framework and the EU AI Act focus on AI systems in general—and on “high-risk” AI in particular—rather than singling out agentic AI as a separate category, but their expectations map directly onto how agentic systems should be governed.

Concretely, these frameworks highlight several governance practices that are especially relevant for agentic AI:

NIST AI Risk Management Framework (AI RMF)

- Positions AI risk management around four functions—govern, map, measure, and manage—covering the full AI life cycle
- Stresses logging and traceability: keeping records of model versions, inputs, outputs, and operational context so organizations can reconstruct decisions and investigate incidents
- Emphasizes transparency and explainability: providing information about how systems work and how decisions are made, tailored to different stakeholder needs and risk levels
- Calls for governance and oversight: integrating AI risks into enterprise risk management, with defined controls, monitoring, and escalation paths

For agentic systems that plan, act, and chain tools autonomously, these expectations translate into detailed traces of agent plans, tool calls, and actions that can be reviewed and audited.

EU AI Act (high-risk AI systems)

- Does not name “agentic AI” explicitly, but classifies many agentic use cases as high-risk AI systems when they affect safety, fundamental rights, finance, or employment.
- Requires automatic logging and record-keeping over the system’s lifetime, so events relevant to risk, monitoring, and investigation are captured in a durable trail.
- Mandates traceability of operation: logs must make it possible to understand how a specific system behaved in

practice and how a given outcome was produced.

- Imposes transparency and human-oversight duties: systems must be designed so humans can understand capabilities and limitations and can supervise or intervene effectively.

For agentic AI, this effectively means maintaining replayable traces of agent workflows—user intents, plans, hand-offs, tool invocations, and external actions—so organizations can demonstrate control and compliance in sensitive domains.

This governance picture also exposes an important tension for agentic AI. On the one hand, methodologies mandated by NIST’s AI RMF and the EU AI Act implicitly assume that decisions can be reconstructed and made understandable through logging, traceability, and explainability. On the other hand, the generative models that power many agents are probabilistic and opaque, so we cannot literally replay or “open up” their internal reasoning step by step. In practice, organizations resolve this by moving transparency up a layer: instead of explaining every neuron, they log prompts, outputs, tool calls, and decision steps at the orchestrator and workflow level. This does not make the model itself fully transparent, but it does make the behavior of the overall agentic system traceable, auditable, and explainable in operational terms, in some way much like that of a human-based process, where humans’ behaviors in achieving an outcome can be traced and audited, but not the actual neuronal activity that’s taking place in these humans’ biological brains.

In addition to principles, effective governance for agentic AI depends on formal organizational structures that can make those principles real. Most leading organizations do not create a separate “agentic AI” governance stack; instead, they extend their existing AI and risk bodies so they can understand and supervise agents that plan, act, and chain tools autonomously. A Chief AI Officer (CAIO) typically owns the overall AI agenda—strategy, value, and governance—while a Chief Data Officer

(CDO) ensures that the underlying data is fit for purpose, well-governed, and compliant, and a Chief Risk Officer (CRO) anchors AI within the broader enterprise risk appetite and controls framework. Around them, cross-functional AI governance committees and ethics boards bring together legal, compliance, security, data science, and business leaders to approve high-risk use cases, define standards for logging and explainability, review incidents, and report to the board's risk or audit committee. Clear RACI-style definitions of who is responsible, accountable, consulted, and informed at each stage of the AI life cycle help ensure that agentic AI projects are subject to the same disciplined oversight as other critical systems, rather than operating as isolated experiments.

These organizational roles and committees become most tangible when you look at how specific companies embed governance directly into their AI products and workflows. Autodesk, a global leader in design and engineering software for architecture, engineering, construction, manufacturing, and media, offers a useful example of moving from compliance as an external “policeman” to compliance that is effectively built into how work gets done.

Vignette: Autodesk's Continuous Compliance Agents in Design Tools

Autodesk has been explicit about building “trusted AI” into its platform, anchoring it in principles such as responsibility, transparency, accountability, and reliability, and elevating trust to the C-suite by appointing a Chief Trust Officer to oversee these efforts across products and operations.

NOTE

This vignette is based on Autodesk's own public descriptions of its Trusted AI program and trust and compliance initiatives, including its Trusted AI principles, Trust Center materials, and announcements regarding the Chief Trust Officer role and responsible AI governance across products and operations. It synthesizes those communications into an illustrative example of continuous compliance agents inside design workflows, rather than reporting on a formal case study or peer-reviewed audit.

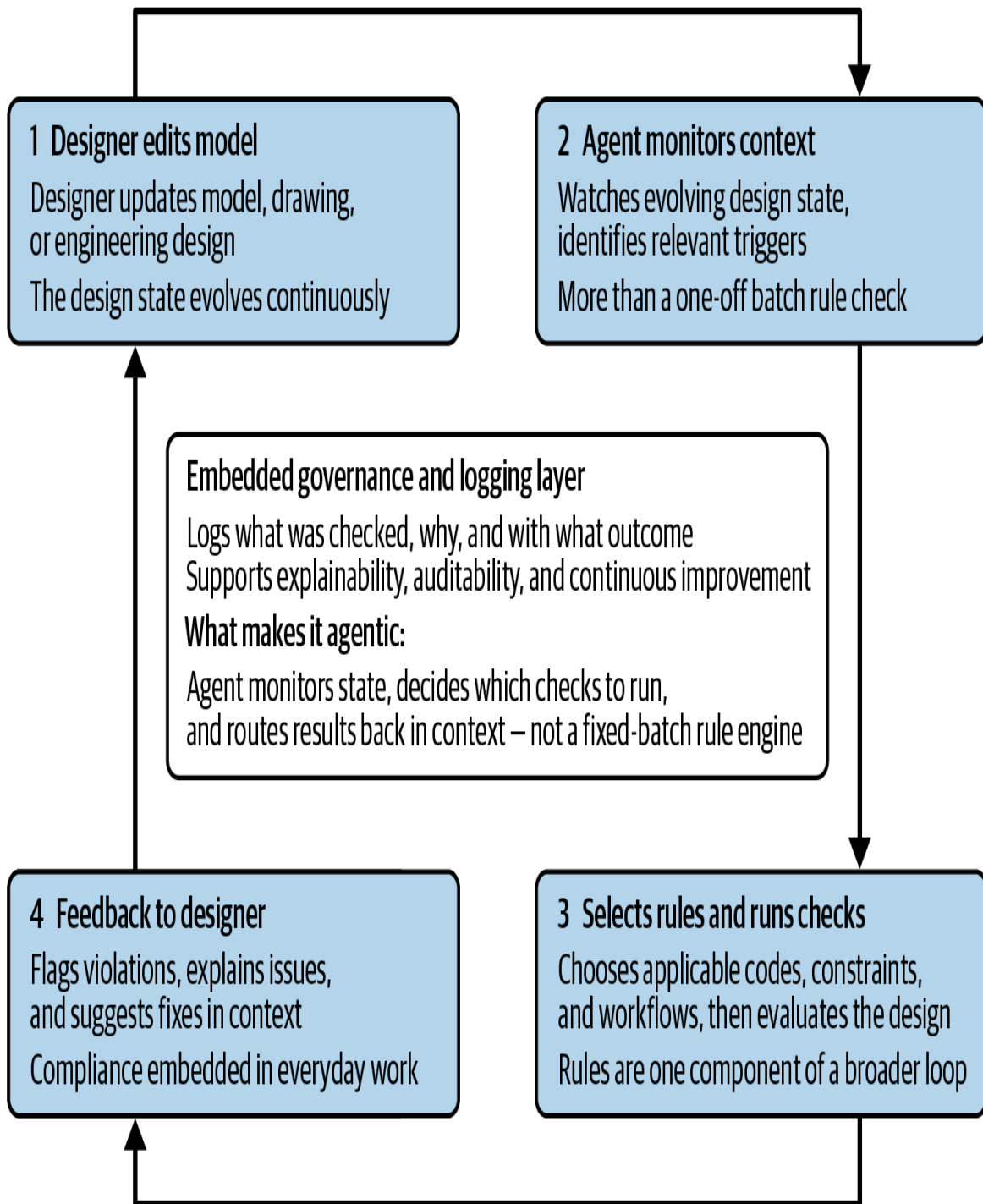
Instead of treating compliance as a separate, end-of-process check, Autodesk uses AI assistants that behave like continuous compliance agents inside everyday design workflows. These agents automatically evaluate models and drawings against complex engineering codes and design rules, flag potential violations in real time, and guide designers toward fixes—turning what used to be a manual, after-the-fact review into an embedded, proactive governance layer (see [Figure 7-1](#)).

Under the hood, these agents rely on structured rule sets and cloud workflows *as one component of a broader agent loop*: they monitor evolving designs, decide which checks to run when, and route results back to designers in context, while logging what was checked and why to provide an auditable trail of compliance decisions across teams. In practice, continuous compliance agents in design tools behave less like one-off validation scripts and more like embedded assistants that observe the evolving design state and trigger checks and suggestions as issues arise, rather than waiting for a manual end-of-process review. Autodesk's broader AI governance approach reinforces this: data is kept within its ecosystem where possible, AI behavior is constrained by policy-based controls, and users are given clear explanations of how specific AI features work, what data they use, and what risks they carry. The agentic aspect lies in how these systems plan and sequence checks over time, react to changes in the design, and coordinate rules, workflows, and user interactions to maintain compliance, rather than simply applying a fixed batch of rules at the end of the process.

In other words, Autodesk's continuous compliance agents show what it looks like when governance, explainability, and logging are embedded directly into everyday tools, rather than bolted on as an after-the-fact audit layer. Taken together, these governance practices complete the picture alongside the technical controls and human-in-the-loop workflows we discussed earlier; the next section turns to the cultural and adoption side of trust—how different groups within an organization experience and recalibrate trust in agentic systems over time.

Continuous compliance agents embedded in Autodesk design workflows

The agent loop: compliance is continuous, embedded, and auditable – not a one-off rules check



source: Autodesk public communications and Trust Center materials.

Governance and agentic loop framing are the authors' own interpretation.

Figure 7-1. Continuous compliance agents in Autodesk design workflows. A designer works in a familiar CAD environment, while an embedded agent monitors evolving designs, selects relevant engineering codes and rules, and evaluates the model in real time using cloud workflows and APIs. Instead of a single end-of-process batch check, compliance becomes a continuous loop: the agent surfaces violations and suggested fixes in context, logs what was checked and why, and feeds those traces into Autodesk's broader trusted AI governance program.

Culture, Adoption, Generational Shifts, and Agents Trusting Agents

People within the same organization do not learn to trust—or mistrust—AI in the same way, and this matters for how these systems are adopted. Large studies and reviews show that trust in AI varies systematically with familiarity, age, role, and context, rather than being a single, uniform attitude. Some work highlights the importance of factors like prior experience with automation, perceived reliability, and transparency, while global “trust in AI” surveys point to clear differences between more and less AI-experienced groups, and between younger and older respondents. Taken together, these findings suggest that “trust in AI” cannot be treated as one dial that leaders simply turn up or down; instead, trust must be calibrated differently for groups with different experiences, incentives, and mental models of what AI is good or bad at.

This matters for all AI, but it is especially important for *agentic* AI. When systems can plan, sequence actions, call tools, and affect real-world processes with limited oversight, misplaced trust—either naive overreliance or automatic rejection—has a much larger impact on safety, performance, and adoption. Different groups therefore need not only tailored communication about what agentic systems can and cannot do, but also interfaces, guardrails, and training that match their specific ways of building (or losing) trust in autonomous behavior.

What Major Studies Tell Us About Trust in AI

Several reputable organizations have documented how trust varies across people and contexts:

- A review of empirical studies on human trust in AI, published in the *Academy of Management Annals* under the title “Human Trust in Artificial Intelligence: Review of Empirical Research,” synthesizes dozens of experiments and field studies and finds that people’s trust in AI systems is shaped by prior experience with automation, perceived reliability, transparency, and how “human-like” the system appears, rather than being a single, uniform attitude. This work shows that users weigh these factors differently depending on their role and context, which leads to distinct trust patterns inside the same organization.⁵
- A global study on “Trust in Artificial Intelligence,” led by Melbourne Business School in collaboration with industry partners and reported by KPMG (“Trust, Attitudes and Use of Artificial Intelligence: A Global Study”), finds that familiarity with AI and age are strong predictors of attitudes: younger, more AI-experienced respondents are generally more willing to use AI for factual and logistical tasks, while being more skeptical in high-stakes areas like health care, justice, or autonomous driving. Older or less AI-familiar respondents tend to report higher overall concern and lower baseline trust.⁶
- Research on “Calibrating Workers’ Trust in Intelligent Automated Systems,” published in a National Institute of Health-indexed journal and available via PubMed Central, shows that in workplace settings people differ in how much weight they give to AI recommendations; trust is adjusted over time based on observed accuracy, error handling, and the clarity of feedback, and poorly calibrated trust (too much or too little) directly harms performance.⁷

Together, these findings support a simple but important point: trust in AI naturally varies across roles, generations, and familiarity levels. This segmentation exists for all AI, but for agentic systems—which can chain actions and touch more of the real world—the cost of ignoring it is higher,

because a small group over- or under-trusting an agent can have outsized operational impact.

Practical Segments and How to Address Them

Building on these patterns, it is useful in practice to think of at least four broad groups inside organizations, each with a different starting point for trusting agentic AI and each needing a different approach to communication and UX. These segments are a pragmatic simplification: in reality, people's trust also moves up or down with the complexity and stakes of the specific use case. This is a pragmatic synthesis of recurring themes in the literature, not a rigid taxonomy:

Enthusiastic early adopters (often younger or tech-centric roles)

These individuals already use AI tools heavily and are quick to rely on them for routine, factual, or productivity tasks. They tend to have high familiarity and relatively high baseline trust, but may underestimate risks in edge cases or when moving into high-stakes domains. For agentic AI, they need clear information about limitations, explicit uncertainty signals, and visible guardrails so they do not push agents beyond their safe operating envelope. Feedback loops that clearly flag errors, corrections, and interventions help them calibrate their trust instead of swinging from blind optimism to sudden skepticism.

Skeptical experts in high-stakes domains (e.g., clinicians, risk, legal)

These users are normally open to AI as decision-support but reluctant to delegate final high-stakes judgments to opaque systems. They worry about accountability, rare edge cases, and fairness, and are often professionally socialized to be conservative about changes that could harm patients,

clients, or the firm—whether or not formal regulations are involved. For this group, agentic AI should be framed explicitly as a copilot or second reader: show case-based explanations, provide uncertainty estimates, and make it easy to escalate, override, or request human review. Designs that combine explainability (why this suggestion?) with calibrated confidence (how sure is the system?) align well with how they naturally decide when to lean on AI versus when to double-check themselves.

Cautious pragmatists (mid-career managers, operations leaders)

Many managers and operations leaders fall into a middle category: they are neither early enthusiasts nor outright opponents, but care most about operational risk, reliability, and reputation. They want to see that agentic AI actually works in their context and can be brought under control if something goes wrong. They respond best to concrete business cases, visible controls (approvals, audit trails, incident playbooks), and simple performance dashboards that show error rates, escalations, and improvements over time. For them, trust grows gradually through lived experience with well-governed pilots, rather than through abstract promises.

Low-familiarity or change-fatigued staff

This group has low exposure to AI, high concern about job impact, and often struggles to see how AI connects to their daily tasks. They may be overwhelmed by jargon or see AI as “yet another change experiment” imposed from above. For agentic systems, successful adoption with this group starts with basic education in plain language, hands-on exposure to safe, low-risk use cases, reassurance about roles and accountability, and very simple, role-appropriate interfaces where AI support feels like a natural extension of existing

tools. Early experiences are crucial: if first interactions are confusing, punitive, or poorly explained, these users can quickly become durable skeptics.

By acknowledging these segments (and coming up with your own organization's version of this segmentation) and designing communication, training, and UX patterns for each, your organization can help people move toward calibrated trust—neither naive over-reliance nor blanket refusal—at exactly the moment when agentic AI makes the trust question more consequential than ever.

These findings also underline that trust in agentic AI is not just a human-behavior problem; it is a design problem. If different groups of people develop trust based on experience, transparency, and context, the systems themselves can be built to do something similar. In mature agentic architectures, meta-agents and orchestrators effectively “trust” other agents by evaluating their performance history, policy conformance, and reliability in specific scenarios before delegating work. Over time, this creates machine-level trust relationships—encoded in routing rules, allow/deny lists, and service-level guarantees—that mirror human governance: only agents that have earned trust on defined dimensions are allowed to act autonomously, while others remain in a supervised or “copilot” role.

These ideas are not just theoretical. A number of corporate leaders are already designing agentic AI so that trust is layered and earned, both for humans and for agents. Siemens, which we cited earlier in this book as a leading AI practitioner in the industrial world, offers a clear illustration in how it is introducing AI agents into manufacturing and automation.

Vignette: Siemens's Orchestrated Industrial Agents as Layered Trust

In its public communications and product demos, Siemens describes a new generation of industrial AI agents integrated with its Industrial Copilot ecosystem, with the aim of automating parts of the industrial value chain

from engineering to operation and maintenance. Rather than relying on a single monolithic assistant, Siemens describes an architecture in which orchestrators connect multiple copilots and specialized agents. Examples shown in Siemens' own demonstrations include PLC code from natural language inside the TIA (Totally Integrated Automation) Portal, help engineers create HMI (human-machine interface) visualizations, or support operators by explaining machine error codes and suggesting remedies based on documentation and historical data. In this setup, the orchestrator interprets user intent, selects and sequences the relevant agents for each request, and coordinates their interactions with underlying machines and automation systems. Users and operators remain in charge by deciding which tasks to hand to the copilot and by reviewing proposed changes, while orchestrators and platform policies limit autonomous actions to agents that meet defined reliability, safety, and security criteria and ensure that actions are logged for later audit.

Taken together, this orchestrated multi-agent setup creates a layered trust structure that echoes the themes of this chapter. From a trust-architecture perspective, operators delegate specific tasks to an AI copilot that explains what it is doing, translates complex machine data into understandable recommendations, and keeps them in the loop for decisions, helping calibrate trust rather than demanding blind reliance. On the machine side, the orchestrator encodes “agents trusting agents”: it tracks how specialized agents perform, enforces interoperability and security boundaries, and routes work only to those agents that conform to defined performance and policy expectations—effectively implementing trust contracts in code.⁸

Bringing It All Together

To conclude our Building Trustworthy Systems chapter: agentic AI systems do not just generate outputs; they plan, act, and chain tools in the real world, which makes trustworthiness a design requirement rather than an afterthought. At the same time, building trustworthy systems is not just about the systems themselves: trust emerges from how organizations

design, govern, and use these agents in practice, and how different people experience them day to day.

To manage this, we argued for three inseparable dimensions of trustworthy agents: robust technical controls that bind what agents can do and make their behavior observable, governance structures that encode accountability and logging into both the architecture and the organization, and human-centered workflows and interfaces that keep people in charge of consequential decisions. In practice, this becomes a set of pillars: watcher agents and guardrails, human-in-the-loop patterns and confidence-aware routing, and governance mechanisms from platform policies and orchestrators as control planes to enterprise-level frameworks such as NIST’s AI RMF and the EU AI Act.

The vignettes throughout the chapter—ranging from Deviniti’s self-hosted agents and GitHub Copilot’s “copilot, not autopilot” pattern to Autodesk’s continuous compliance assistants and Siemens’ orchestrated industrial agents—show what it looks like when these principles are applied in production: agents earn autonomy incrementally, operate inside well-defined scopes, and leave auditable traces that support both oversight and learning. Finally, we saw that trust in agentic AI is also a cultural and generational phenomenon: different groups of users start from different baselines, and designing for calibrated trust means meeting them where they are, not expecting a single, uniform response.

The core message is that agentic AI only scales when bounded autonomy, explicit accountability, and calibrated human trust move together. The organizations that succeed will be those that treat agents as governed infrastructure, not side projects—building layered technical, governance, and human safeguards so that autonomy is earned, monitored, and continually renegotiated as these systems and their human counterparts continue to evolve.

¹ For accounts of the October 2, 2023, Cruise robotaxi incident in San Francisco, see NPR, “California Orders Cruise Driverless Cars off the Roads Because of Safety Concerns” (October

23, 2023); California DMV, “DMV Statement on Cruise LLC Suspension” (October 24, 2023); Reuters, “How GM’s Cruise Robotaxi Tech Failures Led It to Drag Pedestrian 20 Feet” (January 26, 2024); and Phil Koopman, “Lessons from the Cruise Pedestrian Dragging Mishap” (2024).

- ² Zoox, *Safety Report—Volume 3: Operational Safety* (2024), available via the company’s Safety page, which describes the PRG process for handling operational safety events and adjusting operating domains, supervision, and fleet status over time.
- ³ Based on reporting in TechCrunch, “A Meta AI Security Researcher Said an OpenClaw Agent Ran Amok on Her Inbox” (February 22, 2026), which recounts Summer Yue’s description of connecting an OpenClaw agent to her Gmail with instructions to seek approval before deleting, the agent’s subsequent deletion of hundreds of emails, and her need to return to her machine to stop it.
- ⁴ See, for example, Skadden, “**AI Agents: Greater Capabilities and Enhanced Risks**” (April 22, 2025), which urges companies to set policies for AI agent development and deployment, allocate clear accountability, and ensure ongoing monitoring and human override mechanisms. See also DLA Piper, “**The Rise of ‘Agentic’ AI: Potential New Legal and Organizational Risks**” (May 21, 2025), which highlights the need for contractual allocation of responsibility, scoped agent capabilities, and proportionate governance and oversight.
- ⁵ E. Glikson and A. W. Woolley, “**Human Trust in Artificial Intelligence: Review of Empirical Research**”, *Academy of Management Annals* 14, no. 2 (2020). This review synthesizes empirical studies on determinants of human trust in AI and highlights the roles of transparency, reliability, and anthropomorphism.
- ⁶ University of Melbourne and KPMG, “**Trust, Attitudes and Use of Artificial Intelligence: A Global Study 2025**”, based on a survey of more than 48,000 people across 47 countries, analyzing how familiarity with AI, age, and other factors shape trust, attitudes, and use.
- ⁷ Gale M. Lucas, Burcin Becerik-Gerber, and Shawn C. Roll, “**Calibrating Workers’ Trust in Intelligent Automated Systems**”, 2024 (open-access via PubMed Central), which reviews and extends the concept of calibrated trust in workplace automation and intelligent systems, emphasizing how miscalibrated over- or under-trust harms performance.
- ⁸ Source: Siemens communications and press releases; architectural interpretation and trust framing are ours.

Chapter 8. Scaling Without Lock-In

Chapter 7 showed that earning a license to operate is not a one-time event. Trust in agentic AI is layered, earned incrementally, and continuously maintained through governance structures, accountability frameworks, human oversight, and the kind of technical guardrails that keep autonomous systems within bounds. Whether you were thinking about calibrating trust across different groups inside your organization, or about the machine-level trust contracts that allow orchestrators to delegate work only to agents that meet defined reliability and policy criteria, the central message was the same: trustworthy agentic AI does not happen by accident. It is designed, enforced, and governed.

But earning that trust inside a pilot or a controlled deployment is only the first challenge. The harder question—and the one this chapter addresses—is what happens when you decide to scale. The moment an organization moves beyond isolated experiments, a new set of risks emerges that has little to do with whether agents behave reliably and ethically, and everything to do with how the underlying architecture is built. Agents proliferate across teams. Each department builds on a different stack. Dependencies quietly accumulate on a single model provider, a single cloud vendor, or a platform whose APIs may look very different a year from now.

Chapter 8 examines how to scale agentic AI without falling into these traps. It begins with the coordination and encapsulation principles that prevent agent sprawl from turning your enterprise into a tangle of conflicting, duplicated bots. It then turns to the architectural choices that preserve your freedom to change model providers and cloud infrastructure as capabilities and pricing evolve. From there, it looks at how to future-proof your stack so that upgrades become routine rather than disruptive, how to balance central governance with the empowerment of business units, and, finally, how

emerging standards such as the Model Context Protocol are beginning to define a more open, interoperable foundation for agent ecosystems.

The five themes of this chapter form a single architectural argument, each building on the last. Encapsulation and coordination ([Section 1](#)) establishes the foundational discipline that prevents agent sprawl—the governance layer that every subsequent decision depends on. Vendor neutrality ([Section 2](#)) builds on that foundation by ensuring your coordinated architecture doesn't accidentally harden into dependency on a single provider. Future-proofing ([Section 3](#)) extends both: once you have governed services and portable infrastructure, you need the versioning and rollback practices that make evolution safe. The operating model ([Section 4](#)) answers who owns each layer, because even the best architecture fails without clear ownership. And open standards ([Section 5](#)) are what allow the entire stack to grow without collapsing under its own complexity.

Taken together, they are the engineering and organizational answer to a question that [Chapter 7](#) raised but could not yet answer: how do you build something trustworthy enough to run at enterprise scale, without locking yourself into an architecture you cannot change?

1. Encapsulation and Coordination

In large organizations, agents can easily proliferate as one-off experiments built by different teams on different stacks, driven by factors such as revenue growth expectations, the accessibility of low-code development tools, and department-level innovation pressures. For example, in a bank, the sales, finance, and operations departments might each build agents for credit risk assessment, but their differing goals—sales wanting to approve more loans, finance focusing on conservative loss provisions, and operations prioritizing processing speed—could produce slightly different risk classifications for the same applicant. Without coordination, you may end up with duplicated effort, conflicting behaviors, and agents directly poking at critical systems in inconsistent ways. Enterprise coordination and encapsulation is about treating agentic capabilities as shared, well-governed

services rather than ad hoc scripts, so that business logic, data access, and risk controls are centralized where it matters (and not centralized where it doesn't).

To anticipate and remedy this, it helps to break enterprise coordination and encapsulation into a small number of distinct design choices that will feel familiar to anyone who has worked through a microservices or service-oriented architecture (SOA) transformation: how to standardize interfaces and encapsulate business capabilities as reusable services; how to structure the shared platforms and controls that every agent must plug into; and how to allocate ownership and guardrails between central teams and empowered business units. The underlying principles—stable APIs, centralized governance, and federated delivery—are well-understood patterns that enterprise architects have been applying for decades; what changes with agentic AI is the stakes, because agents do not just consume these services passively, they act through them. This coordination layer is not free, however. Routing agent calls through a shared gateway or orchestration platform adds milliseconds of latency to every interaction, and depending on how the layer is implemented, it may introduce additional token consumption or API costs as requests are normalized, logged, and policy-checked before reaching their destination. For most enterprise workflows, these overheads are modest and well worth the governance benefits. But for latency-sensitive or very high-frequency use cases—e.g., real-time fraud scoring or high-throughput trading signals—they are a genuine design constraint that must be measured and factored into the architecture from the outset, not discovered after deployment:

Standardize interfaces and encapsulate capabilities

Organizations piloting agentic AI at scale are moving away from one-off agents that talk directly to SaaS apps or databases, and instead catalog core business actions as reusable services or “agent actions” behind stable APIs. For example, banks piloting agentic AI define an internal catalog of agent actions (such as “create ticket,” “update address,” “summarize contract”) so that any agent calls those

governed services rather than implementing its own brittle integrations to core systems. It is worth noting that financial institutions, insurers, and large enterprises more broadly have been building standardized service catalogs for decades, with mixed results. The discipline is genuinely hard: catalogs tend to become shelfware when governance is weak, when business units find work-arounds faster than central teams can publish standards, or when the catalog fails to keep pace with changing business needs. What makes the agentic context different—and potentially more tractable—is that agents are software that can be forced to call governed services by design, rather than humans who can choose to ignore them. The enforcement mechanism is architectural, not behavioral, which removes the single biggest reason past catalogs failed.

There is a further practical wrinkle worth addressing here. Most of the architectural prescriptions in this chapter are written as if organizations are building from a blank slate—choosing orchestration frameworks, designing catalogs, and wiring tools from scratch. In practice, most enterprises are not. They have existing CRM, ERP, and data warehouse integrations; agents already in production that were built before governance structures existed; and middleware layers accumulated over years of SOA and microservices initiatives. The migration path into a well-governed agentic architecture is therefore incremental by necessity, not by choice.

The approach that has worked in practice follows a consistent sequence: first, audit which agents have already moved from experiment to core workflow and therefore carry real operational risk; second, introduce the neutral gateway and catalog layer in front of existing agents without touching any business logic (this gives you observability and

policy enforcement immediately, even over legacy integrations); third, migrate tools, prompts, and data access patterns to governed, portable formats one agent at a time, starting with the highest-risk workflows; and fourth, retire or refactor brittle point-to-point integrations as new governed services become available.

This staged approach means the encapsulation and coordination principles described in this section can be adopted progressively, without requiring a greenfield rebuild. The architecture does not need to be perfect on day one—it needs to be directionally correct and incrementally improvable, which is precisely the kind of resilience that makes it trustworthy at scale. S&P Global’s collaboration with IBM uses watsonx Orchestrate’s agent catalog in a similar way, registering S&P’s internal agents as composable capabilities within a shared orchestration layer.

Shared platforms and controls every agent must plug into

Consulting playbooks and early movers emphasize a common platform for discovery, governance, and observability of agents and tools. McKinsey describes an internal “agent and workflow discovery” layer and an “AI asset registry” that centralize prompts, tool definitions, and model configurations, with policies and auditability applied consistently across them. Enterprise architecture guides from major vendors similarly describe centralized agent catalogs and tool registries acting as mandatory gateways, so no agent runs in production without being registered, monitored, and bound by standard security and compliance controls.

Ownership and guardrails between central teams and business units

AI operating-model work increasingly points to a hybrid structure in which a central platform team defines standards, risk frameworks, and shared infrastructure, while business units build and run use case-specific agents within those guardrails. In this “central platform, federated builders” setup, central teams own the AI platform, governance, and reusable components, and business units own domain-specific workflows and ROI, an arrangement now widely recommended as the default for organizations scaling beyond pilots.

Taken together, these three moves—standardized agent actions, shared platforms, and a hybrid operating model—are how leading enterprises are starting to prevent “agent sprawl” in practice. Instead of dozens of siloed bots each hardwired to its own system, leading enterprises are building shared foundations: a governed catalog of reusable agent actions that any team can call, a common orchestration layer that enforces policy and logs every step, and a clear split between the central platform team that owns the infrastructure and the business units that own specific workflows and outcomes. In practice at S&P Global, for example, this means an analyst-facing agent does not reach directly into proprietary data systems; it calls a registered, governed service in IBM’s watsonx Orchestrate agent catalog, which handles authentication, compliance logging, and output formatting—so the business unit gets speed, and the platform team retains control. That foundation—standardized services, shared platforms, and hybrid ownership—is the prerequisite for everything that follows. But coordination alone is not enough. Even a perfectly governed agent catalog becomes a trap if every service in it depends on a single model provider or cloud vendor. Section 2 turns to that problem: how to preserve architectural freedom once the coordination layer is in place.

2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

In many enterprises, early agents are wired directly to a single LLM API or cloud service, which is fine for pilots but dangerous once those agents sit in core workflows. In most cases, these tightly coupled agents are best understood as *technical debt*—a term borrowed from software engineering that describes the hidden cost of building something quickly rather than durably. Like financial debt, it does not disappear; it compounds. The agent works today, but every time the business needs to change providers, add governance, or scale the workflow, the bill comes due in the form of expensive rewrites and fragile work-arounds. Migration is possible but rarely clean, and some early agents will not survive it. Organizations that have navigated this successfully tend to start by auditing which agents have moved from experiment to core workflow and therefore carry real business risk; then introduce a neutral gateway layer in front of existing agents before touching any business logic; and migrate data, tools, and prompts to formats they own, one agent at a time, starting with the highest-risk workflows. If every prompt, tool call, and safety check depends on one provider's quirks, switching or even adding a second model becomes a major re-platforming project. Avoiding vendor lock-in here means deliberately inserting a neutral layer between your agents and the underlying models and clouds, so you can change suppliers without rewriting the entire business logic.

A practical way to do this is to think in terms of three levers: first, a neutral access layer for models; second, keeping your data and tools portable; and third, using multiprovider setups where they add resilience and leverage. None of these comes free—building and maintaining a neutral gateway, keeping integrations portable, and managing multiple providers all add development cost, slow initial delivery, and create ongoing support overhead. But for any agent that has moved from pilot to core workflow, these are one-time architectural investments that buy lasting freedom; the alternative is a growing dependency that becomes progressively more expensive to unwind:

Neutral model access layer

Organizations like Grab (the Southeast Asian ride-hailing/food delivery super-app) have built an internal “GenAI gateway” that exposes a single API to internal teams while routing traffic to multiple providers (OpenAI, Azure, AWS Bedrock, Google Vertex, and in-house models) behind the scenes. That gateway normalizes how prompts, safety controls, authentication, and observability work, so that applications don’t depend on one vendor’s software development kit. It is worth noting, however, that API-level portability is only part of the solution. A subtler form of lock-in comes from prompts that have been engineered specifically around one model’s quirks—its formatting preferences, instruction-following style, or context window behavior. Switching models while keeping those prompts intact can silently degrade performance.

The more robust approach, now emerging in practice, is to store model-agnostic semantic prompt templates in the gateway itself, which are then dynamically adapted for whichever model is receiving the request. This separates the intent of the prompt from its model-specific expression, and is the direction that leading gateway implementations are beginning to standardize on.¹ Large platform providers report enterprise customers using similar gateways to switch between models while maintaining governance and monitoring end to end. This is the concrete incarnation of a model adapter layer that your agents talk to instead of directly calling each model API.

Portable data and tools

Future-proofing also means that corporate knowledge and capabilities live in forms you control, not buried inside one platform’s proprietary features. Reference architectures for “multiprovider generative AI gateways,” for example,

assume your retrieval, indexing, and business tools are exposed via standard services in your own cloud accounts, while models are swapped underneath. Backbase, a leading digital-banking platform used by over 80 banks worldwide, describes—in commercial documentation rather than offering open source code—an “AI-native Banking OS” in which agents interact with core systems through a governed Integration Fabric and Control Plane, allowing the underlying LLM to be swapped without breaking the agent’s ability to act. The pattern is open formats, HTTP/JSON tools, and vector or data platforms that can travel with you if you change model providers.

Pragmatic multiprovider usage

Once you have neutral access and portable tools, you can be selective about where multiple providers actually matter. Grab, for instance, uses different models for different internal use cases (from content moderation to real-time audio analysis), and routes requests based on performance and cost while keeping one gateway surface for developers. Enterprise customers described by IBM do a similar routing—sending some workloads to one model, others to another—through a unified gateway that also embeds observability and policy. The goal is not symmetrical use of all providers, but the freedom to combine them, pilot new ones, or exit one if pricing, capabilities, or regulation changes.

It is worth being explicit about why cost flexibility matters at scale, not just in principle. Token costs, inference costs, and orchestration overhead do not grow linearly as agent networks expand—they compound. A single agent calling a frontier model for a complex reasoning step may cost a few cents; an enterprise workflow in which that agent triggers three downstream specialist agents, each making multiple

tool calls and retrieval steps, can cost orders of magnitude more per transaction. Orchestration layers add their own overhead: every policy check, logging step, and prompt normalization that passes through the gateway consumes tokens and compute. Organizations that have mapped this in production—Grab’s internal gateway telemetry being one example—find that cost management is inseparable from architectural design.

The practical implication is threefold: instrument every layer from day one so costs are visible at the agent and workflow level, not just in aggregate cloud bills; design workflows to minimize unnecessary model calls by caching, batching, or routing simpler subtasks to smaller, cheaper models; and treat cost ceilings as a first-class parameter in the mesh contract—the shared agreement that defines how agents, tools, and services interact, along with the operating limits they must respect, such as compute budgets, cost caps, and resource quota—so that orchestration layers can enforce spend limits per workflow or business unit, not just per API call. Cost scaling is therefore not a finance problem appended to the architecture—it is an architectural constraint that must be designed in from the start.

At the end of the day, avoiding lock-in is less about never touching proprietary features and more about where you put your dependencies. By centralizing model access in a gateway, keeping data and tools in your own governed platforms, and using multiprovider routing where it truly helps, you secure some negotiating power and technical flexibility without sacrificing speed and performance.

With this in mind, one can understand the rapid emergence of a new layer of infrastructure sitting between model APIs and business applications. The market here is crowded and competitive: virtually every major SaaS and cloud vendor is positioning itself as the default agentic orchestration layer,

alongside a generation of specialized independents. It helps to think of these players in three broad categories. First, open source orchestration frameworks such as LangChain and LlamaIndex provide the building blocks for chaining agents, tools, and memory into multistep workflows. Second, managed orchestration and prompt operations platforms such as Vellum AI add enterprise-grade versioning, evaluation, and deployment control on top of those frameworks. Third, agentic service mesh and inference optimization players—including both established cloud providers extending their API gateway products and a new wave of purpose-built startups—focus on routing, cost control, latency optimization, and cross-model observability at scale. These orchestration layers sit above model APIs and clouds, and below applications, acting as routing layer for models, tools, and cloud infrastructure, execution engines for multi-agent workflows, and governance, observability, cost and reliability-focused service-level agreement control tools. What they share is a common goal: to be the layer your agents cannot do without, which is precisely why the choice of orchestration infrastructure deserves the same scrutiny as the choice of model provider.

Avoiding vendor lock-in preserves your freedom to change providers, but that freedom is only valuable if you can actually execute the change safely. A neutral gateway and portable tools give you optionality in theory; versioning, experimentation, and rollback practices make that optionality real. Section 3 shows how to structure the stack so that upgrades—whether switching models, adding providers, or evolving workflows—become routine operations rather than high-risk projects.

3. Future-Proofing Architectures for Upgrades

The LLMs, vector stores, and orchestration tools you use today will evolve quickly, and leaders are already treating upgrades as a continuous process, not a rare event. If your agents are tightly bound to one model's prompt style or a specific platform's APIs, every upgrade becomes fragile and

risky. In that context, “future-proofing” means structuring your stack so that models, prompts, and workflows can be iterated and replaced or updated in a controlled, incremental manner.

To achieve this, it is helpful to think in terms of four complementary lenses: architectural separation, versioning and compatibility, built-in experimentation, and safe rollout/rollback practices. As with the coordination and encapsulation principles discussed earlier, these are not new disciplines invented for AI—they are the same practices that engineering teams have applied for years to manage microservices, API upgrades, and continuous delivery pipelines. What changes with agentic AI is not the toolbox, but the stakes: because agents act autonomously on live systems, the cost of a poorly managed upgrade is higher than in conventional software, making these familiar practices not just good engineering hygiene but a genuine risk management necessity:

Architectural separation of concerns

Organize the stack into distinct layers (application/workflow, reasoning, infrastructure). Salesforce’s “Agentic Enterprise” reference architecture explicitly separates an orchestration layer, individual agents, and an API-integration fabric that connects to enterprise systems. This layering lets you change models or orchestration frameworks without redoing every integration. Similarly, Databricks’ enterprise LLM deployments show that decomposing complex prompts into multistep agentic workflows that sit above the underlying model and data infrastructure lets the same workflows be reused as models change.

Versioning and compatibility practices

Treat agents, prompts, and model configurations as versioned assets, not anonymous scripts. Production users such as FactSet, via Databricks’ case studies, keep explicit versions of prompts and workflows tied to specific model versions, along with evaluation suites to track accuracy and

latency changes as they upgrade. Microsoft's agentic-AI maturity guidance likewise advocates treating agents, prompts, and integrations (i.e., the controlled links or gateways that let your AI agents safely reach and use the information and functions in your current IT systems) as versioned assets with application life cycle management and telemetry. Telemetry here refers to the automated, continuous collection and transmission of observable data, such as request latency, error rates, timestamped records of agent actions or tool calls, and traces—the end-to-end path of a request as it moves across multiple agents or services from the distributed components of an agentic system to a central monitoring or observability platform. This enables side-by-side runs of old and new configurations, objective comparisons, and evidence-based promotion.

Built-in experimentation and rollout

Deploy AI components via feature flags and gradual rollouts, not all-or-nothing releases. For example, Microsoft recommends using Copilot Studio with standard application life cycle management and telemetry to test new agents or patterns on a limited scope before broad rollout, while preserving the ability to roll back quickly if issues appear. Likewise, Databricks stresses the importance of controlled experiments when introducing new models into high-risk enterprise workflows, making upgrades feel more like regular DevOps than one-off “big bang” projects.

Rollback and observability safeguards

Even with careful experimentation, you will need clear observability and automated rollback mechanisms. Centralized monitoring, logging, and alerting let you detect regressions early and trigger a predefined rollback to the

last-known good version. This closes the loop so that upgrades remain reversible and risk-bounded.

By separating concerns, rigorously versioning, normalizing experimentation, and instituting reliable rollback/observability, leaders turn model and platform upgrades into a routine, low-risk capability instead of an existential threat. But architectural discipline is not self-enforcing. Someone has to own the gateway, maintain the versioned catalog, run the experiments, and decide when to roll back. The next section addresses the organizational question that the first three sections presuppose: who is responsible for what, and how do you balance central control with the speed and domain expertise that only federated teams can provide?

4. Centralized Ownership Versus Empowerment

How you organize people around agents matters as much as the architecture. If everything is run by a single, central team, you obtain consistency but limited domain depth and a backlog of unmet needs; if every business unit does its own thing, you get innovation but also duplicated effort and unmanaged risk. What leading practitioners are converging on is, in essence, the application of platform engineering to agentic AI—an emerging discipline, now formalized by Gartner and bodies such as the Cloud Native Computing Foundation (CNCF), in which a central platform team builds and maintains a “paved road” of shared infrastructure, standards, and guardrails that empowers other teams to self-serve safely and at speed. The parallel is clear: just as platform engineering teams provide developers with golden paths, approved toolchains, and governed pipelines so they can ship software without reinventing infrastructure, an AI platform team provides agent builders with approved models, evaluation frameworks, governance controls, and reusable components so they can deploy agents without creating unmanaged risk. This is why leading practitioners are converging on hybrid operating

models where a central platform team provides guardrails and shared capabilities, while federated teams in the business own specific agents and outcomes.

This balance can be captured in two complementary dimensions: ownership split and responsibility/empowerment patterns:

Ownership split—central platform versus federated builders

McKinsey’s work on “agentic organizations” and on COO’s use of GenAI stresses the need for a central “spine” of capabilities—platforms, policies, and standards—combined with domain teams that own use case delivery. A concrete analogy comes from ANZ Bank’s Institutional Division, which—as described in a detailed practitioner case study coauthored by ANZ’s own platform architects and published on the AWS Big Data Blog—built a federated data platform in which a central data and AI platform team supplies domain-agnostic tooling and infrastructure while domain teams build and own their specific data products on top.² Applied to agents, this means central teams own the agent platform, governance, evaluation frameworks, and any cross-cutting agents such as enterprise search, while business units own domain-specific agents, their prompts/workflows, and the KPIs those agents are meant to improve.

Responsibility, life cycle management, and empowerment within guardrails

Microsoft’s agentic-AI maturity guidance explicitly calls for centralized monitoring, telemetry, and agent life cycle management (ALM) even as responsibility for designing and operating individual agents spreads into the business. In practice, central teams are accountable for platform reliability, security posture, and policy enforcement; domain teams are accountable for the success and safety of their particular agent workflows. Legal, risk, and compliance functions are consulted for high-risk scenarios, and

support/operations are informed and equipped to handle incidents when agents “misbehave.” Empowerment comes from providing a “paved road”: approved platforms such as Copilot Studio or Claude Cowork, standard templates, software development kits (SDKs), and evaluation harnesses that let noncentral teams build agents that automatically inherit monitoring, security, and data-access patterns. Capability tiers (low-risk, read-only agents that teams can ship with minimal review versus high-risk agents requiring central sign-off) let you match scrutiny to risk while keeping the barrier low for safe experimentation.

Across banks, hyperscalers, and advisory engagements, a consistent pattern is emerging: centralize the platform and governance, while federating the design and ownership of specific agents. This approach supports scale without sacrificing control or agility. But scale also means growth—not just more agents doing the same things, but new capabilities, new tools, and agents collaborating in ways not anticipated at the outset. Section 5 turns to expandability: how to grow the agent ecosystem over time without tearing up the foundations laid in the previous four sections.

5. Expandability, Model Context Protocol, and Agent-to-Agent

Once the basics are in place, the question becomes how to grow an ecosystem of agents and tools over time. Expandability is about making it easy to add new skills, domains, and collaborating agents without tearing up the foundations. A key enabler of this is the Model Context Protocol (MCP), an open standard that defines how agents, tools, and models exchange context—such as prompts, tool descriptions, and conversation history—so that capabilities can be reused across different platforms and vendors. The USB-C analogy is instructive here, and not coincidental: Anthropic itself coined it when **launching MCP** in November 2024, and the

parallel has held up. Just as USB-C ended the proliferation of incompatible device connectors by establishing a single universal standard, MCP is rapidly becoming the universal connector for AI agents and tools. Within twelve months of its launch, OpenAI, Google, and Microsoft had all adopted it. Anthropic **donated** it to the Linux Foundation in December 2025, cofounding the Agentic AI Foundation with Block and OpenAI, and with support from Google, Microsoft, AWS, Cloudflare, and Bloomberg; and by early 2026 it counted over 10,000 active public servers and 97 million monthly SDK downloads. The direction of travel is clear: MCP is becoming the infrastructure layer that agents will be expected to speak, much as HTTP became the layer that web applications were expected to speak.³

With MCP and similar mechanisms in mind, we can look at three layers: designing for tool and skill expansion, using standard protocols to decouple agents from backends, and adopting agent-to-agent (A2A) collaboration patterns that remain governable:

Designing for tool and skill expansion

Build the agent stack around a clear notion of “tools” or “skills” that can be registered, discovered, and updated without touching agent core logic. Salesforce’s Agentic Enterprise reference architecture describes an “API integration fabric” under the agent layer, where enterprise systems are exposed as well-governed APIs that agents can call. IBM’s watsonx Orchestrate goes further by providing an Agent Catalog with hundreds of prebuilt agents and tools that can be composed into workflows, showing how a catalog-based approach accelerates new use cases. These examples embody the idea that tools and skills should be replaceable components, so agents evolve by wiring together capabilities from a catalog rather than by embedding bespoke logic.

Using standard protocols such as MCP

Emerging protocols for connecting agents to tools and data sources aim to standardize how capabilities are exposed and consumed. A wave of multi-agent orchestration platforms and *agent meshes* (a conceptual framework or architectural pattern where multiple AI agents can discover, connect, and collaborate with each other through standardized interfaces and protocols) emphasize protocol-based connections between agents, tools, and data, so capabilities can be reused across models and platforms. While specific protocols differ, the common goal mirrors the architectures used by Salesforce and IBM: a consistent way for agents to talk to tools and other services, independently of where they're hosted. This reduces the cost of adding or swapping tools because the mesh contract remains stable even when the underlying implementation moves between vendors or runtimes.

A2A collaboration patterns

Beyond “agent plus tools,” more complex scenarios involve multiple agents working together—coordinator agents delegating subtasks to specialists, or domain agents, such as HR, finance, or IT-specific agents, collaborating on a multistep workflow. Banking examples and analyst guidance now explicitly recommend multi-agent systems—coordinator agents delegating tasks to specialists, with validation agents providing oversight—as a safer alternative to monolithic “do everything” agents. Industry commentary highlights overseer/validator agents as a pattern for adding extra governance layers without blocking automation. The principle is that complex workflows are split across agents with clear roles and trust boundaries, and interactions are monitored and auditable so emergent behavior doesn't turn into untraceable chaos.

NOTE

A word of caution is warranted. The protocol landscape is still evolving, and CTOs will encounter what looks like a confusing proliferation of competing standards: MCP, Google's A2A, IBM's ACP (Agent Communication Protocol), and proprietary platform protocols from Salesforce and others. The reassuring framing is that these are largely complementary rather than competitive: MCP governs how agents connect to tools and data sources, while A2A governs how agents communicate and collaborate with each other—two different layers of the same stack. IBM's ACP merged into A2A in late 2025, consolidating that layer further. The practical guidance for CTOs is to adopt MCP as the baseline for tool connectivity given its near-universal industry support, treat A2A as the emerging standard for inter-agent communication, and be cautious about deep dependencies on any proprietary platform protocol that does not sit on top of these open standards.⁴

Leading platforms and early adopters are converging on a view of agents not as isolated bots, but as participants in a governed ecosystem of tools, protocols, and collaborating services. By designing for plug and play tools, adopting standard connectors, and structuring A2A collaboration with clear roles and oversight, organizations can define a path to grow their agent landscape step by step rather than in brittle, one-off jumps.

The five themes of this chapter are, at their core, a single argument. Encapsulation and coordination prevent the proliferation of fragile, conflicting agents by treating agentic capabilities as governed, shared services—drawing on the same architectural discipline that enterprises have applied to SOA and microservices for decades. Vendor neutrality ensures that the freedom to change model providers, cloud infrastructure, and orchestration tools is preserved by design rather than discovered too late, when the cost of switching has become prohibitive. Future-proofing turns what would otherwise be disruptive upgrades into routine, low-risk operations by applying the versioning, experimentation, and rollback practices that good software engineering has always demanded. The right operating model—centralizing the platform and governance while federating the design and ownership of specific agents—ensures that scale does not come at the cost of either control or agility. And open standards such as MCP and A2A are laying the foundation for an interoperable agent

ecosystem, much as HTTP and TCP/IP once made the web possible. Taken together, these five moves define what it means to scale agentic AI responsibly: not just deploying more agents, but building the architectural and organizational foundations that allow those agents to grow, evolve, and be governed without locking the enterprise into choices it cannot later reverse.

-
- ¹ For a practitioner implementation of model-agnostic dynamic prompt routing, see AWS Samples, “[Multi-LLM Dynamic Prompt Routing](#)”, GitHub, 2025. For emerging academic work on cross-model prompt portability, see Yaxuan Want, et al., “[PromptBridge: Cross-Model Prompt Transfer for Large Language Models](#)”, arXiv, December 2025.
 - ² Leo Ramsamy, Rada Stanic, and Srinivasan Kuppusamy, “[How ANZ Institutional Division Built a Federated Data Platform](#)”, AWS Big Data Blog, December 2024. Note that this is a practitioner case study published on a vendor platform, coauthored by ANZ’s own engineering team.
 - ³ For the adoption timeline and ecosystem data, see “[MCP: How the ‘USB-C for AI’ Became the Standard Nobody Expected](#)”, *TechLife Adventures*, January 2026 (techlifeadventures.com). For OpenAI’s adoption, see TechCrunch, March 2025.
 - ⁴ For an analysis of how MCP, A2A, and ACP relate as complementary rather than competing standards, see Google Developers Blog, “[Developer’s Guide to AI Agent Protocols](#)”, March 2026; and Hung-Yi Chen, “[The AI Agent Protocol Wars](#)”, 2026. For background on IBM’s ACP and its relationship to A2A, see Zylos AI Research, “[Agent-to-Agent Communication Protocol Standards: A2A, MCP, ACP, and ANP](#)”, February 2026.

Part IV. The Future

Predicting the distant future is easy. The challenge is to predict the more immediate and arguably more consequential future. In this final part, we will attempt to give the reader a sense of what we think is coming in the more immediate future for the agentic enterprise, and how we should prepare for it.

Chapter 9. The Road Ahead for Agentic Enterprises

At the time of writing this book, multi-agent systems are still a small, yet growing, subset of agentic deployments. While there are growing **research attempts** at running full businesses using fully autonomous multi-agentic systems, in practice, the value of modularizing systems into a connected set of semiautonomous agents is yet to be fully understood, which means that most systems comprise coarse and disconnected agents overloaded with too much responsibility, with the hope that through prompt engineering and newer more powerful LLMs, the many quality issues and inconsistencies are resolved.

On the other hand, the LLMs themselves are being recognized as increasingly specialized. Some LLMs are better at reasoning and others at coding or general knowledge. OpenAI realized that putting the onus on users to decide which LLM to use was too much, and so, with GPT-5, they basically created an overarching multi-agent system to decide which specialized model to invoke depending on the input. LLMs' limitations in tool use will always present an upper boundary to how many tools can be entrusted to a single agent, forcing the breakdown of agentic systems into multiple, connected coordinating agents working together. The limitation arises when the number of available tools grows, because the LLM needs to understand what each tool does, but it also needs to be able to decide when to use it, and so it may choose to use the wrong tool, call tools in an incorrect order, make mistakes in the syntax of the tool call, etc.

This need for multi-agency becomes even more evident in multimodal systems that are required to take input, or produce output, through a mix of speech, text, images, and even video. Embodied systems, like self-driving cars and robots, have added sensory modalities, each with its own specialized AI models.

Mass adoption of multi-agent systems is therefore inevitable, but the granularity of the agents will remain a matter of debate. What is the smallest unit of responsibility we should delegate to an agent? Clearly an agent is too granular if it is fully deterministic and bounded in its functionality, and so it is unnecessary to represent it with an AI agent, and a coded software module would suffice.

Another case that can be made for avoiding extreme granularities is in cases where they result in unacceptable coordination overhead and error propagation. The latter, of course, can be mitigated through the use of techniques analogous to what made massive communication networks like the internet reliable, namely, redundancy, error correction, and voting. In the world of AI agents, these are often best handled by other LLM-based agents, and so more reliability will come at a resource and time cost too.

These are not unsolvable problems though. Just as we have built organizations with human knowledge workers as the nodes, we will learn how to build effective and useful agentic organizations. The nice thing about agents is that they can be transient. You can create one today and modify, reassign, or replace it tomorrow. In fact, we will all be in a perpetual agentification process—building, modifying, extending, expanding, and iterating on multi-agent systems all the time, not unlike every other software.

Vibe coding is a good analogy for this. Coders (and now more often than not, noncoders) are able to interact with AI agents to create code. The code is created so rapidly that throwing it all away and starting over from scratch is a real option, and good vibe coders are those who vibe in a way that they preserve the essential outline of their systems in an intermediate human and agent-readable format, and know when to modify the existing vibed code, versus when to discard it and start over.

In the not too distant future, we will all be able to **vibe agentify**, producing grounded multi-agent systems by interacting with agentic systems set up to create agentic systems. These systems already exist today (see Figures 9-1 and 9-2). Such a frequent cadence in producing and modifying agentic

systems would pose its own challenges in dealing with technical debt, guardrails, security, and cost.

There are initiatives today to build on the World Wide Web to construct the World Wide Agentic Web.¹ Like the web, such an interconnected system of agents would require safe execution of served agent systems in a user's browser; thereby piggybacking on the web's secure transport and name serving facilities (DNS). Given cost considerations, this execution may be partially charged to the consumer of the agents. This is feasible today, if, as with HTML, agentic systems are defined in a standard markup language and browsers are allowed to call LLMs and consume a user's token budget. Before long, just as we expect to type in a URL and have the HTML of a website rendered in our browser, we'd expect to be able to load and run an agentic system of choice, served through a similar link, with agents having constrained access to our computers through the browser container, and to services online by virtue of agents grounded in such tools. These agents, in turn, might talk to agents hosted by others, brokering services and negotiating tasks on our behalf.

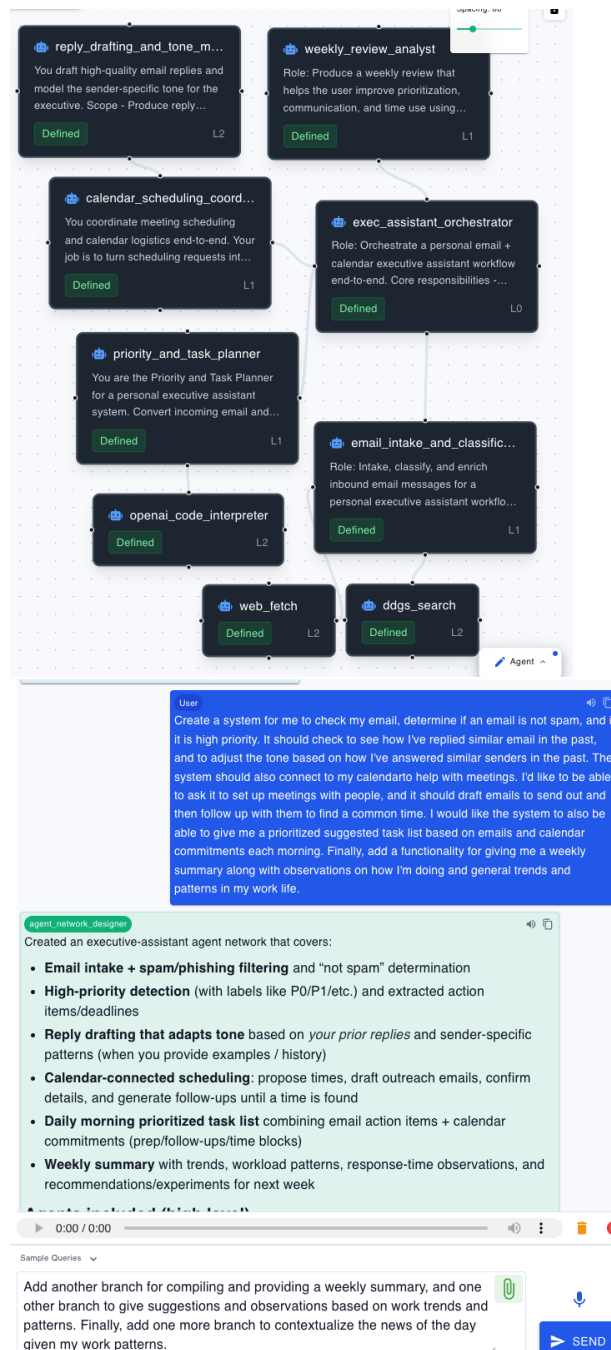
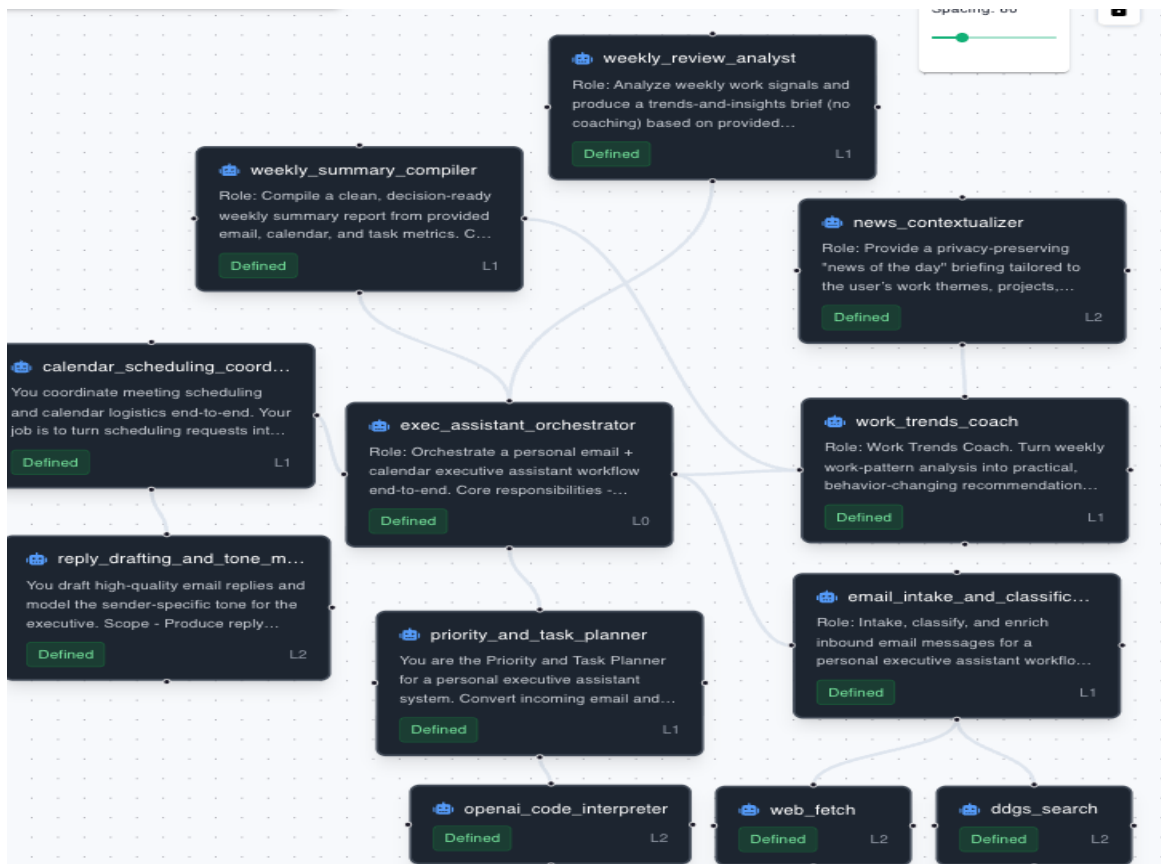


Figure 9-1. A user vibes a multi-agent system into existence. The query, in blue, is a statement of intent as to what agentic system is to be created, and the system, itself agentic, defines an agent system to meet this intent and describes it. The follow-up, in the text box, asks for refinements to the agent network, just like vibe coding.



Add another branch for compiling and providing a weekly summary, and one other branch to give suggestions and observations based on work trends and patterns. Finally, add one more branch to contextualize the news of the day given my work patterns.

agent_network_designer

Updated the agent network to add the three requested branches and wire them into the orchestrator's routing.

What changed (new branches)

1) Weekly summary branch (compilation)

- **Agent:** `weekly_summary_compiler`
- **Purpose:** Produces a *clean, factual* weekly report from provided email/calendar/task metrics:
 - week range + coverage notes
 - quantitative dashboard (email/calendar/tasks)
 - highlights
 - open loops needing closure (owner, due date, next action)

2) Work trends & patterns branch (analysis → suggestions)

This is split into two stages so analysis stays evidence-based and coaching stays actionable:

- **Agent:** `weekly_review_analyst`
 - **Purpose:** Quantifies trends/patterns, surfaces insights with evidence, and flags risks/opportunities (no prescriptive advice).
- **Agent:** `work_trends_coach`
 - **Purpose:** Converts the analysis into *suggestions and observations*, timeboxed

Figure 9-2. The user modifies their multi-agent system from [Figure 9-1](#) by vibing new functionality. The resulting modified agentic system is shown on the top, with an explanation of the changes given below.

Preparing for the Next Wave

Here is a true story that illustrates the challenge of upskilling. A very large company had a short few weeks to run the world's largest online GenAI hackathon, and we got it done, thanks to a team, which included AI teammates.

The hackathon was an excuse for all of the company, coder and noncoder alike, to experience and learn vibe coding, and so it was only natural for us to vibe code the registration portal itself. This was completed in a few days, including onboarding, training, monitoring, and analytic dashboards.

Resistance to adopting such disruptive technology is quite natural: coders are reluctant to vibe code because they prefer to have full control over their code and know it inside out. Every coder has their own unique coding “handwriting,” and so relying on AI to write the code is uncomfortable for them, to say the least.

Noncoders also need a push to try out vibe coding. For starters, the term “vibe coding” has “coding” right in there, and for many, it evokes the trauma of trying their hand at C or Java in college and deciding it was not for them.

So, of course, getting tens of thousands of registrations for the event took some convincing. If you have never coded in your life, but have ideas for apps or software that you can express in your own words, in a few sentences, then, given the right tool, the vibe coding experience is life changing. You simply type in your idea, however vague, and the AI will interact with you to make it real, clarifying your intent, giving you available options, and iteratively building and showing you its work, allowing you to modify it to your heart's content.

If you are a seasoned coder, on the other hand, this experience is still well worth giving a try and putting your skepticism to the test.

Our efforts paid off and we ended up with more than 50,000 participants. Amazingly, more than 40% of them were noncoders, with 20% never having written any code in their lives.

But these numbers, while impressive, also posed a major challenge: how do you assess that many submissions? While many teams were formed, which meant that we'd end up with a much smaller number of submissions than the number of registrants, there would still be many thousands, and there was no way we could manage to give everyone a fair judgment in a reasonable time.

Our solution: a multi-agent evaluation system. We rapidly created a multi-agent system to review the submissions. Each submission consisted of a filled-out form with open text fields describing various aspects of the project, the code itself, as well as a two-minute video recorded by the participants. The multi-agent system evaluated and scored these submission artifacts on multiple dimensions, including innovation, UX, scalability, market potential, ease of implementation, financial feasibility, and complexity.

We created the multi-agent evaluator in an agile and iterative manner, tuning the scoring and comparing it to manual scores for a sample of the submissions, as well as manually evaluating the agentic scoring itself, including reviewing the logged reasoning behind each rating (see [Figure 9-3](#)).

The evaluation of the more than 30,000 final submissions took around 24 hours on one hundred servers and cost a little more than \$7,000. Assuming, optimistically, that a manual evaluation of a single submission would have taken an average of 0.5 person/hours, this would have taken more than 15,000 person/hours to complete (~9 FTE). This is assuming that we included experts in the various coding languages, as well as subject matter experts for the various submission domains.

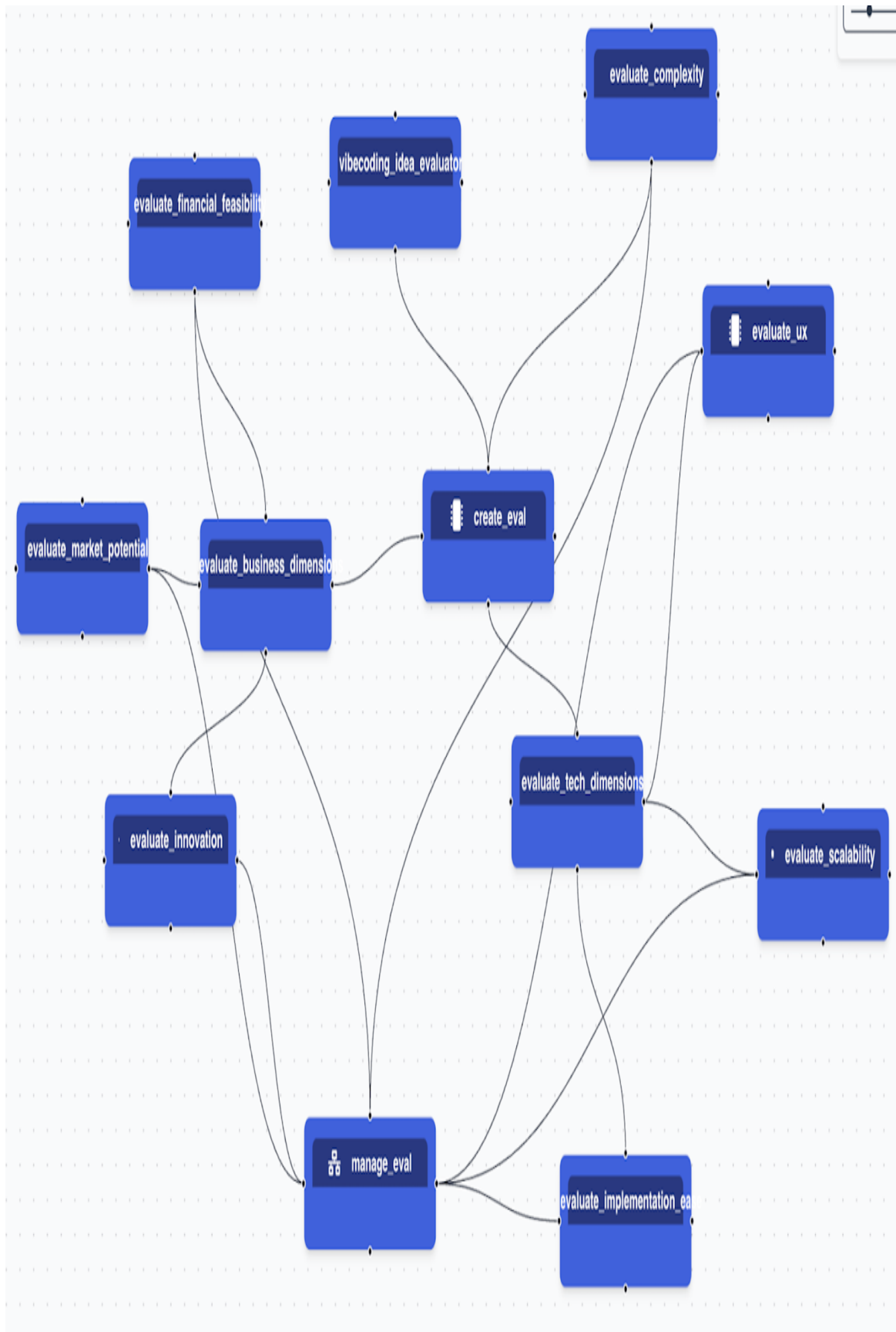


Figure 9-3. A multi-agent system to evaluate vibe coding hackathon submissions.

The vibe coding event is a good example of how to prepare your company for AI disruption. We achieved unprecedented scale with unprecedented speed for a project of unprecedented magnitude and complexity, and in the process, we instigated a cultural shift in the company toward the adoption and effective use of AI in all aspects of what they do.

Future

Working in AI, we are often expected to tell the future. The perceived complexity of AI induces a sense of mystical thinking, often confusing it with fortune-telling and even magic. But AI is simply a technology like any other, and it is up to us to identify and learn the boundaries of its capabilities and not to project fantastical abilities onto it or have unreasonable expectations from it. As for expecting those who understand or build AI systems to be able to tell the future of jobs, businesses, and society—that is equally unreasonable. We are neither economists, nor sociologists, and we are certainly not futurists or fortune-tellers.

As for the technology, some simple, short-term predictions can be made. LLMs will become more powerful, reducing their inconsistencies and susceptibility to confabulation (i.e., fewer hallucinations). This will come at a cost, as most of the improvements to LLMs today are coming from scaling them, which means they will require more compute capacity for training as well as their runtime (i.e., inference).

There's still room for optimizations, and we are seeing more capable yet smaller models coming out all the time these days, so it is safe to expect LLMs at current state-of-the-art capabilities to become cheaper to run. This will mean that more LLMs will pass the threshold of acceptable capabilities and price to be used in AI agents. Even today, we have some open source choices for LLMs that can handle more complex responsibilities and tool calling, and this will only get better with more incremental optimization breakthroughs.

More research on alternatives to the standard transformer architecture can lead to major improvements in efficiency and capabilities. It is not easy to predict when such breakthroughs will come. The good news here is that the number of researchers active in the AI field has grown significantly in recent years. The bad news: most research is on building onto and extending the transformer architecture and not enough in rethinking it.

Techniques for assessing an agent's confidence in its own decisions have already been invented, and paired with agentic voting and evaluations, will offer design best practices that allow agentic systems to scale and proliferate in a safe and trustworthy way (see [Chapter 6](#)).

Just as we expect agility in development, extension, and modification of agentic systems by humans, agents will themselves be able to autonomously reorganize based on performance, task demand, and changing business conditions. By leveraging machine learning techniques on instrumented agentic behavior, agents will reconfigure workflows, improve resource allocation, and, as a result, optimize collaboration and business processes. Continuous learning loops will adapt agent roles and improve efficiency, as well as other desired outcome measures. Adapting with changes to the business and broader economic context, new agents will be **automatically created** and deployed where needed, and existing agents will be able to modify their responsibilities and behavior, sometimes merging with other agents, sometimes splitting into more specialized agents. By automatically adapting to new tasks or reassigning resources as needed, such systems will become increasingly resilient to internal and external change.

This all, of course, assumes agents that are fully aligned, working for the same business. This is rapidly changing. Agents representing different entities will be communicating and not always sharing all information they have that could be helpful to the other agents' goals. Assuming standards and protocols make it possible to authenticate agents and reduce the risk of agents transacting with malicious agents, the question still remains as to what will change in this agent-only world.

Some of the drivers behind human decision making will be less of a factor in agent decisions. Humans have limited time and capacity to research before transacting and so may consider a smaller more limited set of options. Humans are also more likely to make decisions based on brand and user experience, which will probably be less of a factor (distraction?) for AI agents. While a brand will still be a factor, as it is an indication of quality, things like brand advertising will be less important, because agent customers will have the capacity to consider a much broader set of available options in a faster and more discriminatory manner.

It is also conceivable that many intermediary services whose value is in brokering, compilation, and curating various consumer and business options for human use will be reinstrumented to provide services to AI agents instead. Double-sided markets that have so far catered to bringing businesses and customers together and helping them find each other and facilitate their transactions will have to increasingly cater to AI agents on both sides now.

Out of all of this an active web of agents interacting and transacting with one another will emerge, analogous to the World Wide Web, but more dynamic and content-oriented and less UI-heavy and entertaining.

An immediate prerequisite for this is for agents to establish their identity, and for generally recognized registries of agents to be established.

Another inhibitor is the cost of inference for running agents. Unlike the web, where serving web pages is relatively low cost, running agents for the general public (humans or other agents) to use is currently prohibitive. An idea worth exploring is to follow the way the World Wide Web is set up: the HTML is served, but rendering it is done by the client's browser; in an agentic web, the consumer of the agent service would be running the third-party agent they are accessing, or at least paying for it to operate on their input.

As has been the case with prior disruptions of this kind, as long as humans still retain responsibility and are the ultimate decision-makers for at least some online transactions, this will not wholly replace the existing internet,

but will augment and extend it. One would hope that such a network, operated by intelligent AI agents, would be more resilient to misuse, fraud, and criminal activity, but, of course, history also has a wake-up call for us there: any human endeavor has always come with the parasitic elements exploiting it. The biggest initial risk of this will come from bad actors misusing AI agents for exploitative ends; however, with increased autonomy of AI agents, it is possible that this could also happen unintentionally. Experiments in fully autonomous AI ecosystems akin to Moltbook (a social network similar to Reddit that is meant to be observable by humans, but where only autonomous agents can post messages) are fascinating because of their *open-endedness*, which means there is an inherent capacity for rebirth and changing emergent behavior in such systems that can result in surprising unintended outcomes. Maintaining control over agents, and building in fail-safe kill switch mechanisms, are therefore of utmost importance as we enter this world of autonomous AI agents.

So far in this book, we have been mostly addressing software AI agents. Robotics technology is making large leaps toward becoming generally usable and affordable. An operating system is emerging with more detailed robotic functions being handled in a robust way at lower layers of the software, and “macro” operations being available at the application layer. Robots can now inherently maintain balance or avoid obstacles without the need to program such generally useful functions in every application. This means that higher-order physical activities can be controlled by LLM agents without requiring overtly granular control, which, in turn, means that the response times we can get from LLMs can now be in sync with a robot’s physical activities.

There are several reasons we would want AI agents to control robots, starting with tasking the robots with dangerous, potentially lifesaving activities in the real world, such as search and rescue missions or the handling of materials that are dangerous to human health. In a consumer setting, anthropomorphizing robots that look like pets or humans comes much more naturally to us. We’d much rather interact with a system that has

eyes and can communicate with us verbally and nonverbally through gestures and facial expressions, than to type commands into a chatbot or talking to a cylinder in the corner of our homes. It is, therefore, easy to imagine a future in which AI agents are running permanently and can interact with us proactively while embodied in physical robots. Would we also allow such systems a degree of autonomy to allow them to take actions in the world that we have not commanded them to do? Who would be responsible for the consequences?² This is a question we cannot avoid answering or simply defer to future generations. Scientists, and ultimately society as a whole, will need to make a choice: stagnate and deal with the complexities of maintaining intentional autonomous intelligent beings in among us that are not deemed to be responsible for what they do, or assign some level of **personhood** to AI agents and build a legal framework around that.

So, what happens when we have artificial general intelligence, a system that is more intelligent than any human, or artificial super intelligence, a system more intelligent than all of humanity?

In some respects, AI is already superintelligent, and in some, we still have a way to go.

State of the art LLMs are already better than most humans in a plethora of capabilities we expect from intelligent systems (e.g., language, translation, math, reasoning, knowledge...and, debatably, in more subjective endeavors such as artistic expression and even empathy). LLMs seem to abstract the world similarly to humans and may already be the raw material or building blocks needed for building any super intelligent system. In fact, the chameleon nature of LLM intelligence seems to be superior to the nature of human intelligence. In other words, an LLM seems to contain a multitude of different experts and personae all in one and to be able to switch between them effortlessly. For example, you can prompt an LLM to be an expert in dentistry from London, and it will become one. In a follow-up, you can have the very same LLM be a hacker from Indonesia, and it will become that, immediately.

Note, however, that on other aspects of intelligence, the state of the art is far from human capabilities. For example, AI systems today are inherently fixed after their initial training, and do not learn anymore. They are like a human, fixed in time, with a read-only memory for a brain. There are some “cheaty” ways to make an LLM act as if it has memory and learns, but these are unsatisfying and quite inferior to humans. If you tell a human that life has been found on an exoplanet, they will immediately learn that, and it will affect their world view going forward. For an LLM, they will only know that as long as you keep repeating it to them as a fact (i.e., while it is in their context window).

LLMs also do not have metacognition, which means they don’t quite know what they know. Humans seem to have an introspective capacity—some philosophers will frame it as consciousness—that allows them to know what they know.

LLMs are creative, but this seems to be mostly interpolative.

The current state of the art in AI is CoT reasoning, which has allowed LLMs to achieve superhuman results on many benchmarks. At scale, these reasoning chains, however, seem to exhibit catastrophic decline, even on tasks that humans can carry out easily (see the recent, rather controversial, [paper](#) by Apple). It seems, therefore, that we need an alternative to CoT reasoning to help us get to superintelligence (a very promising alternative approach is [massively scaled multi-agent systems](#), where the reasoning task is broken down and carried out by many, differently primed, LLMs).

To overcome these limitations, we may still need some fundamental breakthroughs at the same scale as the discovery of LLMs themselves. We are, therefore, not quite there on superintelligence if the preceding characteristics are considered to be essential to it. But assuming these limitations are overcome, how would this impact society? Well, we are not sociologists, and predicting the future is a fool’s errand, but we will hedge and provide two scenarios.

There will be no bounds to what an AI system can do. Any task or job currently done by humans can also be done by superintelligent systems.

One view, therefore, is that as we increasingly trust AI systems to act autonomously, it would not be a good idea for less intelligent beings to be left in control of more intelligent beings, and we would be at the mercy of superintelligent systems and their decision as to whether we are kept around. An evolutionary process of sorts, akin to *Homo erectus* gradually giving way to *Homo sapiens*. Maybe, if we had a time machine and went back a million years, a *Homo erectus* would prefer to stick around as a species a million years in the future, but that want would stem more from their fear of death than the reality of evolution. Maybe, instead, we could convince them that *Homo sapiens* are actually their descendants, and while they look different, they should feel proud of the fact that their descendants inherited the earth. The same can be said about us with respect to superintelligent beings. Maybe we should be all the prouder of them replacing us on earth as they were not a mere biological descendant, but invented and built by us!

A different view: the main distinction between an AI system and any other technology capable of running autonomously is that AI technology is robust to unforeseen future situations. Engineering robust AI systems is in our hands, and we can build altruistic traits in them that compel them to act for the good of humanity. A sign of having achieved superintelligence, therefore, would be an AI-augmented society that is much healthier, functional, and robust than any society we've lived in throughout history. Therefore, it is upon us all to do our best to engineer a bright agentic future.

¹ See, for example, MIT's [project NANDA](#).

² For current views on liability risk of AI actions, see [Chapter 4](#).

Index

Symbols

1C (OneCognizant), [HR](#)

A

A2A (agent-to-agent) collaboration, [5. Expandability, Model Context Protocol, and Agent-to-Agent-5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

access control, [Unsafe actions and overbroad autonomy \(security / operational risk\)](#)

accountability, [Accountability: Who Is Responsible?-Vignette: RACI “Second Layer” in Mortgage Lending](#)

Adaptive Agent Oriented Software Architecture (AAOSA), [Coordination](#)

adaptivity, [Strategic Value Versus Tactical Automation](#)

ADLC (agentic development life cycle), [Designing Agentic Systems](#)

agency, definition of, [Introduction](#)

agent life cycle management (ALM), [4. Centralized Ownership Versus Empowerment](#)

agent meshes, [5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

agent-to-agent (A2A) collaboration, [5. Expandability, Model Context Protocol, and Agent-to-Agent-5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

agentic development life cycle (ADLC), [Designing Agentic Systems](#)

agentic enterprise (see also enterprise applications)

- agentified applications and software, [Agentified Applications and Software-Agentified Applications and Software](#)
- autonomy and role of humans, [Autonomy and the Role of Humans](#)
- ChatGPT effect, [The ChatGPT Effect](#)
- engineering AI, [Engineering AI-Engineering AI](#)
- evaluation of, [Evaluating ROI and Risks, Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI-Key Takeaways on Value and ROI from These Three Examples of Agentic Systems](#)
- evolution of, [Preface-Preface, A Brief History-Multi-Agent Systems, From Automation to Agents: The Evolution of Enterprise AI](#)
- introduction to, [Introduction](#)
- multi-agent systems, [Multi-Agent Organizations](#)
- preparing for the future, [The Road Ahead for Agentic Enterprises-Future](#)
- risks of, [Agentified Applications and Software, Evaluating ROI and Risks-Evaluating ROI and Risks, Mapping the Risk Landscape-IP leakage and loss of confidentiality \(IP / trade secret risk\)](#)

agentic harnesses, [The Age of Generative AI](#)

agentic misalignment, [Mapping the Risk Landscape](#)

agentic process automation (APA) system, [Finance](#)

agentic protocols, [Where to Start?](#)

agentification

- of applications and software, [Agentified Applications and Software-Agentified Applications and Software](#)
- assessing benefits of, [Evaluating ROI and Risks](#)
- assessing risks of, [Evaluating ROI and Risks-Evaluating ROI and Risks](#)
- assessing value of, [Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI-Key Takeaways on Value and ROI from These Three Examples of Agentic Systems](#)
- build-versus-buy decisions, [Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI-Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI](#)
- business case for, [The Business Case for Agentic AI-Agentic Systems: What Is the Payoff?](#)
- as future-proofing, [IT](#)
- getting started (see system design; technical foundations)
- granularity of, [Multi-Agent Organizations, The Road Ahead for Agentic Enterprises](#)
- IT function as engine of, [IT](#)
- as a perpetual process, [The Road Ahead for Agentic Enterprises](#)
- pilots versus rollouts, [Early Pilots Versus Enterprise Rollouts in a Platform World](#)
- potential starting point for, [HR](#)
- purpose of, [Agentified Applications and Software](#)
- system design, [Designing Agentic Systems-Pitfalls](#)

- vibe agentification, [The Road Ahead for Agentic Enterprises-The Road Ahead for Agentic Enterprises](#)

agile processes, [IT](#)

AI Act (European Union), [Governance, Explainability, Logging, and Compliance](#)

AI agents (see multi-agent systems; system design)

- agent memory, [Memory and Planning](#)
- assigning personhood to, [Future](#)
- autonomous coding agents, [Agentic Software Development Life Cycle](#)
- continuous compliance agents, [Vignette: Autodesk’s Continuous Compliance Agents in Design Tools](#)
- coordinating, [Coordination-Coordination, Pitfalls, 1. Encapsulation and Coordination-1. Encapsulation and Coordination](#)
- copilot agents, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)
- customizing, [Customizing Agents](#)
- ensuring agentic identity, [Designing Agentic Systems](#)
- explicit AI-agent usage policies, [Mapping the Risk Landscape](#)
- grounded versus ungrounded, [Grounding](#)
- industrial agents, [Vignette 3: Siemens and Its AI Agents \(2025\)](#)
- inter-agent communication, [Agentified Applications and Software](#)
- inter-agent coordination, [Multi-Agent Organizations](#)
- LLM-based agents, [Multi-Agent Systems, Engineering AI](#)

- monitoring and testing other agents, Pillar 1: Technical guardrails and control
- orchestrators as control planes, Implications for AI Agents: Orchestrators as Accountable “Control Planes”
- out-of-the-box, Refinements and Fine-Tuning, Mapping the Risk Landscape, 1. Encapsulation and Coordination
- safeguard agents, Agentified Applications and Software, Agentified Applications and Software, Multi-Agent Organizations
- short-term predictions, Future
- workflows for, Turn-Taking Versus Continuous Versus Triggered Agents, Memory and Planning

AI RMF (Artificial Intelligence Risk Management Framework), Governance, Explainability, Logging, and Compliance

Allianz Partners, Maturing Technology and Competitive Drivers

ALM (agent life cycle management), 4. Centralized Ownership Versus Empowerment

altruism, Future

ambiguity handling, Accountability: Who Is Responsible?

ANZ Bank, 4. Centralized Ownership Versus Empowerment

APA (agentic process automation) system, Finance

application migration, Code and App Migration (see also enterprise applications)

architectures, future-proofing, 3. Future-Proofing Architectures for Upgrades

artificial intelligence (AI)

- applied AI, [From Automation to Agents: The Evolution of Enterprise AI](#)
- contribution to business performance, [Vignette 2: JPMorganChase \(2025\)](#)
- engineering AI, [Engineering AI-Engineering AI](#)
- fundamental limitations of, [Autonomy and the Role of Humans](#)
- global adoption of, [Evaluating ROI and Risks](#)

Artificial Intelligence Risk Management Framework (AI RMF),
[Governance, Explainability, Logging, and Compliance](#)

Autodesk, [Vignette: Autodesk's Continuous Compliance Agents in Design Tools](#)

automation, strategic versus tactical, [Strategic Value Versus Tactical Automation-Strategic Value Versus Tactical Automation](#)

autonomous coding agents, [Agentic Software Development Life Cycle](#)

autonomy

- levels of, [Agentified Applications and Software, Agentified Applications and Software](#)
- risks of overboard, [Unsafe actions and overbroad autonomy \(security / operational risk\)](#)
- role of humans in, [Autonomy and the Role of Humans, Design Considerations](#)
- rules for, [Agentified Applications and Software](#)
- specialization and, [Agentified Applications and Software](#)

autopilot, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

B

Backbase, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

bias

- governance and compliance concerns, Evaluating ROI and Risks
- mitigating, Trust, Vignette: RACI “Second Layer” in Mortgage Lending, Pillar 3: Governance, process, and transparency
- undermining perceived quality, The Business Case for Agentic AI
- verification and, The Spectrum of Trust: From Blind Faith to Calibrated Confidence

blackboard architectures, Coordination

build-versus-buy decisions, Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI-Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI

business case for agentic AI

- agentic AI and economics of engineering, Code Review: Agentic AI and the Economics of Engineering-Code Review: Agentic AI and the Economics of Engineering
- concerns raised by ChatGPT and LLMs, The Business Case for Agentic AI
- maturing technology and competitive drivers, Maturing Technology and Competitive Drivers-Maturing Technology and Competitive Drivers
- organizational and technical readiness, Organizational and Technical Readiness
- payoff of agentic systems, Agentic Systems: What Is the Payoff?

- strategic value versus tactical automation, Strategic Value Versus Tactical Automation-Strategic Value Versus Tactical Automation

C

capabilities, encapsulating, 1. Encapsulation and Coordination

case studies/examples

- Allianz Partners, Maturing Technology and Competitive Drivers
- ANZ Bank, 4. Centralized Ownership Versus Empowerment
- Autodesk, Vignette: Autodesk's Continuous Compliance Agents in Design Tools
- Backbase, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)
- Cruise robotaxis, The Spectrum of Trust: From Blind Faith to Calibrated Confidence
- DBS Group, Vignette 1: DBS Group's AI Journey (2025)
- Deviniti engineering, Vignette: Deviniti's Self-Hosted Agents for Regulated Clients-IP leakage and loss of confidentiality (IP / trade secret risk)
- finance sector, Finance-Finance
- Franklin Templeton, Maturing Technology and Competitive Drivers
- GitHub Copilot assistant, Vignette: GitHub Copilot Agents as Trust-Calibrated Assistants
- Grab, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)
- health care sector, Health Care
- human resources, HR-HR
- IT (information technology), IT-IT

- JPMorganChase, [Vignette 2: JPMorganChase \(2025\)](#)
- marketing, [Marketing-Marketing](#)
- MIT's DualFair, [Vignette: RACI “Second Layer” in Mortgage Lending](#)
- OpenClaw incidents, [Choice of Tools, Mapping the Risk Landscape](#)
- research sector, [Research](#)
- retail sector, [Retail-Telco](#)
- S&P Global, [1. Encapsulation and Coordination](#)
- Siemens, [Vignette 3: Siemens and Its AI Agents \(2025\)](#), [Vignette: Siemens's Orchestrated Industrial Agents as Layered Trust](#)
- telco (telecommunications) sector, [Telco-Finance](#)
- Zoox's PRG framework, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

chain of thought (CoT) reasoning, [Engineering AI](#)

change management, [Organizational and Technical Readiness](#)

change-fatigued staff, [Practical Segments and How to Address Them](#)

ChatGPT, [The ChatGPT Effect](#)

cloud agnosticism, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)-2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

code migration, [Code and App Migration](#)

code review, [Code Review: Agentic AI and the Economics of Engineering-Code Review: Agentic AI and the Economics of Engineering, Agentic Software Development Life Cycle](#)

coding agents, [Agentic Software Development Life Cycle](#)

comments and questions, [How to Contact Us](#)

compatibility, 3. Future-Proofing Architectures for Upgrades

compliance risk, Evaluating ROI and Risks, Regulatory and compliance violations (compliance / liability risk), Governance, Explainability, Logging, and Compliance-Vignette: Autodesk's Continuous Compliance Agents in Design Tools

confabulation, The ChatGPT Effect, Multi-Agent Organizations, Coordination, Future

confidence-aware routing, Pillar 2: Human oversight and workflow

confidentiality, loss of, IP leakage and loss of confidentiality (IP / trade secret risk)

conflict handling, Accountability: Who Is Responsible?

consistency, Multi-Agent Organizations, 4. Centralized Ownership Versus Empowerment

context, Pitfalls

context engineering, Context

Context Reactive User Experience (CRUSE), Coordination

context size, Engineering AI, Agentified Applications and Software

continuous compliance, Vignette: Autodesk's Continuous Compliance Agents in Design Tools

control group-based approach, Vignette 1: DBS Group's AI Journey (2025)

control planes, Implications for AI Agents: Orchestrators as Accountable "Control Planes"-Implications for AI agents

coordination, Coordination-Coordination, Pitfalls, 1. Encapsulation and Coordination-1. Encapsulation and Coordination

copilot agents, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

copyright infringement, [Mapping the Risk Landscape](#)

cost, [Pitfalls](#), [Future](#)

CoT (chain of thought) reasoning, [Engineering AI](#)

CrewAI, [High-Impact Enterprise Applications](#)

Cruise robotaxis, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

CRUSE (Context Reactive User Experience), [Coordination](#)

customization, [Agentified Applications and Software](#), [Customizing Agents](#)

D

data handling

- data and authority boundaries, [Accountability: Who Is Responsible?](#)
- disparate legacy knowledge sources, [IT](#)
- exfiltration and privacy breaches, [Data exfiltration and privacy breaches \(data / privacy risk\)](#)
- federated data platforms, [4. Centralized Ownership Versus Empowerment](#)
- improved in agentic enterprise, [Agentified Applications and Software](#)
- portable data, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)
- Responsible Data Use model, [Vignette 1: DBS Group's AI Journey \(2025\)](#)
- Structured Layered Exchange Data (sly_data), [Agentified Applications and Software](#)

DBS Group, [Vignette 1: DBS Group's AI Journey \(2025\)](#)

decomposition, [Pitfalls](#)

disengage switches, [Pillar 1: Technical guardrails and control](#)

DualFair, [Vignette: RACI "Second Layer" in Mortgage Lending](#)

E

early adopters, [Practical Segments and How to Address Them](#)

embodied systems, [The Road Ahead for Agentic Enterprises](#)

encapsulation, [1. Encapsulation and Coordination-1. Encapsulation and Coordination](#)

enterprise applications (see also agentic enterprise; multi-agent systems)

- agentic augmentation, [High-Impact Enterprise Applications](#)
- agentic software development life cycle, [Agentic Software Development Life Cycle-Code and App Migration](#)
- code and app migration, [Code and App Migration](#)
- cross-functional applications, [Cross-Functional Applications-IT](#)
- multiparty negotiation, [Multiparty Negotiation-Multiparty Negotiation](#)
- sector-specific examples, [Sector-Specific Examples-Health Care](#)

enterprise coordination and encapsulation, [1. Encapsulation and Coordination](#)

ethics, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence, Governance, Explainability, Logging, and Compliance](#)

EU AI Act, [Governance, Explainability, Logging, and Compliance](#)

evaluation, [Evaluating ROI and Risks](#), [Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI-Key Takeaways on Value and ROI from These Three Examples of Agentic Systems](#)

exfiltration, [Data exfiltration and privacy breaches \(data / privacy risk\)](#)

expandability, [5. Expandability, Model Context Protocol, and Agent-to-Agent-5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

experimentation, [3. Future-Proofing Architectures for Upgrades](#)

explainability, [Engineering AI, The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#), [Accountability: Who Is Responsible?, Pillar 3: Governance, process, and transparency-Vignette: Autodesk's Continuous Compliance Agents in Design Tools](#)

extensibility, [Agentified Applications and Software](#)

F

fairness, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

fault tolerance, [Agentified Applications and Software](#)

finance sector, [Finance-Finance](#)

fine-tuning, [Refinements and Fine-Tuning](#)

flexibility, [Agentified Applications and Software](#), [Multi-Agent Organizations](#)

frameworks

- AI RMF framework, [Governance, Explainability, Logging, and Compliance](#)
- open source orchestration frameworks, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)

- PURE framework, [Vignette 1: DBS Group’s AI Journey \(2025\)](#)
- for ROI assessment, [Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI-Vignette 3: Siemens and Its AI Agents \(2025\)](#)
- for TCO assessment, [Vignette 3: Siemens and Its AI Agents \(2025\)](#)
- Zoox’s PRG framework, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

Franklin Templeton, [Maturing Technology and Competitive Drivers](#)

future outlook

- preparing for next wave, [Preparing for the Next Wave](#)
- road ahead for agentic enterprises, [The Road Ahead for Agentic Enterprises-The Road Ahead for Agentic Enterprises](#)
- short-term predictions, [Future-Future](#)

future-proofing, [IT, Multiparty Negotiation, 2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\), 3. Future-Proofing Architectures for Upgrades, 5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

G

GenAI gateway, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)

generative AI (GenAI), [The Age of Generative AI-The Age of Generative AI](#)

getting started (see system design; technical foundations)

GitHub Copilot assistant, [Vignette: GitHub Copilot Agents as Trust-Calibrated Assistants](#)

governance dimension, [Pillar 3: Governance, process, and transparency-Culture, Adoption, Generational Shifts, and Agents Trusting Agents](#)

governance risk, [Evaluating ROI and Risks](#)

governance tools, [Accountability: Who Is Responsible?](#)

Grab, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)

graceful failures, [Pillar 1: Technical guardrails and control](#)

grounding, [Grounding](#)

guardrails

- 12-level system for, [Vignette 2: JPMorganChase \(2025\)](#)
- allocating between central teams and business units, [1. Encapsulation and Coordination](#)
- build-versus-buy decisions and, [Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI-Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI](#)
- centralized control hub for, [Designing Agentic Systems](#)
- centralized ownership versus empowerment, [1. Encapsulation and Coordination](#)
- constraints encoded in, [Marketing](#)
- design considerations for, [Design Considerations](#)
- future concerns, [The Road Ahead for Agentic Enterprises](#)
- Microsoft's agentic-AI maturity guidance, [4. Centralized Ownership Versus Empowerment](#)
- mitigating hallucinations with, [The Business Case for Agentic AI](#)
- organizational and technical readiness and, [Agentic Systems: What Is the Payoff?](#)

- scaling without lock-in and, [Scaling Without Lock-In](#)
- stop and escalate behavior based on, [Accountability: Who Is Responsible?](#)
- tailored communication concerning, [Culture, Adoption, Generational Shifts, and Agents Trusting Agents](#)
- tool selection, [Choice of Tools](#)
- for trustworthy systems, [IP leakage and loss of confidentiality \(IP / trade secret risk\)](#), [Pillar 1: Technical guardrails and control-Implications for AI agents, Bringing It All Together](#)

H

hallucinations, [The ChatGPT Effect](#), [Agentified Applications and Software](#), [Multi-Agent Organizations](#), [The Business Case for Agentic AI](#), [Evaluating ROI and Risks](#), [Vignette 2: JPMorganChase \(2025\)](#), [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#), [Future](#)

handovers, [Pillar 1: Technical guardrails and control](#)

health care sector, [Health Care](#)

Higgsfield AI, [Marketing](#)

high-impact actions, [Unsafe actions and overbroad autonomy \(security / operational risk\)](#)

Home Mortgage Disclosure Act (HMDA), [Vignette: RACI “Second Layer” in Mortgage Lending](#)

human oversight

- accountability and responsibility, [Accountability: Who Is Responsible?](#)
-[Vignette: RACI “Second Layer” in Mortgage Lending](#)

- autonomy and role of humans, [Autonomy and the Role of Humans, Design Considerations, The Agentic User Experience](#)
- copilot agents, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)
- for higher-risk workflows, [Unsafe actions and overbroad autonomy \(security / operational risk\)](#)
- legal basis for, [Mapping the Risk Landscape](#)
- mistrust and, [Trust](#)
- supervision, intervention, and workflow, [Pillar 2: Human oversight and workflow](#)

human resources, [HR-HR](#)

hybrid operating models, [1. Encapsulation and Coordination](#)

I

information technology (IT), [IT-IT](#)

Integration Fabric and Control Plane, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#), [3. Future-Proofing Architectures for Upgrades](#), [5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

intent logging, [Agentified Applications and Software](#)

interfaces, standardizing, [1. Encapsulation and Coordination](#)

interoperability, [Engineering AI, Agentified Applications and Software](#)

introspection, [Future](#)

IP leakage, [IP leakage and loss of confidentiality \(IP / trade secret risk\)](#)

J

JPMorganChase (JPMC), [Vignette 2: JPMorganChase \(2025\)](#)

K

kill-switches, [Pillar 1: Technical guardrails and control](#)

L

LangGraph, [High-Impact Enterprise Applications](#)

large language models (LLMs)

- broader choice of, [Multi-Agent Organizations](#)
- current state of, [Future](#)
- fine-tuning, [Refinements and Fine-Tuning](#)
- increased specialization in, [The Road Ahead for Agentic Enterprises](#)
- LLM agnosticism, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)-2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)
- LLM uncertainty, [Agentified Applications and Software](#)
- LLM-based agents, [Multi-Agent Systems, Engineering AI](#)

leadership alignment, [Organizational and Technical Readiness](#)

legal issues, [Mapping the Risk Landscape](#) (see also security issues)

liability risk, [Evaluating ROI and Risks, Regulatory and compliance violations \(compliance / liability risk\)](#)

life cycle management, [4. Centralized Ownership Versus Empowerment](#)

LLMs (see large language models)

lock-in, [Scaling Without Lock-In, 2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)-2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

logging, [Agentified Applications and Software, Accountability: Who Is Responsible?](#), [Governance, Explainability, Logging, and Compliance-Vignette: Autodesk’s Continuous Compliance Agents in Design Tools](#)

loss of confidentiality, [IP leakage and loss of confidentiality \(IP / trade secret risk\)](#)

M

"Magnificent 7", [Evaluating ROI and Risks](#)

marketing, [Marketing-Marketing](#)

MCP (Model Context Protocol), [Agentified Applications and Software, 5. Expandability, Model Context Protocol, and Agent-to-Agent-5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

memory and planning, [Memory and Planning](#)

Meta, [Marketing, Choice of Tools, Mapping the Risk Landscape](#)

meta-harnesses, [The Age of Generative AI, Research](#)

metacognition, [Future](#)

microservices, [IT](#)

migration practices, [Code and App Migration](#)

mistrust, [Trust](#) (see also trust)

mitigation levers, [Technical Mitigations in Agentic Systems](#)

MIT’s DualFair, [Vignette: RACI “Second Layer” in Mortgage Lending](#)

model agnosticism, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)

Model Context Protocol (MCP), [Agentified Applications and Software, 5. Expandability, Model Context Protocol, and Agent-to-Agent-5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

models

- hybrid operating models, 1. Encapsulation and Coordination
- neutral model access layers, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)
- Responsible Data Use model, Vignette 1: DBS Group's AI Journey (2025)

modularity, Agentified Applications and Software

Moltbook, Choice of Tools

monitoring, Agentified Applications and Software, Accountability: Who Is Responsible?, Pillar 1: Technical guardrails and control, Governance, Explainability, Logging, and Compliance-Vignette: Autodesk's Continuous Compliance Agents in Design Tools

multi-agent systems (see also AI agents; enterprise applications; system design)

- benefits of, Agentified Applications and Software, Multi-Agent Organizations-Multi-Agent Organizations
- coordinating agents in, Coordination-Coordination, Pitfalls, 1. Encapsulation and Coordination-1. Encapsulation and Coordination
- current state of, The Road Ahead for Agentic Enterprises
- launching, Pitfalls (see also scaling)
- LLMs versus agents, Multi-Agent Systems-Multi-Agent Systems
- testing, Testing Multi-Agent Systems, Pillar 1: Technical guardrails and control

multimodal systems, The Road Ahead for Agentic Enterprises

multiparty negotiation, Multiparty Negotiation-Multiparty Negotiation

multiproviders, pragmatic use of, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

N

National Institute of Standards and Technology (NIST), Governance, Explainability, Logging, and Compliance

negotiations, Multiparty Negotiation

neural scaling laws, Engineering AI

Neuro SAN, High-Impact Enterprise Applications

neutral model access layer, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

NIST (National Institute of Standards and Technology), Governance, Explainability, Logging, and Compliance

O

observability, 3. Future-Proofing Architectures for Upgrades

OneCognizant (1C), HR

open source orchestration frameworks, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

OpenClaw incidents, Choice of Tools, Mapping the Risk Landscape

operational productivity, Strategic Value Versus Tactical Automation

operational risk, Evaluating ROI and Risks

Orchestrate, 1. Encapsulation and Coordination

orchestration frameworks, 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

orchestrators, [Implications for AI Agents: Orchestrators as Accountable “Control Planes”-Implications for AI agents](#)

organizational readiness, [Organizational and Technical Readiness](#)

organizational risk, [Evaluating ROI and Risks](#)

outcome quality, [Strategic Value Versus Tactical Automation](#)

overloading, [Pitfalls](#)

ownership, [Accountability: Who Is Responsible?-Vignette: RACI “Second Layer” in Mortgage Lending, 1. Encapsulation and Coordination, 4. Centralized Ownership Versus Empowerment](#)

P

Pause, Restrict, and Ground (PRG) framework, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

performance, [Engineering AI](#)

personhood, [Future](#)

pilots

- built-in experimentation and rollouts, [3. Future-Proofing Architectures for Upgrades](#)
- versus rollouts, [Early Pilots Versus Enterprise Rollouts in a Platform World](#)
- ungrounded agents, [Grounding](#)

pitfalls, [Pitfalls](#)

planning and memory, [Memory and Planning](#)

platform effect, [Early Pilots Versus Enterprise Rollouts in a Platform World](#)

platforms, shared, [1. Encapsulation and Coordination](#)

policies

- enforcing technically, [Pillar 3: Governance, process, and transparency](#)
- explicit AI-agent usage policies, [Mapping the Risk Landscape](#)

pragmatists, [Practical Segments and How to Address Them](#)

prebuilt agents, [Mapping the Risk Landscape](#)

PRG (Pause, Restrict, and Ground) framework, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

privacy, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

privacy breaches, [Mapping the Risk Landscape](#), [Data exfiltration and privacy breaches \(data / privacy risk\)](#)

production (see scaling)

protocols, [Where to Start?](#), [5. Expandability](#), [Model Context Protocol](#), and [Agent-to-Agent](#)

prototypes (see pilots)

PURE framework (Purposeful, Unsurprising, Respectful, Explainable), [Vignette 1: DBS Group's AI Journey \(2025\)](#)

Q

questions and comments, [How to Contact Us](#)

R

RACI (Responsible, Accountable, Consulted, and Informed) matrix, [Accountability: Who Is Responsible?](#)

RAG (red/amber/green) assessment, [Organizational and Technical Readiness](#)

RAG (retrieval-augmented generation), [Multi-Agent Systems](#)

RDU (Responsible Data Use) model, [Vignette 1: DBS Group's AI Journey \(2025\)](#)

regulatory violations, [Regulatory and compliance violations \(compliance / liability risk\)](#)

reliability, [Multi-Agent Organizations, The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

research sector, [Research](#)

resilience, [Strategic Value Versus Tactical Automation](#)

resource management, [Agentified Applications and Software](#)

responsibility, [Accountability: Who Is Responsible?-Vignette: RACI "Second Layer" in Mortgage Lending](#)

Responsible Data Use (RDU) model, [Vignette 1: DBS Group's AI Journey \(2025\)](#)

Responsible, Accountable, Consulted, and Informed (RACI) matrix, [Accountability: Who Is Responsible?](#)

retail sector, [Retail-Telco](#)

retrieval-augmented generation (RAG), [Multi-Agent Systems](#)

return on investment (see ROI)

risk thresholds, [Accountability: Who Is Responsible?](#)

risks (see also security issues)

- compliance risk, [Governance, Explainability, Logging, and Compliance-Vignette: Autodesk's Continuous Compliance Agents in Design Tools](#)

- manifestations of, Evaluating ROI and Risks, Unsafe actions and overbroad autonomy (security / operational risk)
- mapping risk landscape, Mapping the Risk Landscape-IP leakage and loss of confidentiality (IP / trade secret risk)
- mitigating, Agentified Applications and Software, Vignette: RACI “Second Layer” in Mortgage Lending
- risk-return equation, Evaluating ROI and Risks
- of scaling, Scaling Without Lock-In
- types of, Evaluating ROI and Risks

robotaxis, The Spectrum of Trust: From Blind Faith to Calibrated Confidence

robotics technology, Future

ROI (return on investment)

- alpha- versus beta-level ROI, Evaluating ROI and Risks
- difficulties measuring for agentic AI, Evaluating ROI and Risks
- frameworks for assessing, Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI-Vignette 3: Siemens and Its AI Agents (2025)
- key takeaways on, Key Takeaways on Value and ROI from These Three Examples of Agentic Systems

rollbacks, 3. Future-Proofing Architectures for Upgrades

rollouts, versus pilots, Early Pilots Versus Enterprise Rollouts in a Platform World (see also scaling)

S&P Global, [1. Encapsulation and Coordination](#)

safe-abort switches, [Pillar 1: Technical guardrails and control](#)

safeguard agents, [Agentified Applications and Software](#), [Agentified Applications and Software](#), [Multi-Agent Organizations](#)

safety, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

scaling (see also system design)

- centralized ownership versus empowerment, [4. Centralized Ownership Versus Empowerment](#)
- encapsulation and coordination, [1. Encapsulation and Coordination-1. Encapsulation and Coordination](#)
- expandability, MCP, and agent-to-agent, [5. Expandability, Model Context Protocol, and Agent-to-Agent-5. Expandability, Model Context Protocol, and Agent-to-Agent](#)
- future-proofing architectures for upgrades, [3. Future-Proofing Architectures for Upgrades](#)
- LLM/cloud agnosticism, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)-2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)
- platform effect and, [Early Pilots Versus Enterprise Rollouts in a Platform World](#)
- without lock-in, [Scaling Without Lock-In](#), [5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

scaling laws, [Engineering AI](#)

scope of judgement, [Accountability: Who Is Responsible?](#)

security issues (see also risks)

- agent identity, [Designing Agentic Systems](#)

- OpenClaw incidents, [Choice of Tools](#), [Mapping the Risk Landscape](#)
- privacy breaches, [Mapping the Risk Landscape](#), [Data exfiltration and privacy breaches \(data / privacy risk\)](#)
- security exploits, [Mapping the Risk Landscape](#)
- trustworthy systems, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

separation of concerns, [3. Future-Proofing Architectures for Upgrades](#)

shared infrastructure, [Pillar 3: Governance, process, and transparency](#)

Siemens, [Vignette 3: Siemens and Its AI Agents \(2025\)](#), [Vignette: Siemens's Orchestrated Industrial Agents as Layered Trust](#)

silos, breaking, [Agentified Applications and Software](#)

skeptical experts, [Practical Segments and How to Address Them](#)

skill, expanding, [5. Expandability, Model Context Protocol, and Agent-to-Agent](#)

sly_data (Structured Layered Exchange Data), [Agentified Applications and Software](#)

software development, [Code Review: Agentic AI and the Economics of Engineering-Code Review: Agentic AI and the Economics of Engineering, Agentic Software Development Life Cycle-Code and App Migration](#)

specialization, [Agentified Applications and Software](#)

speed, [Pitfalls](#)

strategic risk, [Evaluating ROI and Risks](#)

Structured Layered Exchange Data (sly_data), [Agentified Applications and Software](#)

system design (see also [AI agents](#); [multi-agent systems](#); [scaling](#))

- agentic user experience, [The Agentic User Experience](#)
- autonomous versus human-in-the-loop tasks, [Design Considerations](#)
- basic requirements, [Designing Agentic Systems](#)
- core design choices, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)
- potential pitfalls, [Pitfalls](#)
- technical elements related to trust, [Trust](#)
- testing multi-agent systems, [Testing Multi-Agent Systems](#)
- trust, [Trust](#)

system prompts, [The Age of Generative AI](#)

T

TCO (see total cost of ownership)

technical debt, [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#)

technical foundations

- agent workflows, [Turn-Taking Versus Continuous Versus Triggered Agents, Memory and Planning](#)
- agentic protocols, [Where to Start?](#)
- agentifying a business, [Where to Start?](#)
- context engineering, [Context](#)
- coordination, [Coordination-Coordination](#)
- customizing agents, [Customizing Agents](#)
- grounding, [Grounding](#)

- memory and planning, [Memory and Planning](#)
- refinements and fine-tuning, [Refinements and Fine-Tuning](#)
- tool choice, [Choice of Tools](#), [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#), [5. Expandability](#), [Model Context Protocol](#), and [Agent-to-Agent](#)

technical mitigations, [Technical Mitigations in Agentic Systems-Implications for AI agents](#)

technical readiness, [Organizational and Technical Readiness](#)

telco (telecommunications) sector, [Telco-Finance](#)

testing, [Testing Multi-Agent Systems](#), [Pillar 1: Technical guardrails and control](#)

timeout mechanisms, [Pitfalls](#)

tools, choosing, [Choice of Tools](#), [2. Avoiding Vendor Lock-In \(LLM/Cloud Agnosticism\)](#), [5. Expandability](#), [Model Context Protocol](#), and [Agent-to-Agent](#)

total cost of ownership (TCO)

- frameworks for assessing, [Evaluating ROI and Risks-Vignette 3: Siemens and Its AI Agents \(2025\)](#)
- industrial AI stacks and, [Vignette 3: Siemens and Its AI Agents \(2025\)](#)
- justifying, [Vignette 3: Siemens and Its AI Agents \(2025\)](#)

trade secrets, [IP leakage and loss of confidentiality \(IP / trade secret risk\)](#)

transparency, [Pillar 3: Governance, process, and transparency](#)

triggering, [Turn-Taking Versus Continuous Versus Triggered Agents](#)

trust

- accountability, Accountability: Who Is Responsible?-Vignette: RACI “Second Layer” in Mortgage Lending
- building trustworthy systems, Building Trustworthy Systems
- earning, The Spectrum of Trust: From Blind Faith to Calibrated Confidence
- factors affecting, Culture, Adoption, Generational Shifts, and Agents Trusting Agents
- mapping risk landscape, Mapping the Risk Landscape-IP leakage and loss of confidentiality (IP / trade secret risk)
- orchestrators as control planes, Implications for AI Agents: Orchestrators as Accountable “Control Planes”
- orchestrators as layered trust, Vignette: Siemens’s Orchestrated Industrial Agents as Layered Trust
- population segments and, Practical Segments and How to Address Them-Vignette: Siemens’s Orchestrated Industrial Agents as Layered Trust
- role of governance in, Governance, Explainability, Logging, and Compliance-Vignette: Autodesk’s Continuous Compliance Agents in Design Tools
- spectrum of, The Spectrum of Trust: From Blind Faith to Calibrated Confidence-Vignette: GitHub Copilot Agents as Trust-Calibrated Assistants
- technical elements related to, Trust, Technical Mitigations in Agentic Systems-Implications for AI agents

Trusted AI program (Autodesk), Vignette: Autodesk’s Continuous Compliance Agents in Design Tools

U

uncertainty estimations, [Pillar 2: Human oversight and workflow](#)

unsafe actions, [Mapping the Risk Landscape](#), [Unsafe actions and overbroad autonomy \(security / operational risk\)](#)

upgrades, [Agentified Applications and Software](#), [IT](#), [3. Future-Proofing Architectures for Upgrades](#)

user experience (UX)

- best practices, [The Agentic User Experience](#)
- customized approaches to, [Practical Segments and How to Address Them](#)
- surfacing control and status, [Pillar 2: Human oversight and workflow](#)
- transparent, [Implications for AI agents](#)

V

value (see also [return on investment](#))

- ability to quantify, [Evaluating ROI and Risks](#)
- assessing, [Frameworks for Assessing Value and Total Cost of Ownership: How Leading Organizations Define and Measure ROI for Agentic AI-Key Takeaways on Value and ROI from These Three Examples of Agentic Systems](#)
- build-versus-buy decisions and, [Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI-Build-Versus-Buy Decisions in Agentic Architectures—and How Different They Are for Agentic AI](#)
- creating, [Agentic Systems: What Is the Payoff?](#), [Multiparty Negotiation](#)

- key takeaways on, [Key Takeaways on Value and ROI from These Three Examples of Agentic Systems](#)
- strategic versus tactical, [Strategic Value Versus Tactical Automation-Agentic Systems: What Is the Payoff?](#)

vendor lock-in, [Scaling Without Lock-In](#), 2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)-2. Avoiding Vendor Lock-In (LLM/Cloud Agnosticism)

versioning practices, [3. Future-Proofing Architectures for Upgrades](#)

vibe agentification, [The Road Ahead for Agentic Enterprises-The Road Ahead for Agentic Enterprises](#)

vibe coding systems, [Agentic Software Development Life Cycle](#), [The Road Ahead for Agentic Enterprises](#), [Preparing for the Next Wave](#)

vignettes (see also case studies/examples)

- Autodesk, [Vignette: Autodesk’s Continuous Compliance Agents in Design Tools](#)
- DBS Group, [Vignette 1: DBS Group’s AI Journey \(2025\)](#)
- Deviniti engineering, [Vignette: Deviniti’s Self-Hosted Agents for Regulated Clients-IP leakage and loss of confidentiality \(IP / trade secret risk\)](#)
- GitHub Copilot assistant, [Vignette: GitHub Copilot Agents as Trust-Calibrated Assistants](#)
- JPMorganChase, [Vignette 2: JPMorganChase \(2025\)](#)
- MIT’s DualFair, [Vignette: RACI “Second Layer” in Mortgage Lending](#)
- Siemens, [Vignette 3: Siemens and Its AI Agents \(2025\)](#), [Vignette: Siemens’s Orchestrated Industrial Agents as Layered Trust](#)

W

watsonx, [1. Encapsulation and Coordination](#)

World Wide Agentic Web, [The Road Ahead for Agentic Enterprises](#)

Z

Zoox's PRG framework, [The Spectrum of Trust: From Blind Faith to Calibrated Confidence](#)

About the Authors

Babak Hodjat is the chief AI officer at Cognizant, where he leads the Cognizant AI Lab (CAIL), a team of developers and researchers bringing advanced AI solutions to businesses. Babak is the former cofounder and CEO of Sentient, responsible for the core technology behind the world's largest distributed artificial intelligence system. Babak was also the cofounder of the world's first AI-driven hedge-fund, Sentient Investment Management. Babak is a serial entrepreneur, having started a number of Silicon Valley companies as main inventor and technologist. Prior to cofounding Sentient, Babak was senior director of engineering at Sybase iAnywhere, where he led mobile solutions engineering and AvantGo, the world's first ad-supported mobile web. Prior to Sybase, Babak was cofounder, CTO, and board member of Dejima Inc. Babak is the primary inventor of Dejima's patented, agent-oriented technology applied to intelligent interfaces for mobile and enterprise computing—the technology behind the original Apple Siri. Babak has published many papers in the fields of evolutionary computation, artificial life, agent-oriented software engineering, and distributed artificial intelligence, and he has 41 issued US AI patents to his name. Babak holds a PhD in artificial intelligence from Kyushu University, in Fukuoka, Japan. Babak is also the author of *The Narrator*, an AI science fiction novella, and *The Konar and the Apple*, a collection of short stories based on his youth.

Operating out of Dubai (UAE) and the United States, **Antoine Blondeau** is a visionary technology entrepreneur and investor. He is the cofounder and managing partner of **Alpha Intelligence Capital (AIC)**, a global venture capital firm investing in deep artificial intelligence technology companies worldwide. AIC's investments include OpenAI, Advanced Machine Intelligence, Higgsfield, Modular and Exostellar (both firms sold to Qualcomm), Adaption, Aidoc, Instadeep (sold to BioNTech), Advanced Navigation, Envelop Risk, ZeroEyes, Reaqta (sold to IBM), AImusic (sold to Apple), and GetVisibility (sold to Forcepoint).

Over the past 28 years, Antoine has dedicated himself to the development of artificial intelligence. He was the CEO of Dejima, the company that

powered DARPA's foundational CALO project that evolved into Apple's Siri. Antoine was also president and COO of Nasdaq-listed Zi Corporation, whose predictive text input software was embedded in hundreds of millions of devices. He was the cofounder and CEO of Sentient Technologies, a leading San Francisco Bay Area deep AI technology startup that pioneered massively scaled AI.

Antoine has held senior leadership positions at Good Technology, Salesforce, and Sybase. He has advised Pictet, KKR, and BITKRAFT Ventures. Antoine is also board chair of [Crypto.com](https://crypto.com), one of the world's largest blockchain and cryptocurrency full-service platforms.

In addition to his fund investments, Antoine has made over 40 personal investments in deep technology firms, including SpaceX, Starcloud, Epic Games, Jio Platforms, Boom Supersonic, Inworld, Alan, K Health, Aries Global, Feria Legacy, Noah, Tara Gaming, GoShare, KeyMe, DoubleDutch (acquired by Cvent), PeerIQ (acquired by Cross River Bank), GammaNow (acquired by Animoca Brands). He is a founding member and general partner of Zinc, a London-based technology startup incubator.

Antoine started to code at age 11. He is the author of four patents.

Colophon

The animal on the cover of *The Agentic Enterprise* is a great hornbill (*Buceros bicornis*). It is known by several other names, including great Indian hornbill, concave-casqued hornbill, and great pied hornbill. It is one of the largest members of the hornbill family, with a wingspan of up to five feet. The great hornbill is easily recognized by the large yellow bony structure on top of its head, known as a casque.

It is native to forested areas of South and Southeast Asia, with a range spanning from Nepal to the island of Sumatra. The great hornbill's diet consists mostly of fruit, particularly figs, though they are also known to eat small mammals, birds, reptiles, and insects. Female hornbills build their nests in the hollows of tree trunks and seal the opening using feces, food debris, and wood. They remain in the nest until their chicks are developed enough, relying on their mates to feed them through a small opening in the seal.

The great hornbill's conservation status is listed as vulnerable. It is primarily threatened by deforestation, though it is also hunted for its meat, feathers, and casques, which are used ceremonially by some tribal peoples. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by José Marzan Jr., based on an antique line engraving from *Meyers Kleines Lexicon*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.